

Commodity Price Intelligence & Market Risk Monitoring Platform

Forecasting • Anomaly Detection • Time Series Analytics •
Decision Support

Prepared By: Haaris Shaikh

2025-2026

Contents

- 1) Abstract
- 2) The Challenge of Agricultural Price Volatility in the Indian Context
- 3) Economic Implications for Farmers in Maharashtra
- 4) The Role of Price Forecasting in Mitigating Risk and Empowering Stakeholders
- 5) Research Objectives and Key Questions
- 6) Dataset Description
- 7) Data Preprocessing
- 8) Exploratory Data Analysis (EDA) Visualizations
- 9) Seasonal Decomposition
- 10) Anomaly Detection
- 11) Paradigms in Time Series Forecasting
 1. Statistical Forecasting Model
 2. Support Vector Regression (SVR)
 3. Random Forest and XGBoost
 4. Facebook's Prophet
- 12) Time Series Split Cross Validation
- 13) Performance Evaluation Metrics
- 14) Hyperparameter Tuning with Optuna
- 15) Model Comparison Results
- 16) Future Forecasting
- 17) Conclusion
- 18) Directions for Future Research

Abstract

Price swings for agricultural goods are a big problem in India, affecting both farmers and the stability of our food supply. This is especially true in a state like Maharashtra, where farming is so important. For crops like onions, which don't last long on the shelf, unpredictable prices can create a lot of financial risk for the people who grow them. If farmers had a better idea of what prices to expect, they could make smarter decisions about when to plant, harvest, and sell their crops. This project is all about building and testing different ways to forecast these prices.

I looked at two main types of forecasting models. The first type includes the classic statistical methods that have been used for a long time, like SARIMAX and ETS. The second type includes more modern machine learning models, like Support Vector Regression (SVR), Random Forest, XGBoost, and a model from Facebook called Prophet. To make sure I was getting the best performance out of these models, I used a tool called Optuna to automatically find the best settings for each one, rather than just using the default settings.

To test the models fairly, I used a special kind of cross-validation that respects the order of time, so I was always using past data to predict the future, just like in a real-world scenario. The main goal was to see which model could predict prices most accurately, which I measured using a metric called Root Mean Squared Error (RMSE). The best model turned out to be XGBoost. To understand *why* it was making its predictions, I used another tool called SHAP, which helped me look inside the "black box" and see which factors, like past prices or weather, were most important. I also used a technique called bootstrapping to figure out a range of likely prices, not just a single number, to show how much uncertainty there was in the forecast. The overall goal of this project was to find a reliable way to forecast prices that could one day be used to give farmers in Maharashtra real, actionable information to help them run their farms more profitably and with less risk.

The Challenge of Agricultural Price Volatility in the Indian Context

Agriculture is a huge part of India's economy. More than half of the country's workers are in this sector, and it's a big contributor to our national GDP. But farming in India is full of challenges, and one of the biggest is that prices for crops can change wildly and without warning. These price swings are a major headache for everyone, from the government trying to manage the economy to the farmers trying to make a living, and even for consumers who have to buy the food. When prices shoot up, it can be hard for people, especially those with lower incomes, to afford basic food. And while farmers might like high prices, when prices crash, it can be a disaster for them.

This problem is even worse for perishable goods like fruits and vegetables. Because they go bad quickly and we don't always have enough good storage, their prices are much more unstable than things like grains. There are a lot of reasons for this. The weather is a big one, since a bad monsoon can ruin a harvest. Problems with the supply chain, like not having enough refrigerated trucks or warehouses, mean a lot of produce gets wasted before it even gets to market. What's happening in global markets, the cost of things like fertilizer, and government policies all play a role too. All these factors mix together to create a really unpredictable market where small problems can cause huge price swings.

Economic Implications for Farmers in Maharashtra

Maharashtra is one of India's most important agricultural states, producing a lot of key crops. But even here, many farmers are in a tough financial spot, and the constant up-and-down of market prices makes things much harder. A lot of farmers, especially those with small farms, have trouble getting loans from banks because they don't have much to offer as collateral. This pushes them toward informal lenders who charge very high interest rates, trapping them in a cycle of debt.

This is where price volatility really hurts. When a farmer can't be sure what price they'll get for their onions or tomatoes, it's almost impossible to plan. How do you decide what to plant, or how much to spend on fertilizer, if you have no idea what your income will be? This uncertainty makes it hard to invest in the farm, like buying better equipment or improving irrigation. It keeps farmers stuck in a cycle of low productivity and financial stress, which isn't good for the long-term health of farming in the state.

The Role of Price Forecasting in Mitigating Risk and Empowering Stakeholders

If we could give farmers a good idea of where prices are headed, it would be a game-changer. Accurate price forecasting is a powerful tool that can help reduce some of this risk. It gives farmers market intelligence, which means they can move from just accepting whatever price they're offered to making strategic decisions. If forecasts show that prices are likely to go up in a few weeks, a farmer with some storage might decide to wait to sell. If forecasts show a certain crop is likely to have a price crash, they might decide to plant something else. By giving them a clearer picture of their potential income, forecasting can help make farming more profitable and stable.

And it's not just farmers who would benefit. Policymakers could use these forecasts to get ahead of food inflation or to plan when to import or export certain goods. Traders and food processors could use them to manage their inventory better. What's really needed is a kind of "market intelligence system" that can track what's happening in the markets and give everyone a heads-up about what's coming. This project is a step in that direction, by trying to build and test the kind of forecasting models that would be at the heart of such a system.

Research Objectives and Key Questions

The main goal of our project was to build and test a bunch of different models for forecasting the wholesale price of onions in some of the big markets in Maharashtra. I wanted to see which ones worked best and why. To do that, I focused on a few key questions:

1. **Which type of model is better for this job: the old-school statistical models or the newer machine learning ones?** I wanted to see if the flexibility of machine learning gives it a real advantage when dealing with messy, unpredictable data like crop prices.
2. **Can I make the models even better by using an automatic tuning tool like Optuna?** Instead of just using the default settings, I wanted to see how much of a difference it makes to really search for the best possible configuration for each model.

3. **What are the most important things that drive onion prices?** If I could build a really accurate "black-box" model, could I use a tool like SHAP to look inside it and understand what it learned about the market?
4. **How certain can we be about these forecasts?** A single number isn't always that useful. I wanted to find a way to provide a range of likely prices to give a better sense of the possible risks and opportunities.

Dataset Description

The dataset used in this project, is a structured agricultural market dataset that captures daily records of commodity transactions along with associated weather conditions. Each row in the dataset corresponds to a single observation for a given commodity in a particular market on a specific date. This makes it a panel time series dataset, where the temporal dimension is daily, and the cross-section all dimensions are defined by markets and commodities.

The dataset is composed of both economic and environmental variables.

Variable	Type	Description
Arrival_Date	Date	Date of market observation (daily frequency).
Market	Categorical	Name of the wholesale market/mandi.
Commodity	Categorical	Name of the agricultural commodity (e.g., Onion, Wheat, Tomato).
Modal_Price	Numeric	Modal price (Rs/Quintal). Chosen as the forecasting target .
Min_Price	Numeric	Minimum recorded price in that market on that day (Rs/Quintal).
Max_Price	Numeric	Maximum recorded price in that market on that day (Rs/Quintal).
Avg_Temp_C	Numeric	Average daily temperature (°C).
Min_Temp_C	Numeric	Minimum daily temperature (°C).
Max_Temp_C	Numeric	Maximum daily temperature (°C).
Rainfall_mm	Numeric	Daily rainfall (millimeters).

- The modal price represents the most frequently reported price for a commodity in a market on a given day. It is less sensitive to extreme values than the minimum or

maximum and is therefore considered a more robust representation of the prevailing market rate. In addition to prices, the dataset also records the arrival quantity of each commodity, which measures the physical inflow of produce into the market, expressed in quintals. This quantity provides important insights into the supply side of the market and can explain some of the short-term fluctuations in price. However, the arrivals data often suffers from missingness and inconsistent reporting across markets.

- From an exploratory perspective, the dataset reveals several important characteristics. The time coverage is daily, spanning multiple years of observations, but the distribution of records is not uniform across commodities and markets. Some commodities, such as onions, tomatoes, and wheat, are reported with dense daily coverage, while others appear more sparsely. Larger, urban markets generally have more complete and consistent reporting, whereas smaller rural markets often exhibit gaps. These gaps are especially pronounced in the arrivals and weather data. Missing values in rainfall and temperature are common in some regions, likely due to limitations in local monitoring infrastructure. Such structured missingness requires careful handling, as it can otherwise bias results toward better-monitored markets.

Overall, the dataset provides a rich foundation for building forecasting models. It combines historical price records with weather indicators, allowing us to model both internal market dynamics and external environmental drivers. At the same time, its irregular coverage, missing values, and outliers reflect the real-world complexity of agricultural data, making preprocessing and robust modeling essential steps in this project.

Data Preprocessing

Raw agricultural market data, while rich in content, is rarely ready to be used directly in modeling. In our project, we encountered several issues that required systematic preprocessing. The most important among these were missing values in the weather variables, irregular coverage across markets and commodities, and the presence of outliers and anomalies in the price series. Without addressing these issues, any forecasting model would risk being biased, unstable, or misled by noise. Preprocessing was therefore a crucial foundation for ensuring the reliability of our pipeline. One of the main challenges in the dataset was the incomplete coverage of weather data. Variables such as average temperature, minimum and maximum temperature, and rainfall showed frequent gaps, especially in smaller markets. These missing values were not random; they followed a structured pattern, depending on the location of the market and the reliability of local meteorological monitoring. To address this, we implemented a hierarchical imputation strategy that progressively filled missing values in a way that preserved both the local characteristics of each market and the seasonal cycles observed in the data.

```
weather_cols = ['Avg_Temp_C', 'Min_Temp_C', 'Max_Temp_C', 'Rainfall_mm']
```

```
df = df.sort_values(by=['Market', 'Arrival_Date'])
```

```
# Convert 'Arrival_Date' to datetime
```

```
df['Arrival_Date'] = pd.to_datetime(df['Arrival_Date'])
```

```

# Market-wise forward and backward fill using transform
for col in weather_cols:
    df[col] = df.groupby('Market')[col].transform(lambda x: x.ffill().bfill())

# Fill remaining missing values using market-wise mean
for col in weather_cols:
    df[col] = df.groupby('Market')[col].transform(lambda x: x.fillna(x.mean()))

# Final backup using market-month average (for seasonal effect)
df['Month'] = df['Arrival_Date'].dt.month
for col in weather_cols:
    df[col] = df.groupby(['Market', 'Month'])[col].transform(lambda x: x.fillna(x.mean()))

```

- The first step in this strategy was market-wise forward filling and backward filling. When data was missing for a few consecutive days, we filled those gaps using the most recent available values from the same market. This method assumes that short gaps in weather reporting are best approximated by the conditions immediately before or after the missing period. It is particularly appropriate for temperature, where day-to-day changes are generally gradual rather than abrupt.
- For longer gaps, forward and backward filling alone was insufficient, so we introduced a second layer of imputation based on market-wise averages. For each market, we computed the mean values of each weather variable and used these to replace missing observations. This approach preserved market-specific climate characteristics; for example, a coastal market typically experiences different average temperatures and rainfall compared to an inland market, and our method ensured that such distinctions were maintained.
- Despite these two steps, some missing values still remained, particularly in cases where entire blocks of data were absent for certain months. To address this, we applied a seasonal fallback strategy based on market-month means. For each market and calendar month, we computed the average of historical values for that variable and used this to fill any remaining gaps. This step was especially valuable for rainfall, which follows strong seasonal patterns linked to the monsoon. By relying on historical seasonal averages, we were able to preserve the typical rainfall cycle even in periods where recent observations were incomplete. The combination of forward and backward filling, market-specific averages, and market-month seasonal means created a robust imputation strategy that minimized bias while maintaining realistic weather patterns.

Minimum Observation Threshold:

After handling missing weather data, we turned our attention to the price series itself. The dataset included daily observations, but not all market–commodity pairs had consistent coverage. Some commodities in certain markets had only a few months of observations, while others had several years

of daily records. For forecasting purposes, we required a minimum of one full year of data, which corresponds to at least three hundred and sixty-five daily observations. This threshold was necessary to allow models to capture seasonal patterns as well as year-to-year fluctuations. Market–commodity pairs with fewer observations were excluded from modeling to ensure that the models were trained only on sufficiently rich time series.

Feature Engineering

To prepare the data for machine learning models, we engineered additional features from the modal price series.

- We created lagged variables representing the price one day ago, one week ago, two weeks ago, and up to twenty-eight days ago. These lag features provided the models with memory of past values, allowing them to capture autoregressive dependencies.
- We also computed rolling statistics, such as a seven-day rolling mean of the modal price, which smoothed out daily noise and represented short-term trends.
- Finally, the weather variables that had been imputed were added as exogenous regressors, ensuring that the models could account for external climate drivers alongside internal price dynamics.

Check For Stationarity:

For statistical models such as ARIMA and ETS, stationarity was a prerequisite. Stationarity means that the statistical properties of the time series, such as mean and variance, remain constant over time. Many agricultural price series, however, exhibit trends and seasonality, making them non-stationary in their raw form. To check for stationarity, we applied the Augmented Dickey–Fuller test. If the test indicated non-stationarity, we applied differencing to the series until stationarity was achieved. This transformation ensured that the assumptions of ARIMA and ETS were met, improving their forecasting performance.

The result of preprocessing was a clean, consistent, and feature-enriched dataset that was ready for modelling. Each market–commodity pair was represented as a continuous daily time series with complete weather variables, a sufficient historical span, lagged and rolling price features, calendar effects, and stationarity-adjusted values where necessary. This preprocessing step transformed a raw and irregular dataset into a structured and reliable foundation for building forecasts.

Exploratory Data Analysis (EDA) Visualizations

The exploratory analysis of the dataset was an essential step in understanding the structure and behavior of agricultural commodity prices before applying forecasting models. Visualizing the data revealed patterns that could not be detected from descriptive statistics alone and provided insights into the relationships between market dynamics, seasonal effects, and weather conditions.

- When examining price trends over time, we observed that different commodities exhibited very different behaviors. Vegetables such as onions and tomatoes were highly volatile, with frequent and sharp fluctuations in modal prices. These spikes were often linked to harvest

cycles or unexpected supply disruptions, which are common for perishable crops. In contrast, cereals like wheat and rice displayed relatively smoother patterns, with gradual changes that were more aligned with broader seasonal and policy-driven cycles. This distinction highlighted the importance of tailoring forecasting methods to each commodity, since methods that perform well on stable series may not capture the abrupt jumps characteristic of more volatile crops.

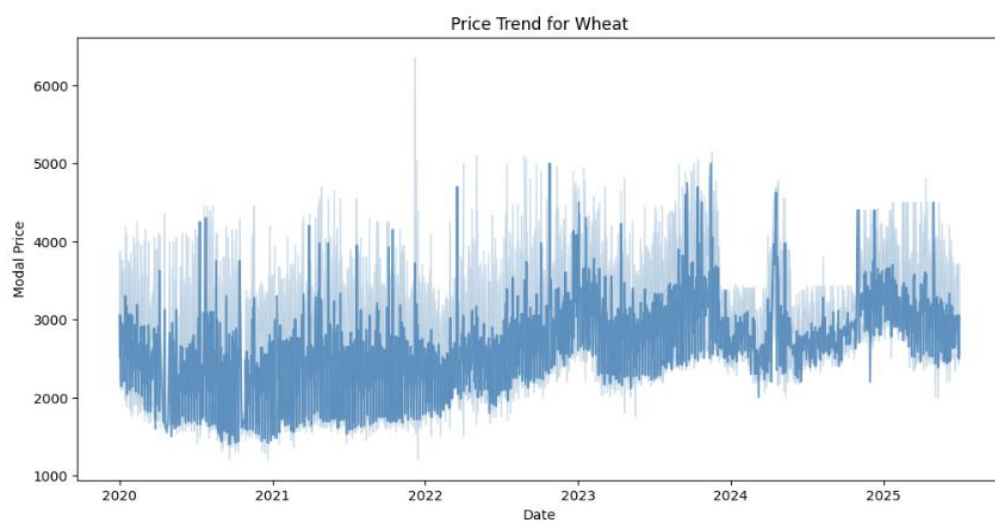
Price Trend:

```
def plot_price_trend(commodity):
    temp = df_eda[df_eda["Commodity"] == commodity]

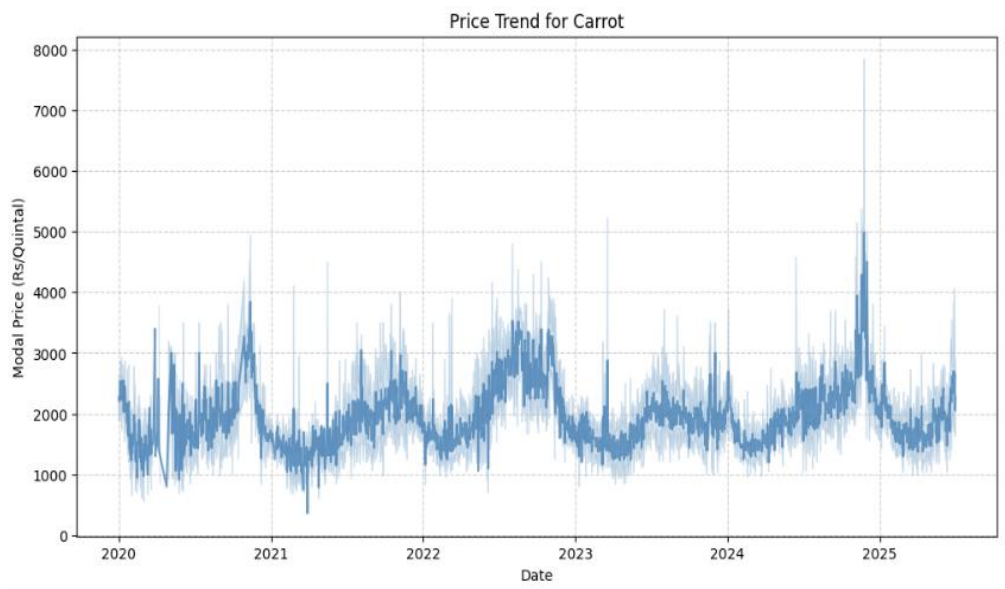
    # Check if there's any data to plot
    if temp.empty:
        print(f"No data available for {commodity}.")
        return

    plt.figure(figsize=(12, 6))
    sns.lineplot(x="Arrival_Date", y="Modal_Price", data=temp, alpha=0.7)
    plt.title(f"Price Trend for {commodity}")
    plt.xlabel("Date")
    plt.ylabel("Modal Price (Rs/Quintal)")
    plt.grid(True, linestyle='--', alpha=0.6)
    plt.show()

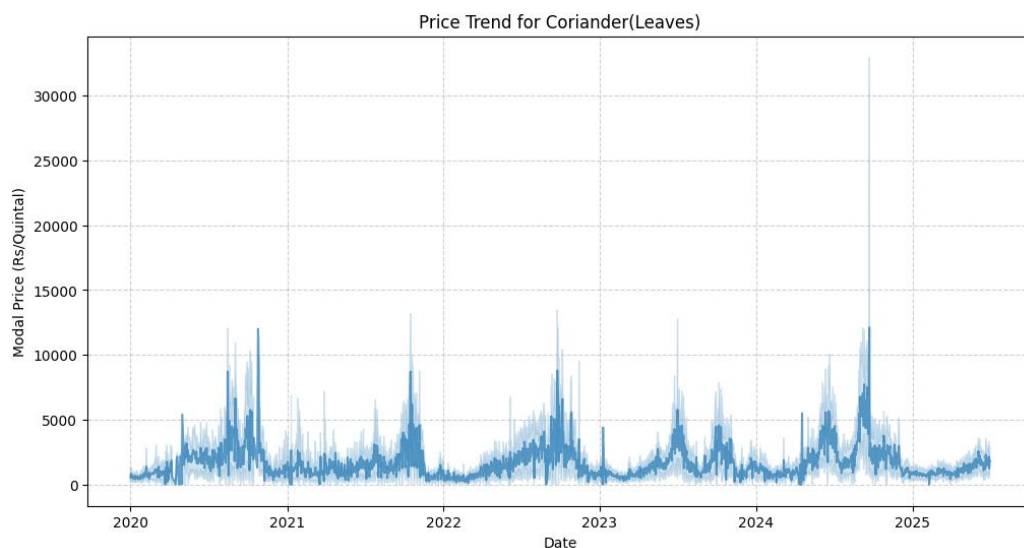
# 3. Call interact and pass the widget you already created
interact(plot_price_trend, commodity=commodity_dropdown);
```



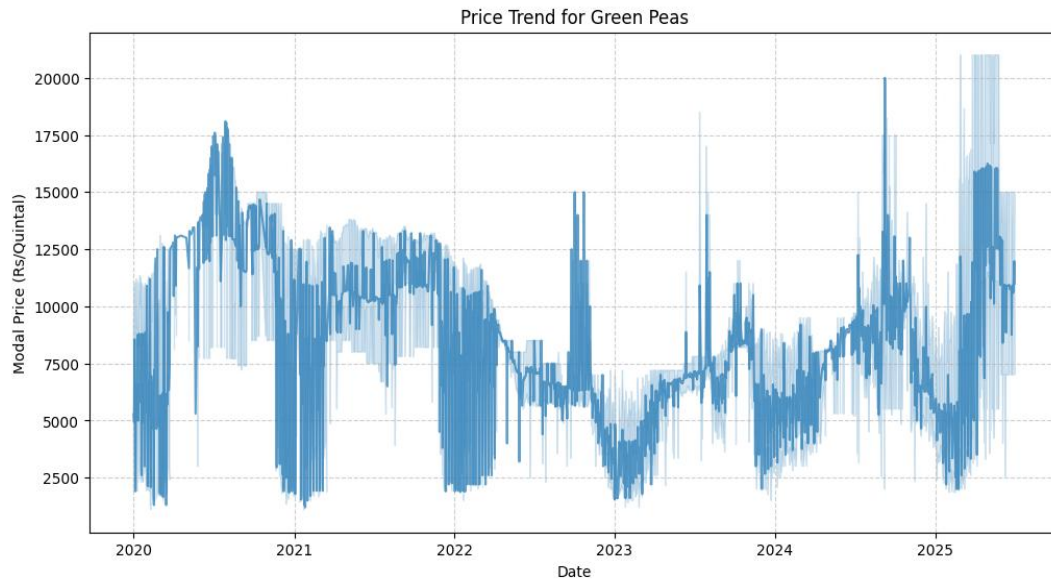
- This plot shows how the **modal price of wheat** changed from 2020 to 2025. Prices fluctuate heavily with clear ups and downs, some sharp spikes, and seasonal patterns—showing why forecasting is needed to help farmers plan better.



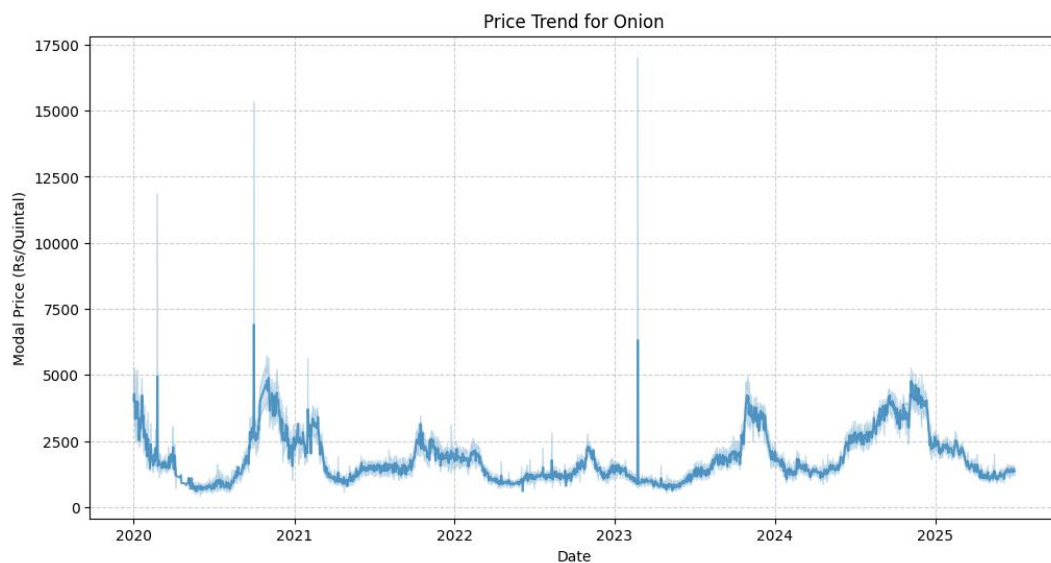
- This plot shows the **modal price trend of carrots** from 2020 to 2025. Prices rise and fall in clear seasonal cycles, with sharp spikes during certain periods, reflecting supply–demand fluctuations. Overall, it highlights strong seasonality and volatility, making price forecasting very useful for planning.



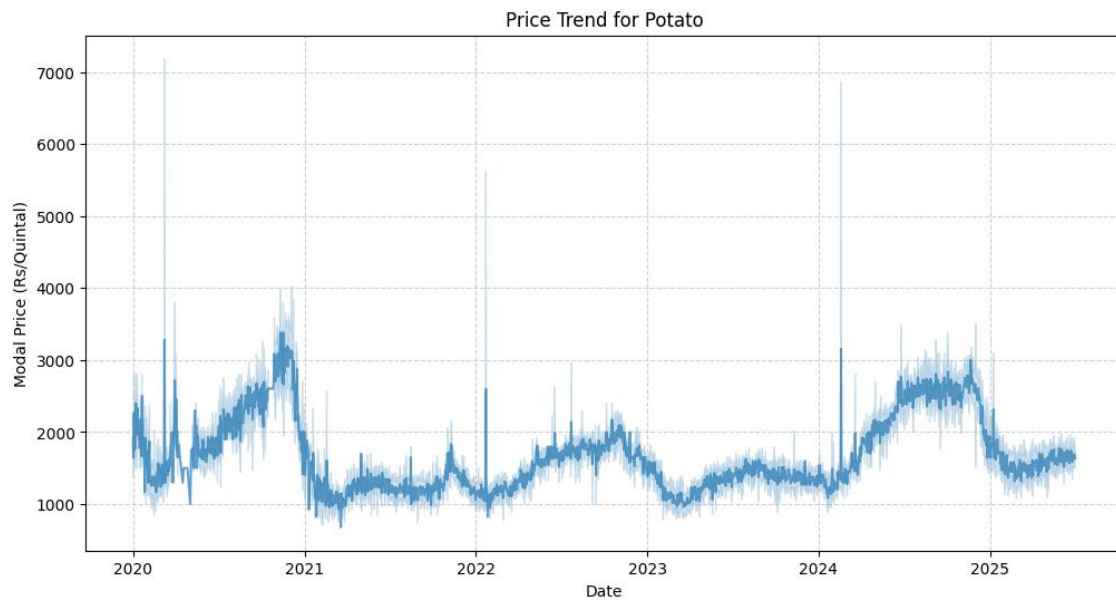
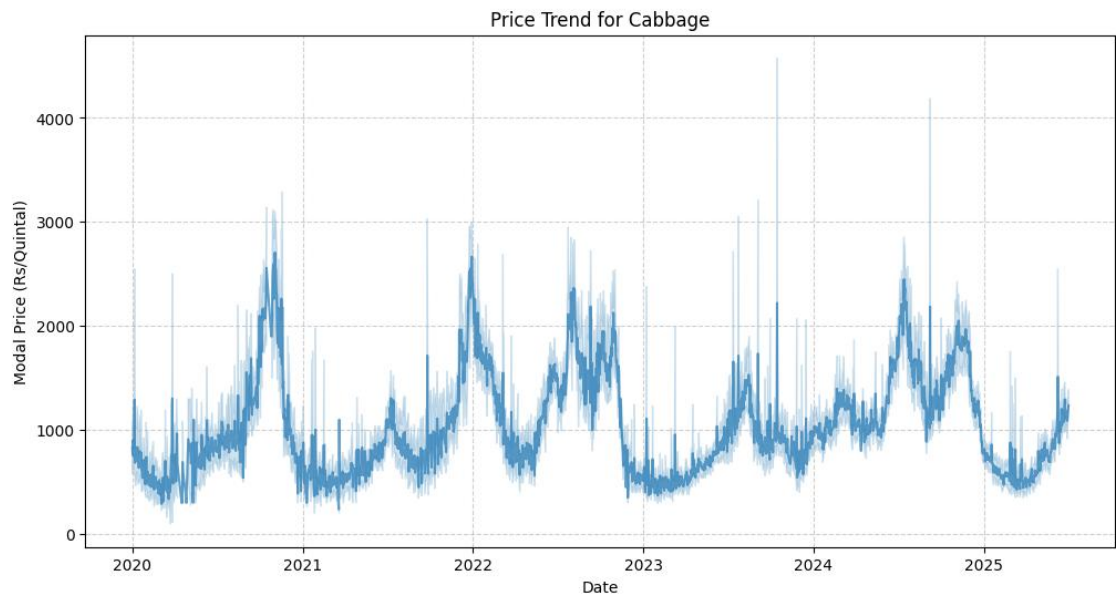
- This plot shows the **modal price trend of coriander leaves** from 2020 to 2025. Prices are highly volatile with frequent sharp spikes, sometimes crossing very high levels, reflecting perishability and demand shocks. The strong fluctuations highlight why reliable forecasting is crucial for such commodities.



- This chart shows the **modal price trend of green peas** from 2020 to 2025. Prices display sharp volatility, with peaks above ₹20,000/quintal during certain seasons and significant drops in others. The strong ups and downs reflect seasonal availability and market shocks, highlighting the need for accurate forecasting.



- This plot shows the **modal price trend of onions** from 2020 to 2025. Prices generally stay in the lower range but experience sudden sharp spikes, sometimes reaching above ₹15,000/quintal. These extreme fluctuations are due to seasonal cycles, supply shortages, and market shocks—making onion one of the most volatile commodities.



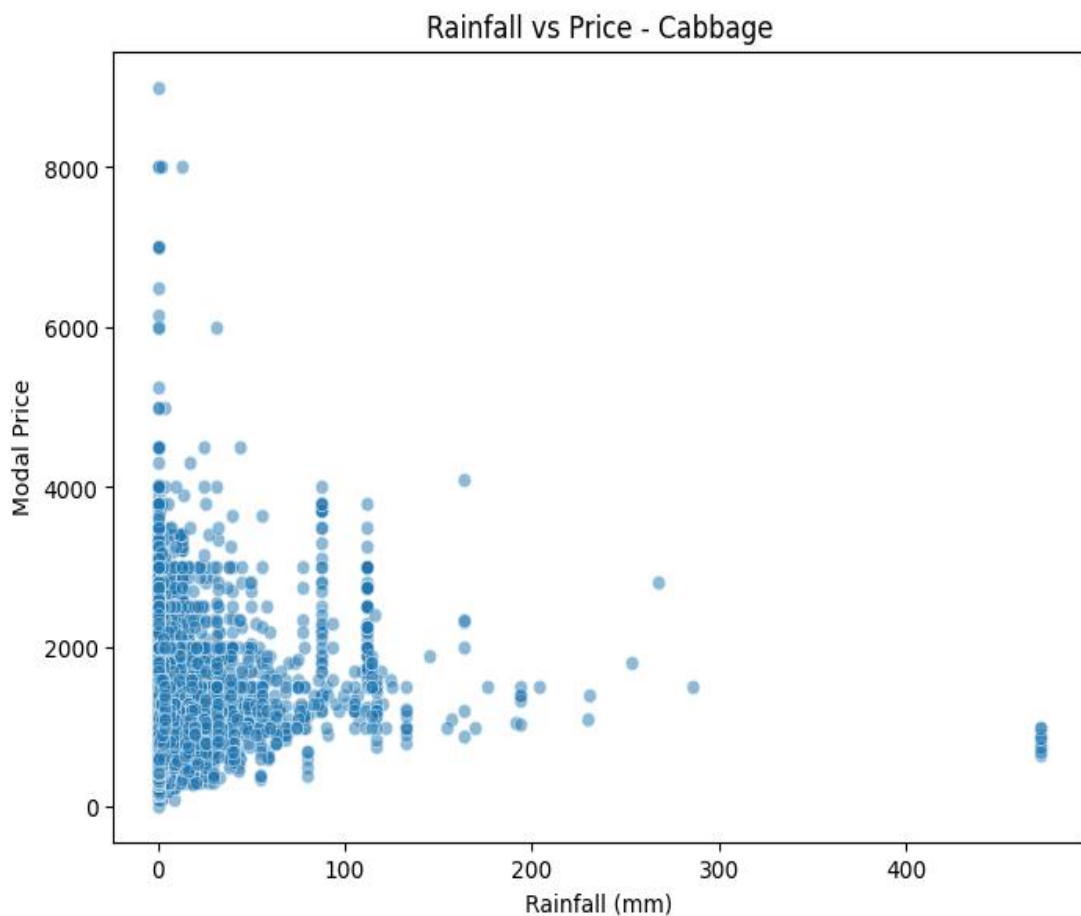
Rainfall vs Price:

```

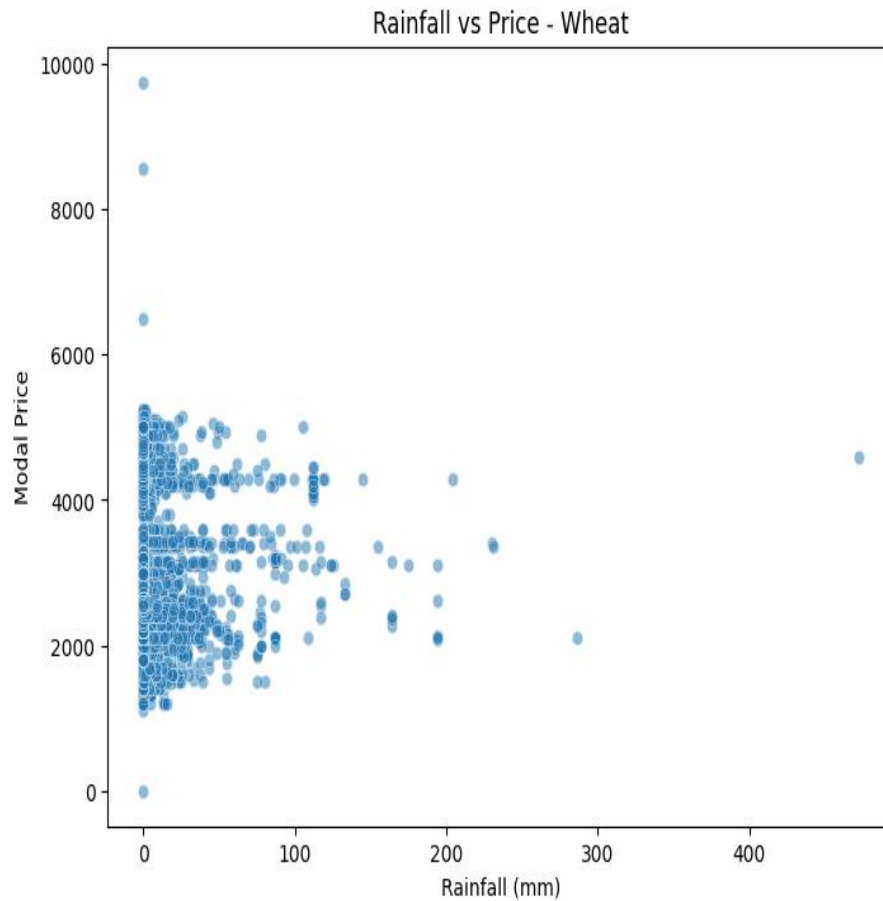
commodity_dropdown = Dropdown(
    options=sorted(df_eda["Commodity"].unique().tolist()),
    description="Commodity:",
    value=sorted(df_eda["Commodity"].unique().tolist())[1],
    style={'description_width': 'initial'},
    layout={'width': '300px'})
)
def plot_rainfall_vs_price(commodity):
    temp = df_eda[df_eda["Commodity"] == commodity]
    plt.figure(figsize=(8,6))
    sns.scatterplot(x="Rainfall_mm", y="Modal_Price", data=temp, alpha=0.5)
    plt.title(f"Rainfall vs Price - {commodity}")
    plt.xlabel("Rainfall (mm)")
    plt.ylabel("Modal Price")
    plt.show()

interact(plot_rainfall_vs_price, commodity=Dropdown(options=df_eda["Commodity"].unique(), description="Commodity:"))

```



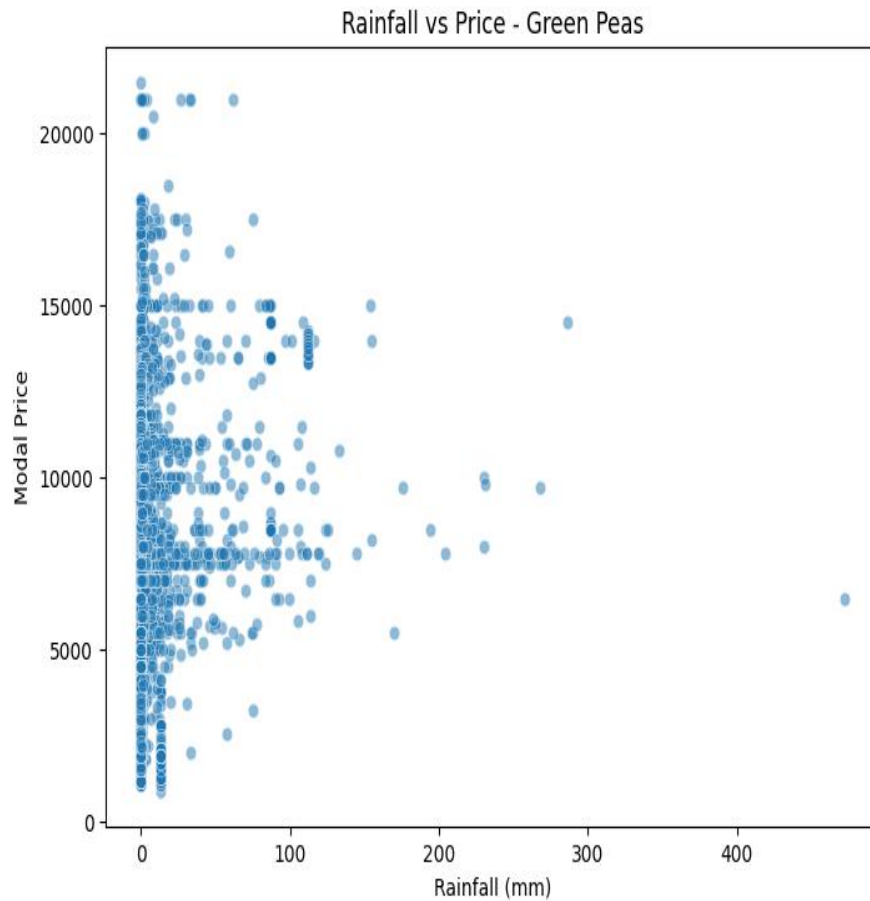
- The relationship between rainfall and price was another important pattern revealed by visualizations. Scatterplots comparing daily rainfall to modal prices suggested that moderate rainfall often coincided with lower prices, which is consistent with improved crop yields during favorable conditions. However, extreme rainfall events told a different story. Heavy rains, particularly during monsoon months, often corresponded with temporary price increases rather than decreases. This was likely due to disruptions in transportation and storage, which reduced the effective supply reaching urban markets. Thus, rainfall had a nonlinear impact on price: modest amounts reduced prices through higher yields, while excessive rainfall increased prices through logistical bottlenecks.



This scatter plot shows the relationship between rainfall (mm) and modal price of wheat.

- Most data points are clustered at low rainfall (0–50 mm), suggesting wheat prices are mostly observed when rainfall is limited.
- Prices range widely (₹1000–₹5000) regardless of rainfall, meaning no strong direct correlation is visible.
- A few extreme points (very high prices or very high rainfall) appear as outliers.

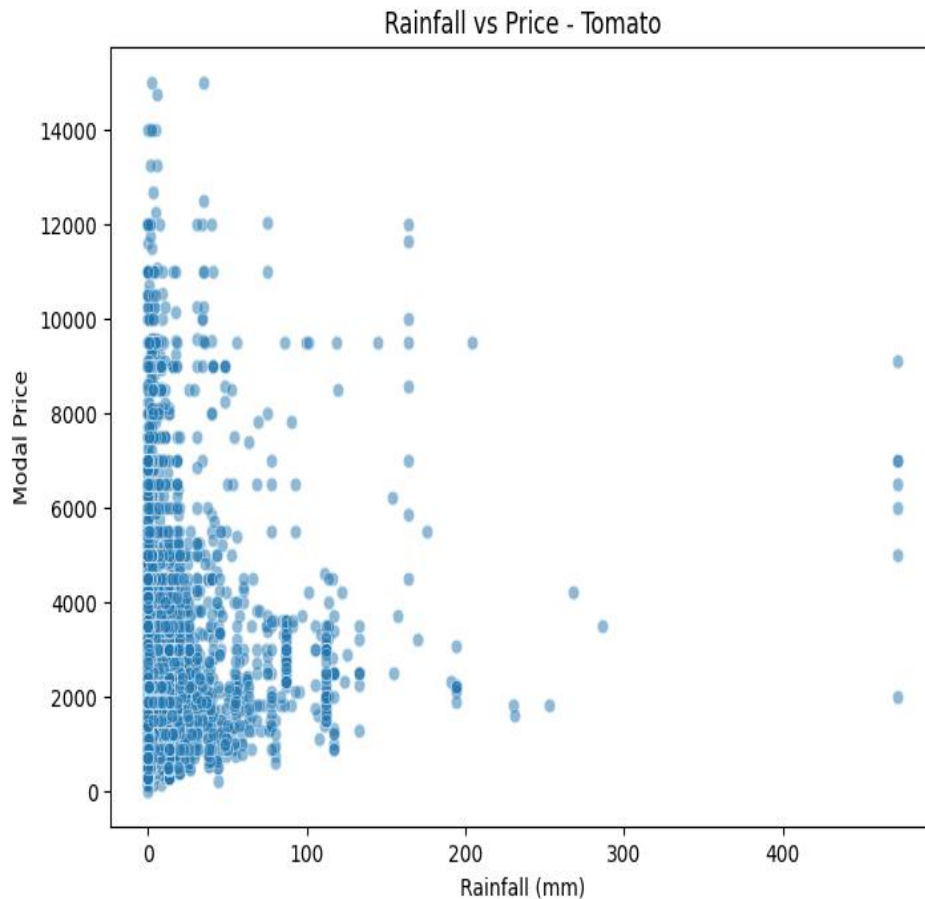
Summary: Rainfall alone does not strongly determine wheat prices, but it may contribute indirectly when combined with other factors like supply, demand, and seasonality.



This scatter plot shows the link between **rainfall (mm)** and the **modal price of green peas**.

- Most points are concentrated around **low rainfall (0–50 mm)**, where prices vary widely from ₹2,000 up to ₹20,000+.
- There isn't a clear linear relationship—prices can be high or low regardless of rainfall.
- A few outliers at higher rainfall values exist but are less common.

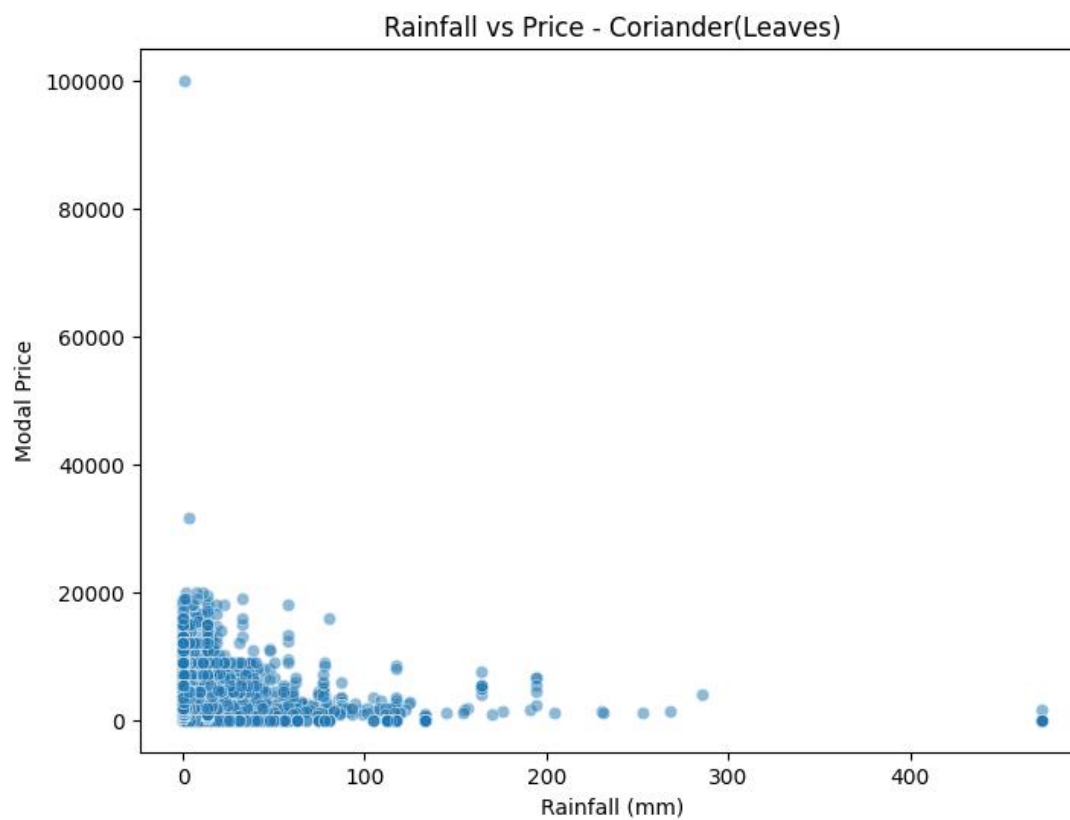
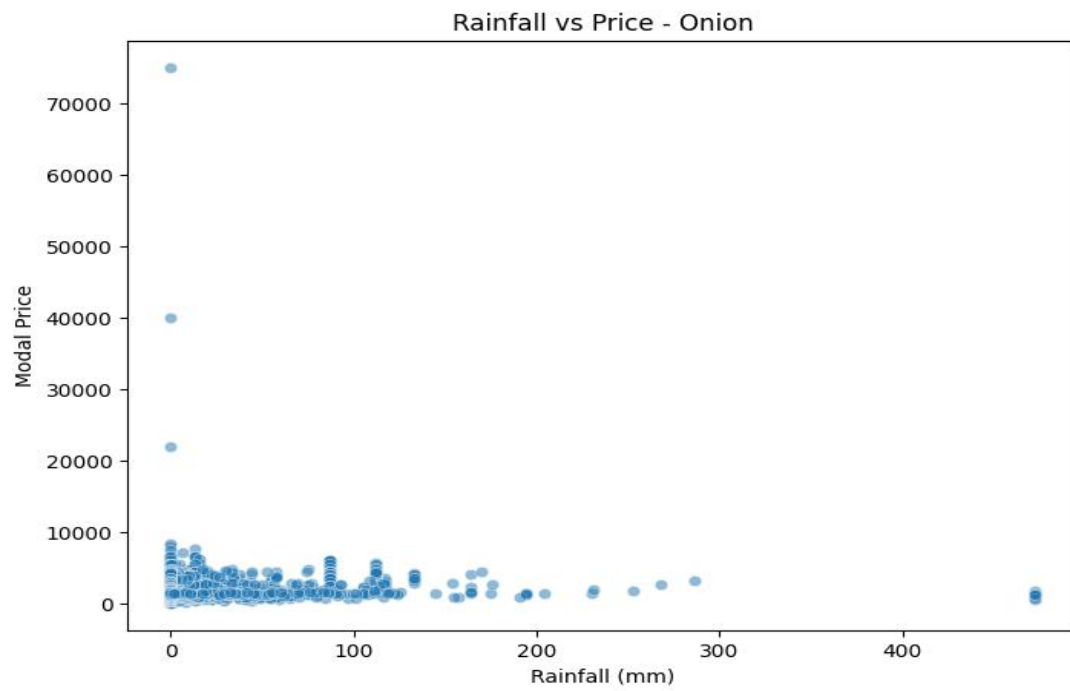
Summary: Green pea prices are highly volatile and not directly dependent on rainfall alone, though rainfall combined with other seasonal factors may still influence supply and price.

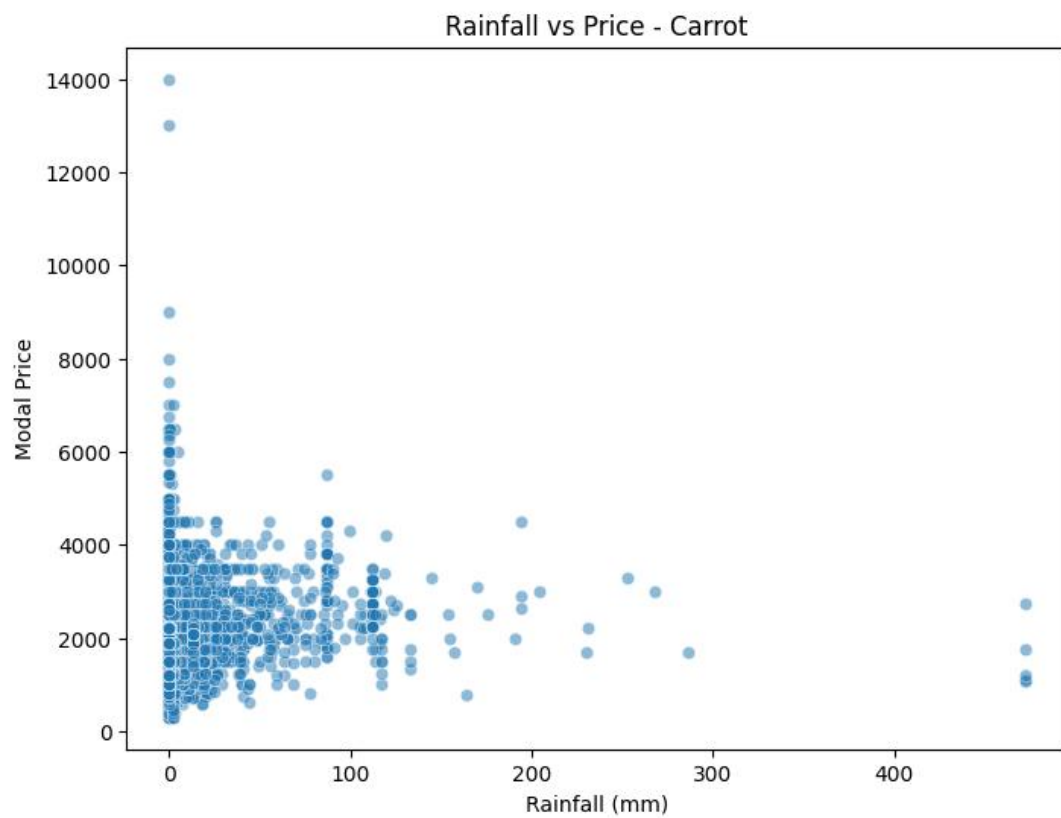
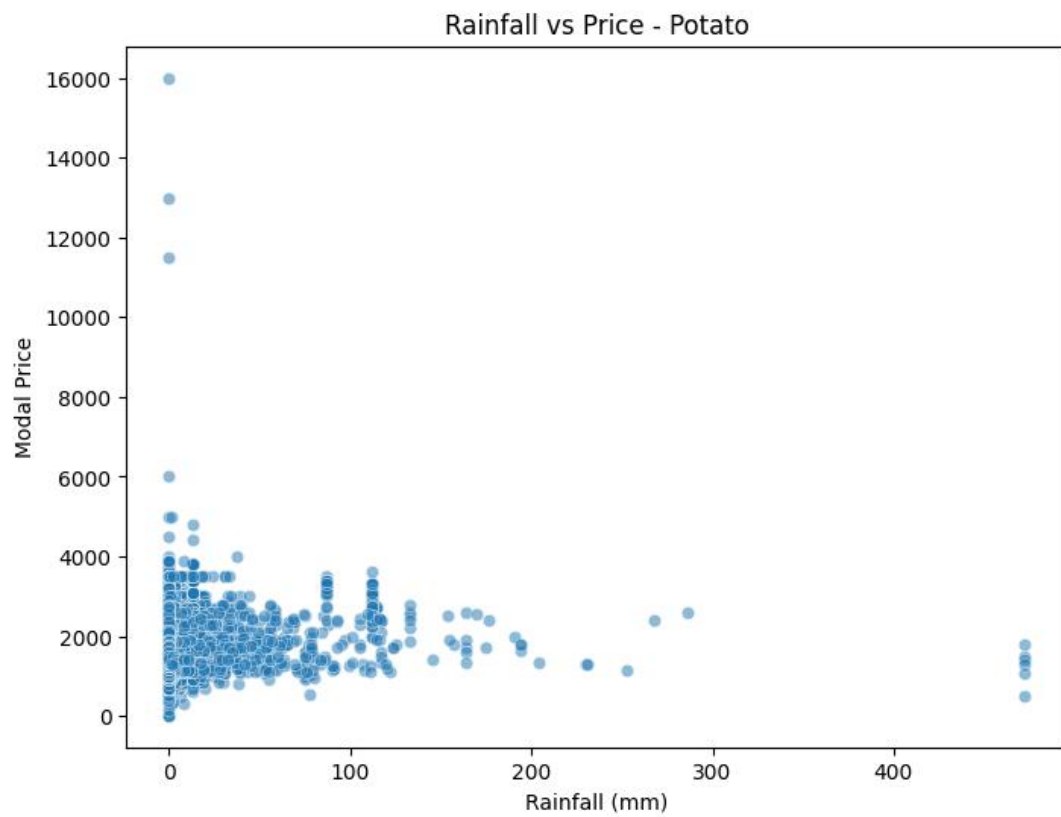


This scatter plot shows the relationship between **rainfall (mm)** and **modal price of tomatoes**.

- Most points are concentrated at **low rainfall levels (0–50 mm)**, where prices vary widely between ₹500 and ₹10,000+.
- Prices don't follow a clear trend with rainfall—sometimes high rainfall still corresponds to low prices and vice versa.
- Outliers exist at very high rainfall, but they are rare.

Summary: Tomato prices are highly scattered and **not directly controlled by rainfall alone**; instead, they depend on multiple factors like supply chain, seasonality, and market demand.





Market Distribution:

```
def plot_market_distribution(commodity):
    temp = df_eda[df_eda["Commodity"] == commodity]
    plt.figure(figsize=(12,6))
    sns.boxplot(x="Market", y="Modal_Price", data=temp)
    plt.xticks(rotation=90)
    plt.title(f"Price Distribution by Market - {commodity}")
    plt.show()

interact([plot_market_distribution, commodity=Dropdown(options=df_eda["Commodity"].unique(), description="Commodity:")])
```

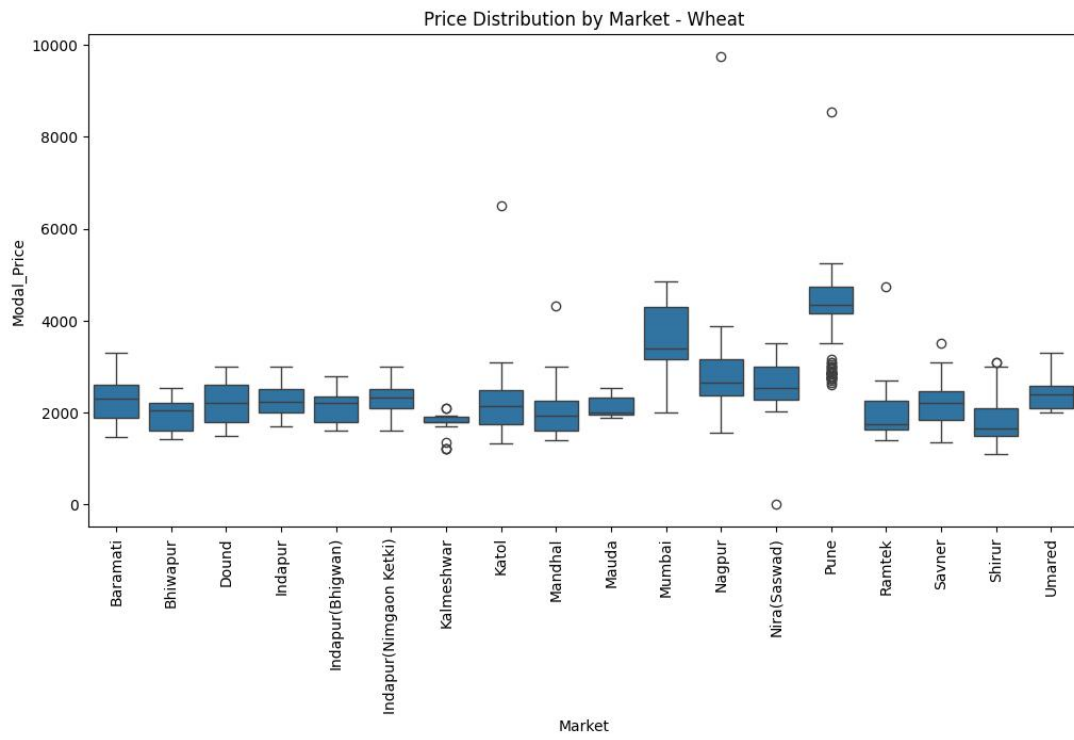
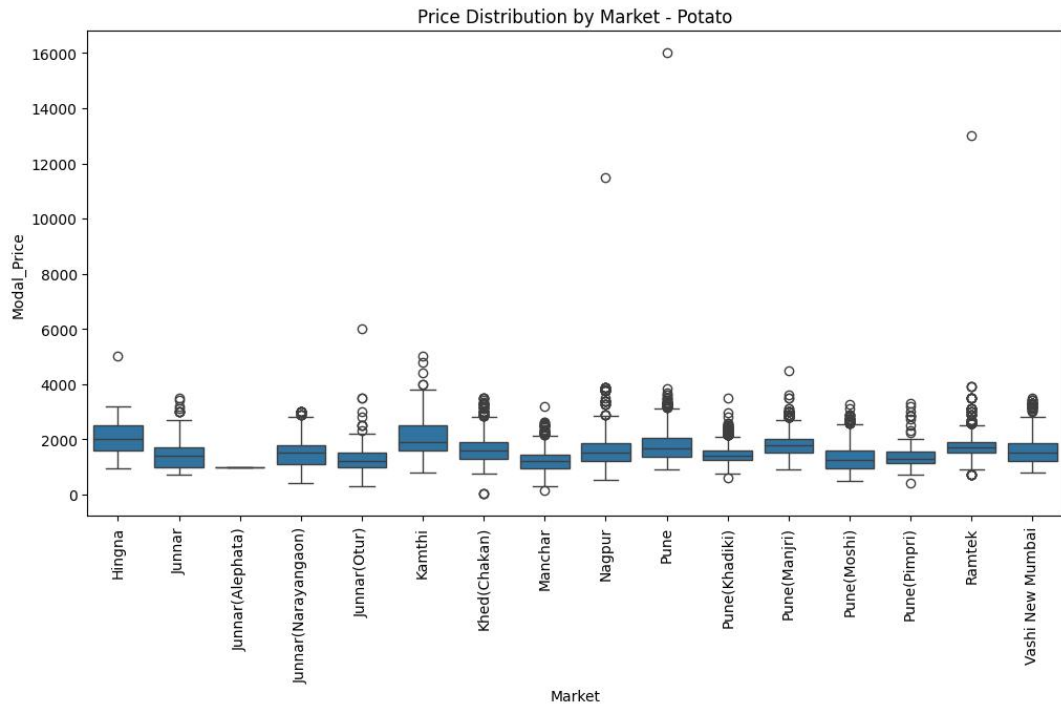
Analysis by Price Behavior

The commodities can be grouped into three distinct categories based on their price stability and volatility.

1. *Stable Price Commodities: Wheat & Potato*

These commodities show the most stable pricing.

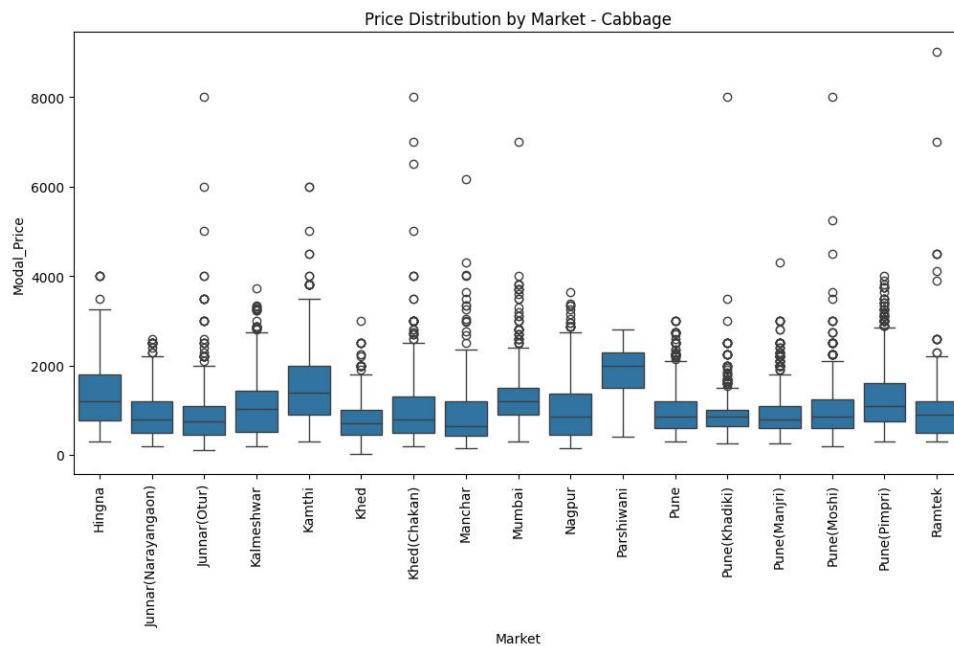
- **Low Volatility:** The boxes are generally short and compact, indicating that prices don't fluctuate dramatically. The middle 50% of prices are tightly clustered around the median.
- **Consistent Medians:** The median price (the line in the box) is consistent across different markets.
- **Observation:** This suggests a more stable supply chain, less perishability, and more predictable pricing compared to other produce. **Wheat** is particularly stable, with most prices falling between ₹2,000 and ₹3,000.

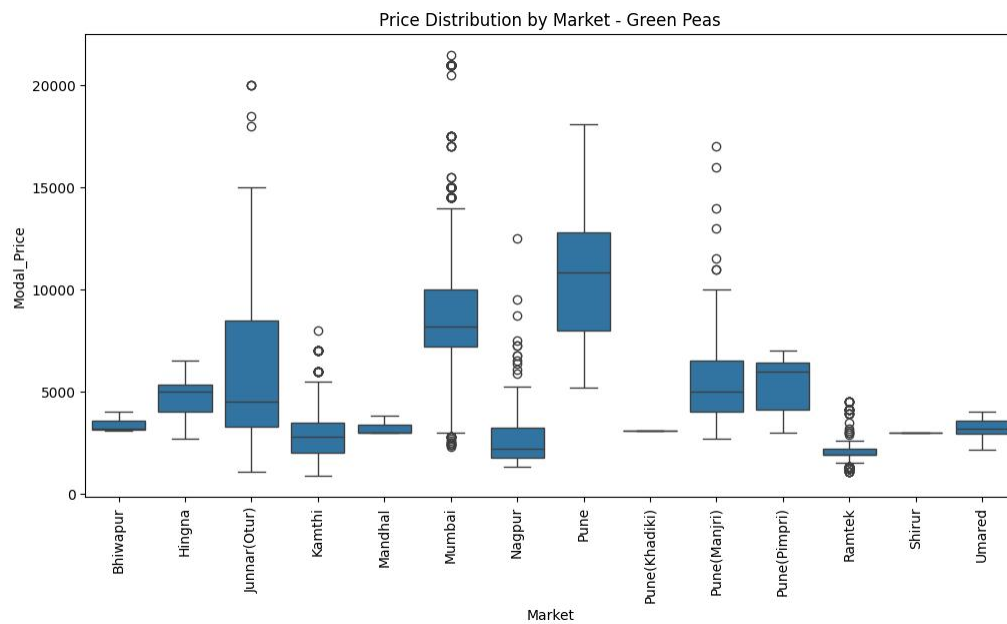
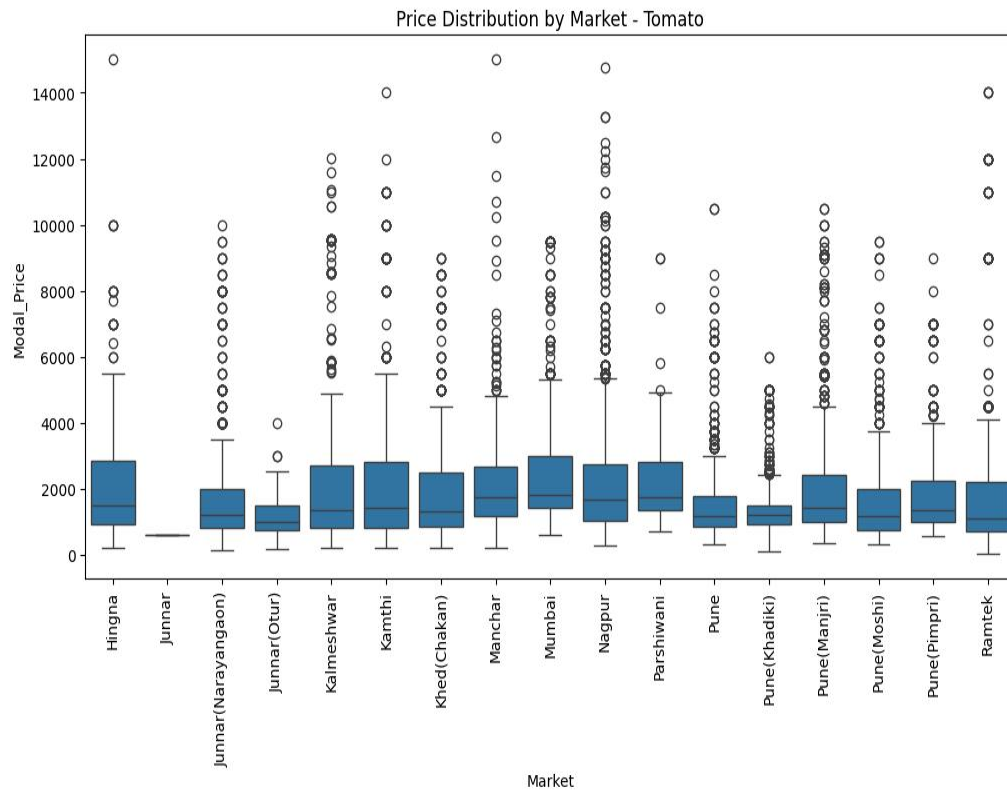


2. Volatile Price Commodities: Tomato, Cabbage & Green Peas

These commodities exhibit significant price fluctuations.

- **High Volatility:** The boxes and whiskers are much taller, showing a wide spread of prices. This means that even within a normal period, the price can vary significantly.
- **Market-Dependent Prices:** The price levels differ greatly from one market to another. **Green Peas**, for example, have a very high and volatile price in Mumbai and Pune but are much cheaper and more stable in places like Bhiwapur.
- **Numerous Outliers:** There are many instances of unusually high prices (outliers) in most markets.
- **Observation:** This pattern is common for fresh produce where factors like seasonality, weather, and logistics have a strong and immediate impact on price.

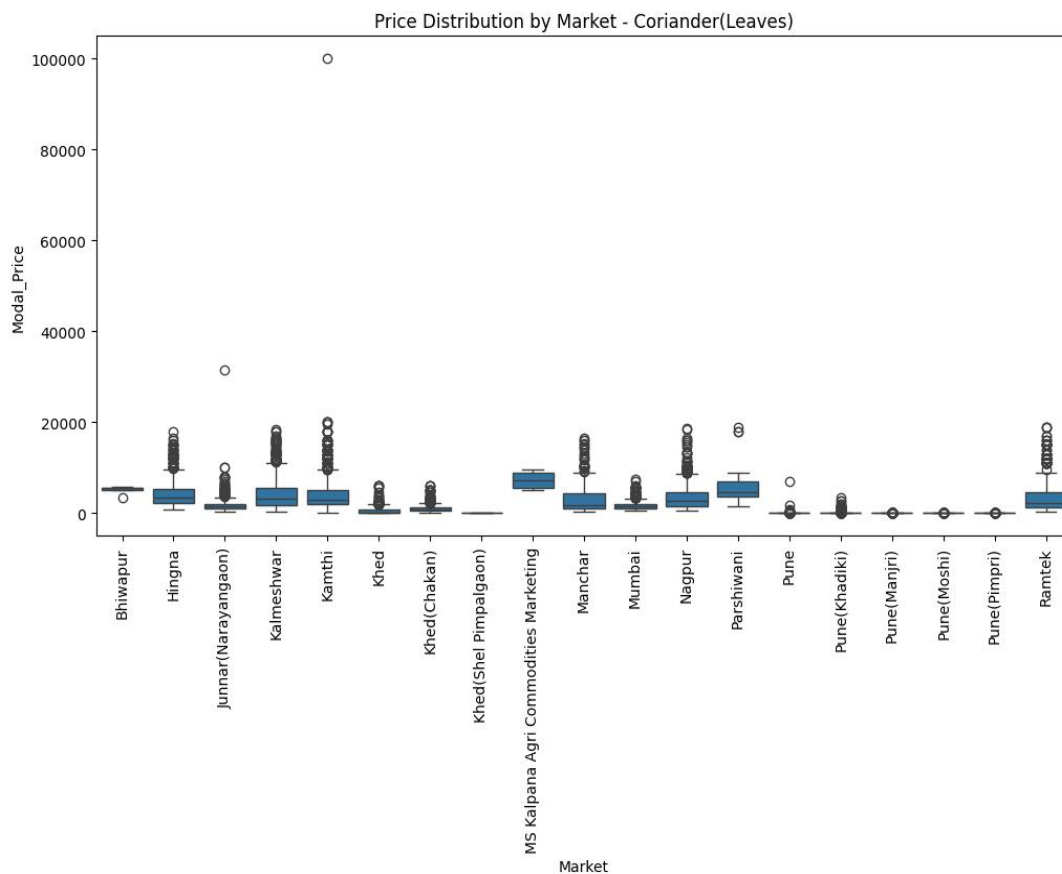


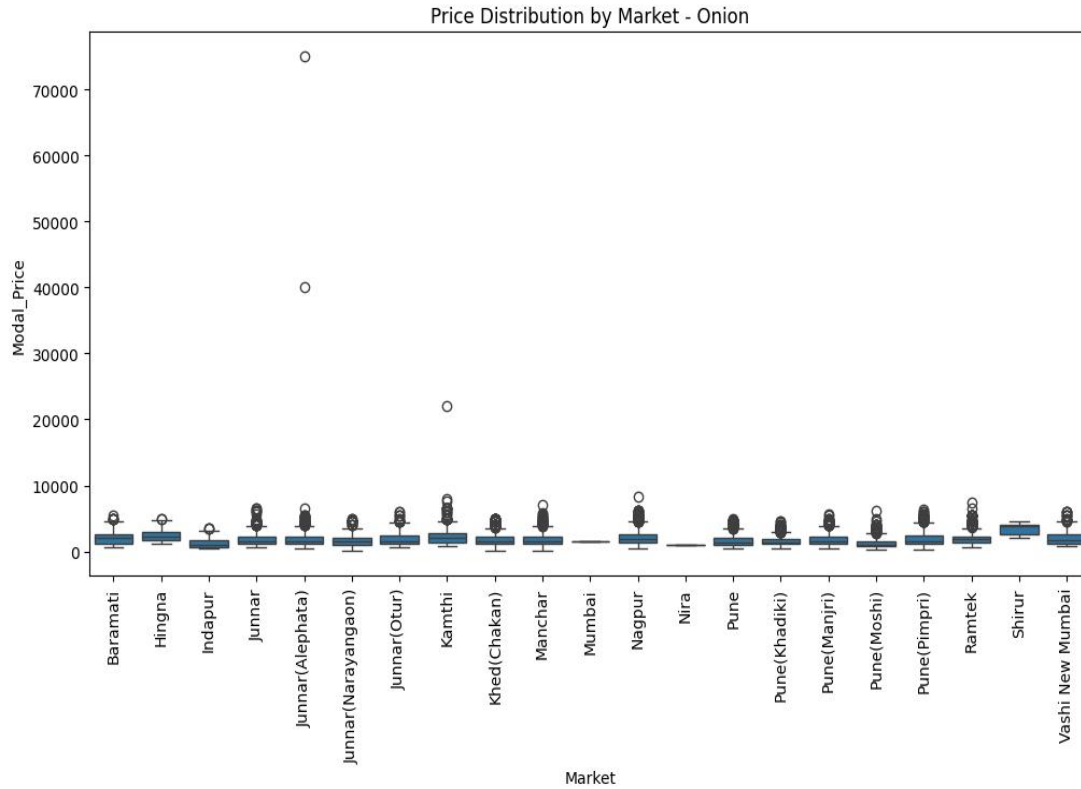


3. "Spiky" Price Commodities: Onion & Coriander (Leaves)

These commodities have the most dramatic price behavior.

- **Usually Low & Stable:** The boxes are extremely small and compressed at the very bottom of the chart. This means that most of the time, the price is very low and does not change much.
- **Extreme Price Spikes:** The charts are dominated by many extreme outliers. The **Onion** chart shows price spikes going above ₹70,000, and **Coriander** has an outlier reaching an incredible ₹100,000.
- **Observation:** This is a classic pattern for highly perishable goods. A sudden supply shortage (due to bad weather, crop disease, or transit issues) can cause prices to skyrocket temporarily before returning to their low baseline.

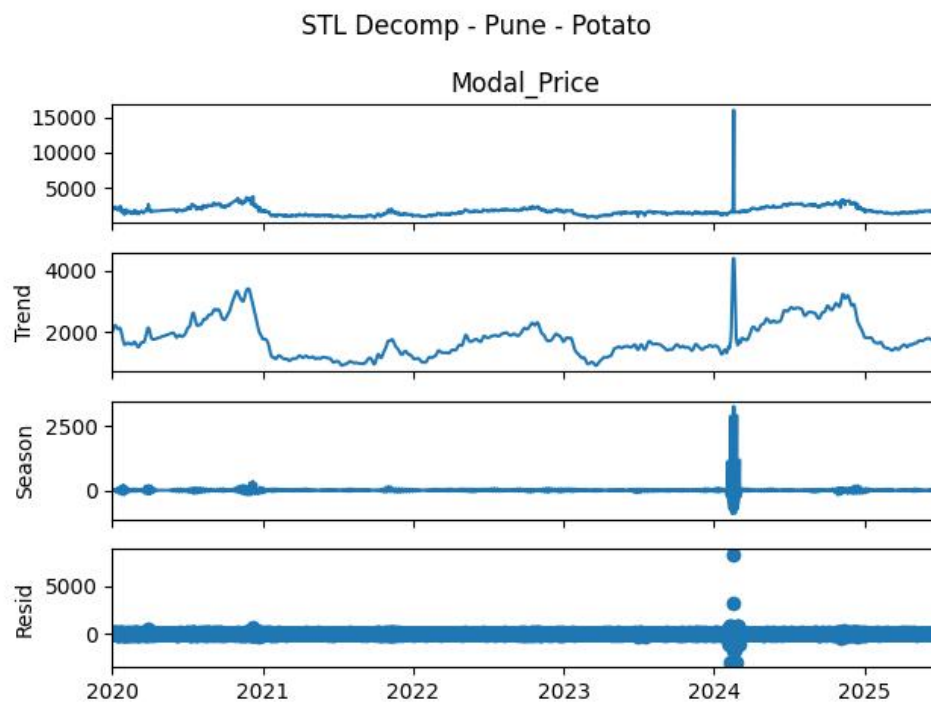




The exploratory analysis confirmed that agricultural commodity prices are influenced by a combination of predictable seasonal cycles, weather-related factors, and sudden anomalies. It also established the strong heterogeneity across commodities and markets. These findings guided the design of our forecasting pipeline by justifying the inclusion of exogenous regressors, the use of models with flexible seasonal structures, and the need for anomaly detection prior to model training.

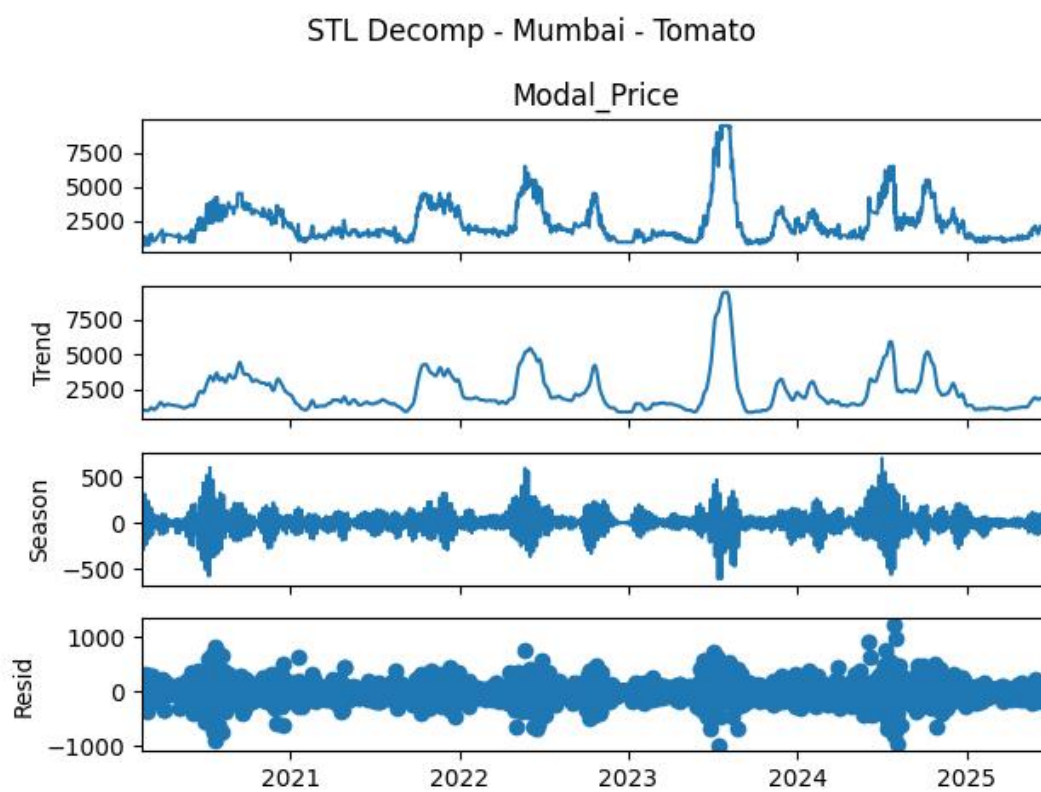
Seasonal Decomposition:

To better understand the underlying structure of commodity price series, we applied Seasonal-Trend decomposition using Loess (STL). STL splits the observed price into long-term trend, seasonal cycles, and residual components, allowing us to isolate systematic patterns from irregular shocks.



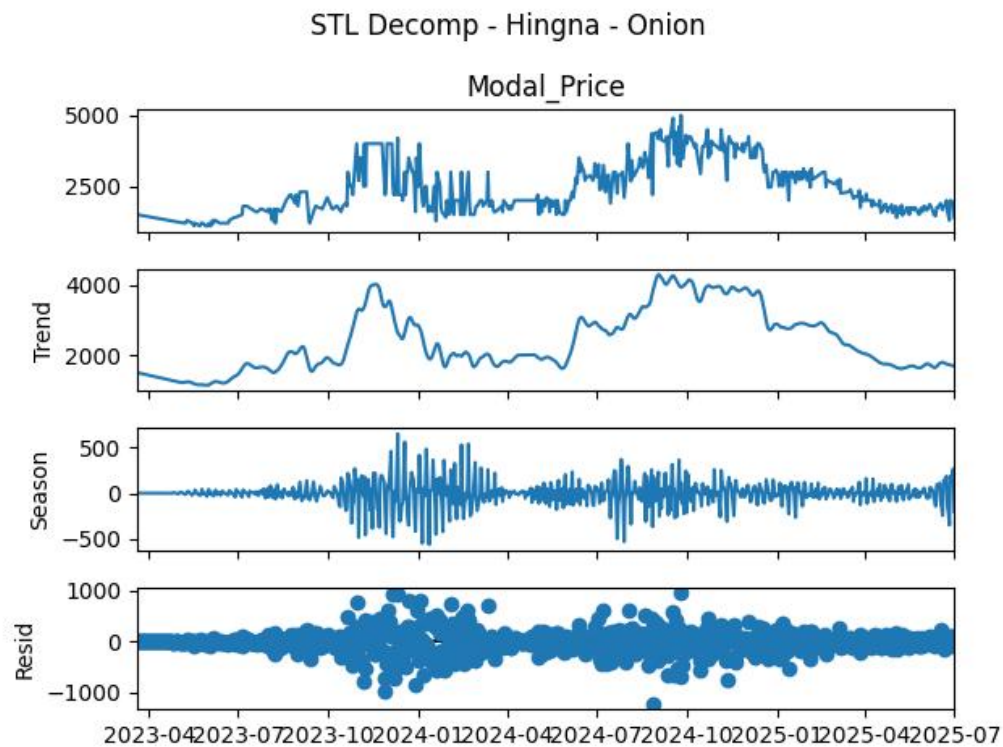
Potato – Pune

- Trend shows gradual upward movement with one large spike.
- Seasonality is weak, meaning potatoes are less driven by cycles.
- Residuals highlight sudden shocks in 2024.



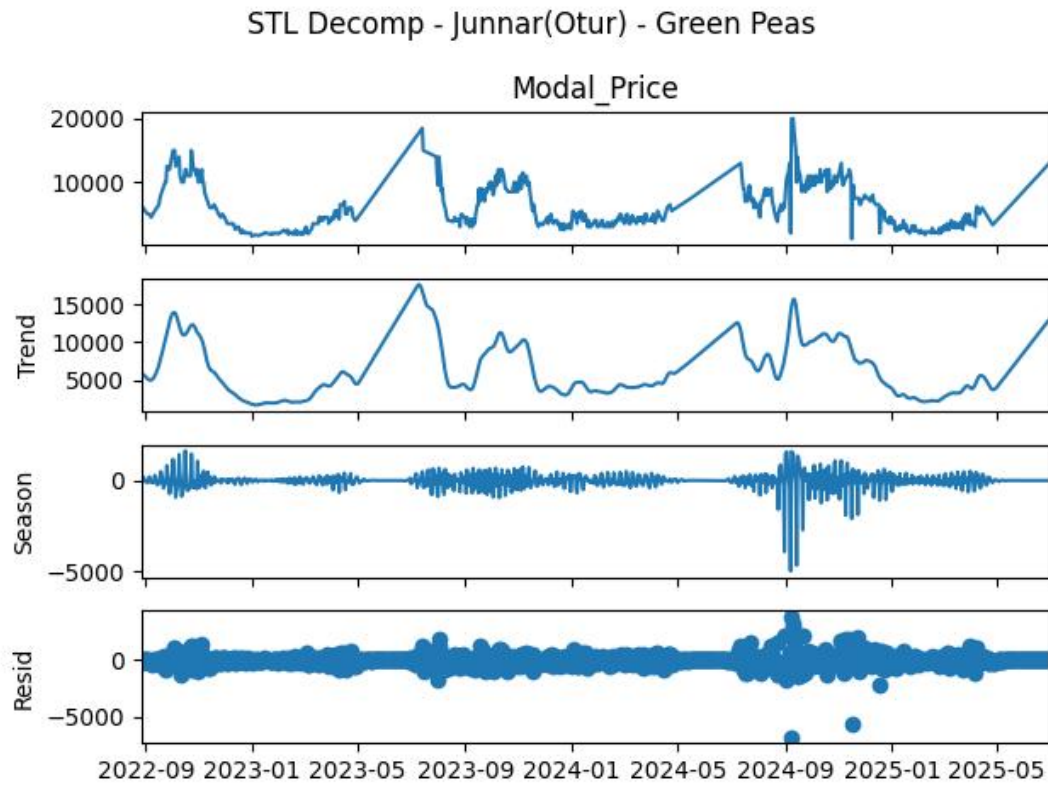
Tomato – Mumbai

- Strong seasonal cycles with clear yearly peaks.
- Trend shows long-term fluctuations.
- Residuals show large irregular shocks, consistent with perishability.



Onion – Hingna

- Trend indicates long upward and downward swings.
- Seasonality stronger during harvest and monsoon periods.
- Residuals reveal unpredictable short-term volatility.



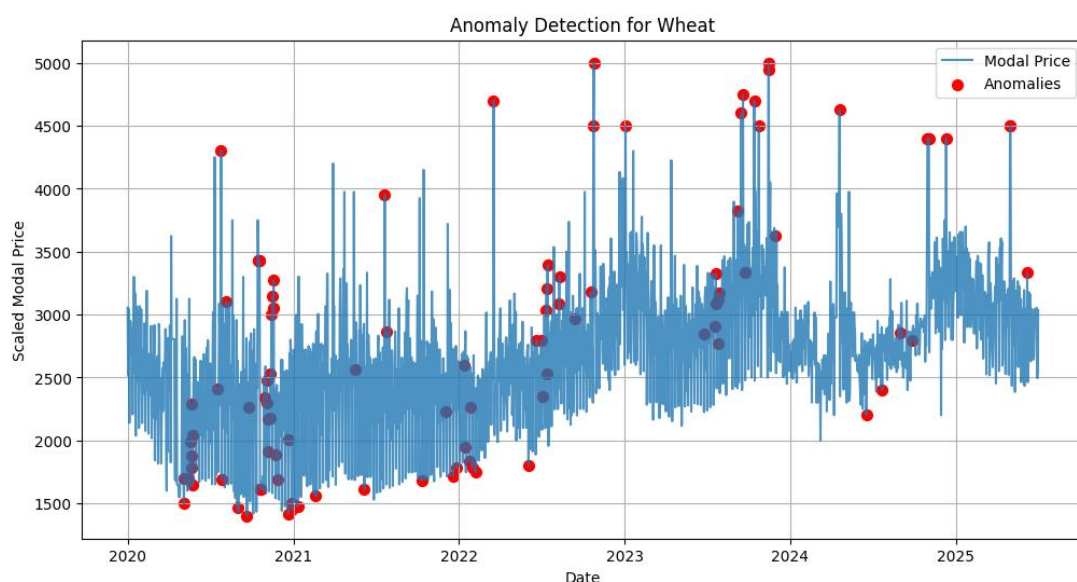
Green Peas – Junnar (Otur)

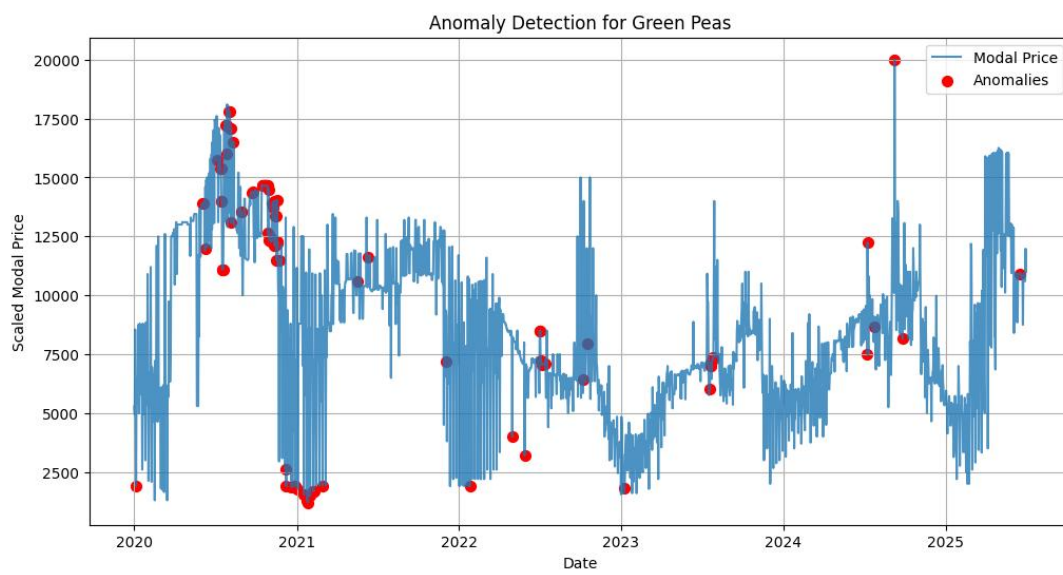
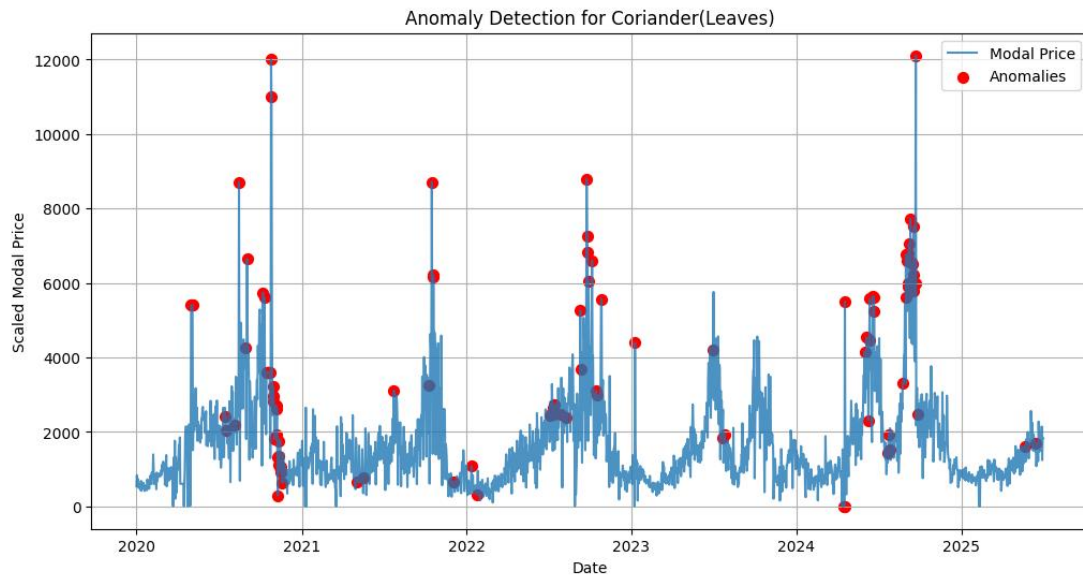
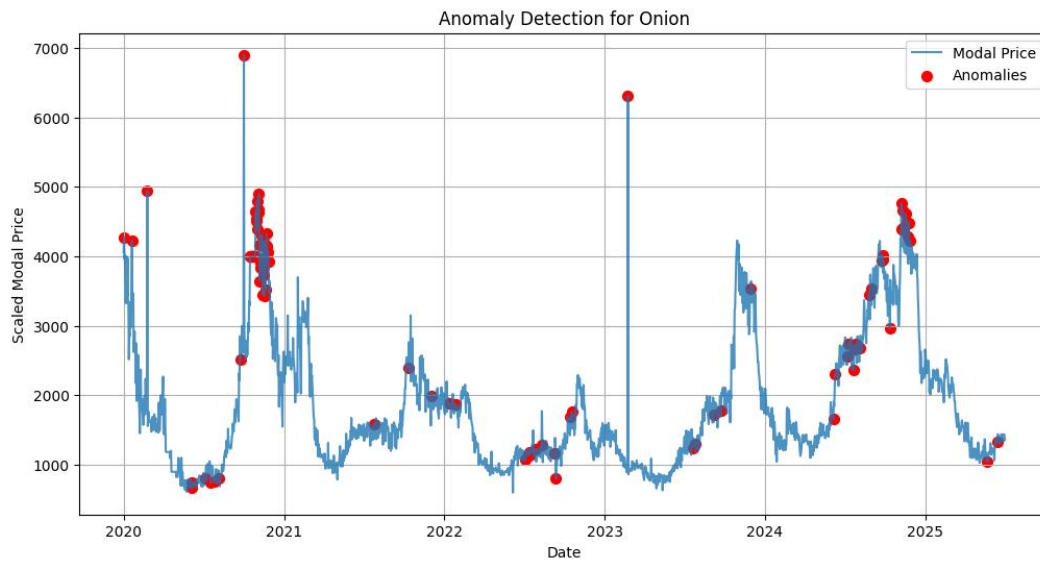
- Trend highly volatile, prices rise and crash rapidly.
- Seasonality visible but irregular, suggesting strong external influences.
- Residuals very large → confirms forecasting difficulty.

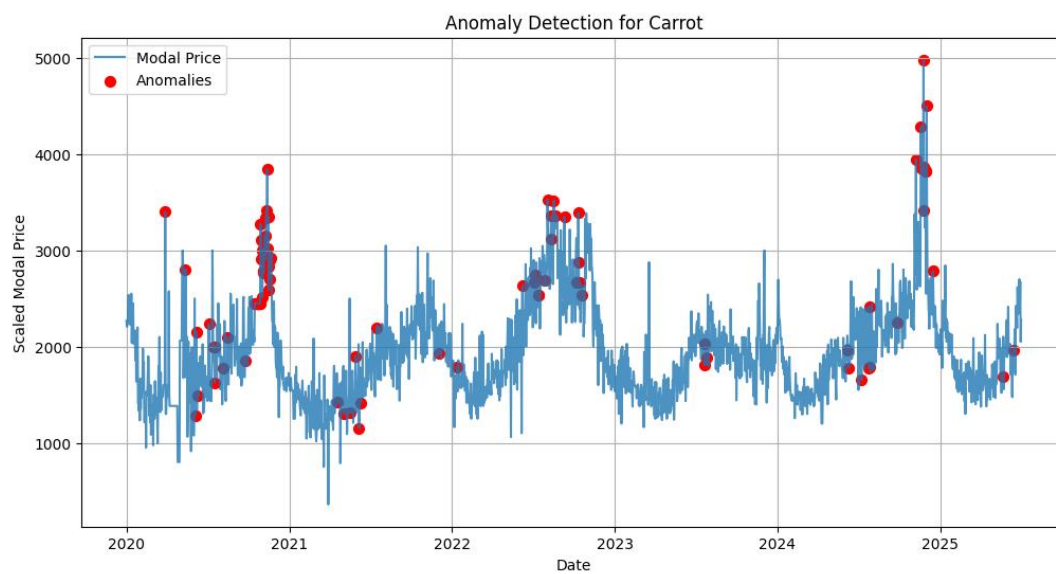
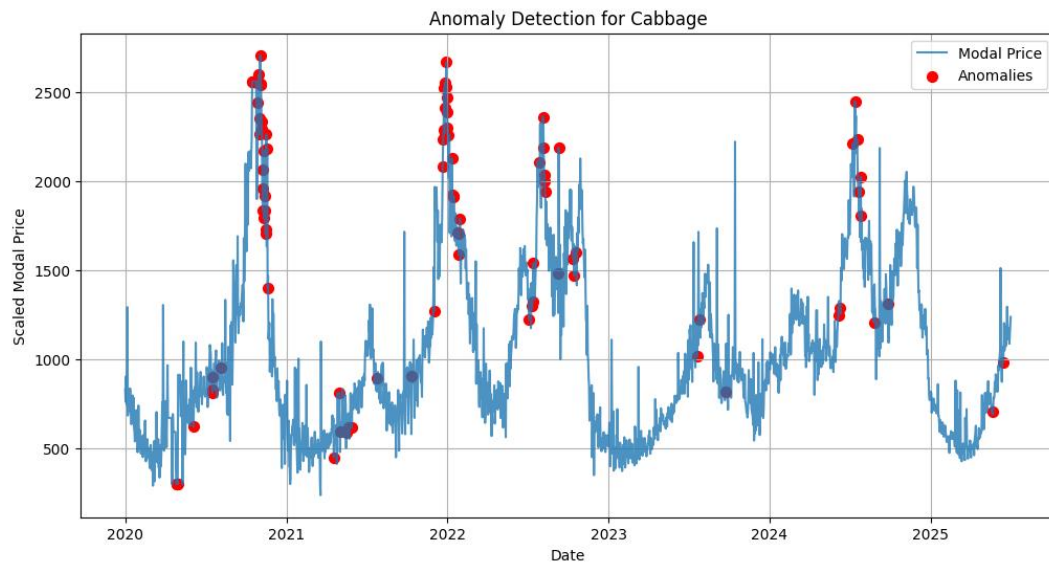
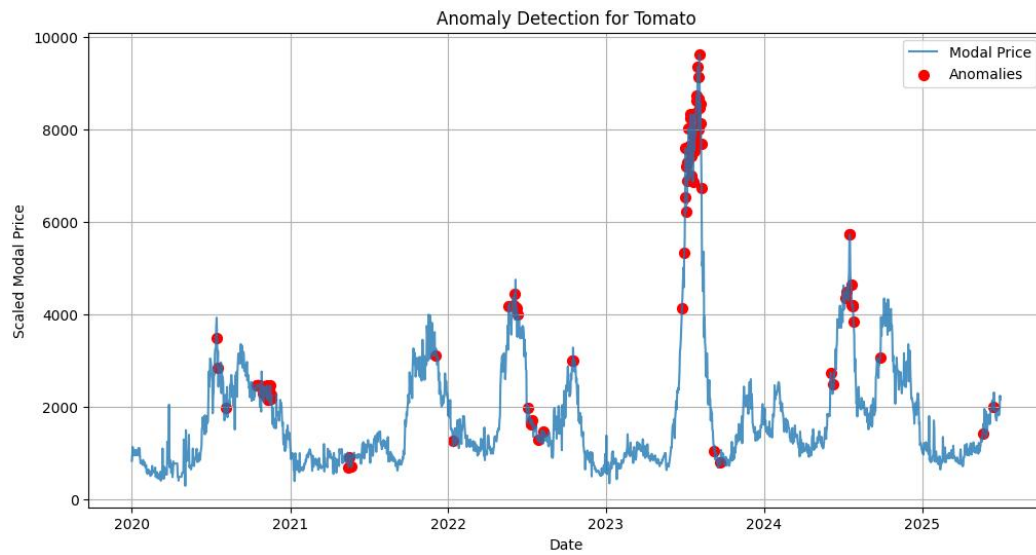
Anomaly Detection

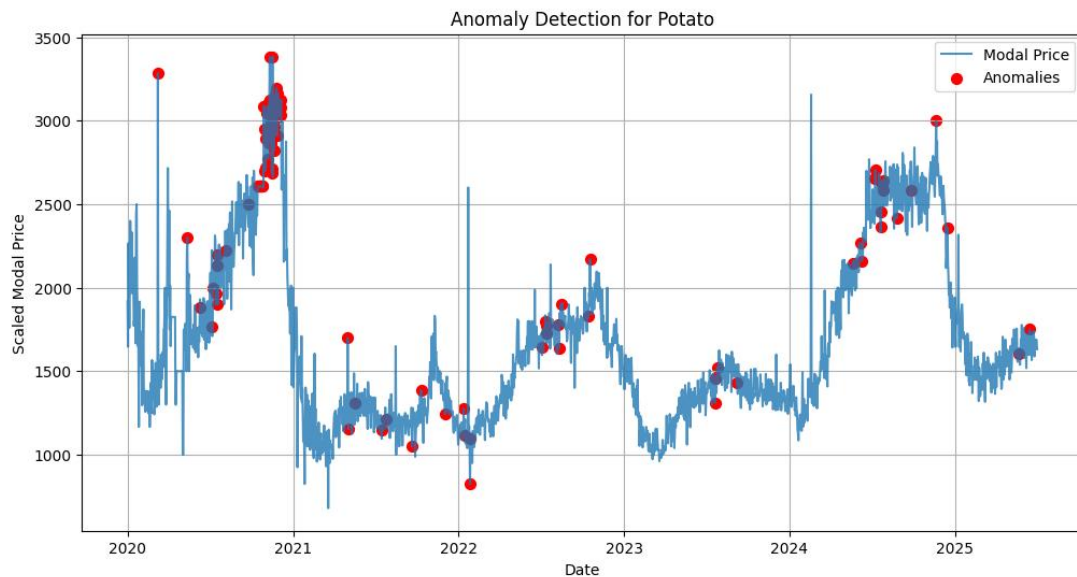
Agricultural market prices are often affected by unusual events that create sudden deviations from their expected behavior. These deviations, or anomalies, can arise from multiple sources. Some anomalies represent real-world shocks such as sudden policy interventions, extreme weather events, transportation strikes, or unexpected surges in demand. Others may stem from reporting errors, including misentered prices or missing observations that are later backfilled incorrectly. Regardless of their origin, anomalies pose a challenge for forecasting models. If they are not detected and appropriately treated, models may interpret them as part of the regular pattern of the data and adjust forecasts in ways that are misleading. For this reason, anomaly detection was an important stage in our pipeline.

- We used the Isolation Forest algorithm as the core method for anomaly detection. Isolation Forest is an unsupervised learning approach that works by randomly partitioning the feature space into smaller segments. Anomalies are easier to isolate than normal observations because they tend to be rare and distant from the majority of the data. The algorithm therefore detects anomalies by measuring how quickly an observation is separated from the rest of the dataset. Observations that are isolated in fewer steps are flagged as potential anomalies. This approach has several advantages in our context. It is non-parametric, which means it does not assume a specific distribution of the data. It also handles multivariate inputs, allowing us to consider price alongside weather variables when detecting unusual behavior.
- In practice, we applied the algorithm to a feature set that included the modal price, rainfall, and average temperature for each market–commodity pair. Including weather variables in addition to prices ensured that the algorithm could distinguish between natural fluctuations explained by environmental conditions and truly abnormal deviations. For example, a price drop that coincided with favorable rainfall might not be considered anomalous, while a sudden price spike during stable weather conditions would be more likely to be flagged.









Paradigms in Time Series Forecasting

When it comes to forecasting, there are two main schools of thought: **the classic statistical approach** and **the modern machine learning approach**. The biggest difference between them is how they think about the problem.

The statistical approach, with models like ARIMA and ETS, is based on theory. These models start with the assumption that a time series can be broken down into understandable parts, like a trend, a seasonal pattern, and some random noise. The goal is to find a mathematical equation that describes these parts. When this works, it's great because you get a simple, easy-to-understand model.

The machine learning approach is different. It's all about the data. Models like Random Forest and XGBoost don't start with many assumptions. Instead, they are designed to be flexible enough to learn almost any pattern, no matter how complex or non-linear, just by looking at the data. They basically turn the forecasting problem into a general regression problem, where you use past information as features to predict a future value. This often leads to more accurate predictions, but it can be harder to understand exactly *why* the model is making a certain prediction. People often call these "black-box" models. A big part of our project was to compare these two approaches on real-world data to see which one was better for this specific problem.

Statistical Forecasting Models

1. The ARIMA and SARIMAX Frameworks

The ARIMA model is one of the most well-known methods in time series forecasting. The name ARIMA stands for Autoregressive Integrated Moving Average, and each part of the name describes something the model does. It's usually written as ARIMA(p,d,q).

- **Autoregressive (p):** This part of the model looks at past values of the data to make a prediction. The 'p' tells you how many past time steps the model looks at. For example, if $p=2$, the model uses the values from the last two periods to help predict the current one.
- **Integrated (d):** Many time series have a trend, meaning they tend to go up or down over time. This makes them "non-stationary," which can be a problem for this type of model. The 'I' in ARIMA stands for integrated, which refers to a process called differencing. This is where we transform the data by looking at the difference between consecutive values, which usually gets rid of the trend and makes the data stationary. The 'd' tells you how many times you need to do this.
- **Moving Average (q):** This part of the model doesn't look at past values, but at past forecast errors. The idea is that if the model was off by a certain amount in the past, it can use that information to adjust its current forecast. The 'q' tells you how many past errors the model considers.

- **Advantage of ARIMA:**

ARIMA has long been a popular model for time-series forecasting, and it brings several advantages to a commodity price prediction project. Its biggest strength lies in its ability to model temporal dependencies directly, which makes it well-suited for sequential data such as daily or weekly mandi prices. By combining autoregressive (AR) terms, moving average (MA) terms, and differencing (I – integration), ARIMA is capable of capturing short-term momentum, shocks, and gradual trends in the data. For commodity prices, which often display seasonality and trend, ARIMA can provide relatively accurate forecasts without requiring complex feature engineering. Another advantage of ARIMA is interpretability: its parameters (p, d, q) have clear statistical meaning and help explain the underlying structure of the time series, making the model easier to present to stakeholders such as policymakers or academics. In addition, ARIMA works well with small datasets, which is valuable in cases where historical data is limited or incomplete. When seasonality is strong, its extension SARIMA can explicitly model repeating seasonal cycles, offering useful insights into price behavior during harvest months or festival seasons.

- **Disadvantage:**

ARIMA also has limitations that must be considered. A major drawback is that it assumes linear relationships in the data, which means it may not capture nonlinear dynamics often present in commodity prices, such as the interaction of weather, demand spikes, and government policy changes. ARIMA also requires the time series to be stationary, so preprocessing steps like differencing or transformation are often needed, which adds complexity. Another limitation is scalability: ARIMA is designed for univariate forecasting,

meaning it relies primarily on past values of the target series and cannot easily incorporate multiple external features like rainfall, temperature, or international price shocks without modifications such as ARIMAX. This makes it less flexible compared to machine learning models like XGBoost or Random Forest. In addition, ARIMA can struggle with long-term forecasts since it tends to project recent patterns forward and may not adapt well to structural breaks or sudden regime shifts. Model selection is also a challenge, as choosing the correct values of p , d , and q requires careful analysis using tools like ACF and PACF plots, and mis-specification can lead to poor performance.

A more advanced version of this is the **SARIMAX** model. The 'S' stands for Seasonal, which means it can also handle repeating patterns, like prices being higher every winter. The 'X' stands for exogenous variables, which is a fancy way of saying it can include other information in the model. For this project, that's where I added things like weather data, because rainfall and temperature can obviously have a big impact on crop prices.

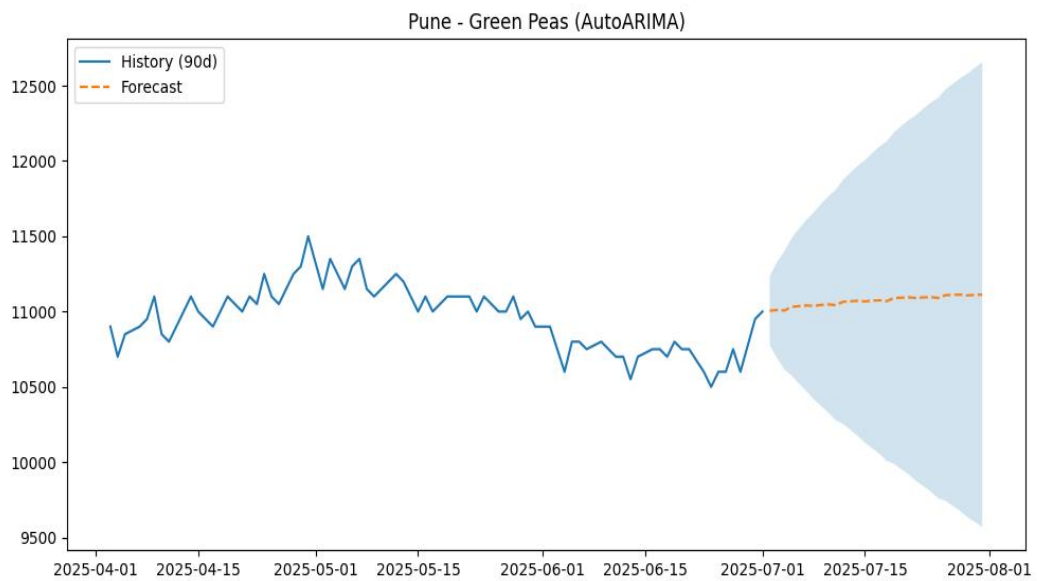
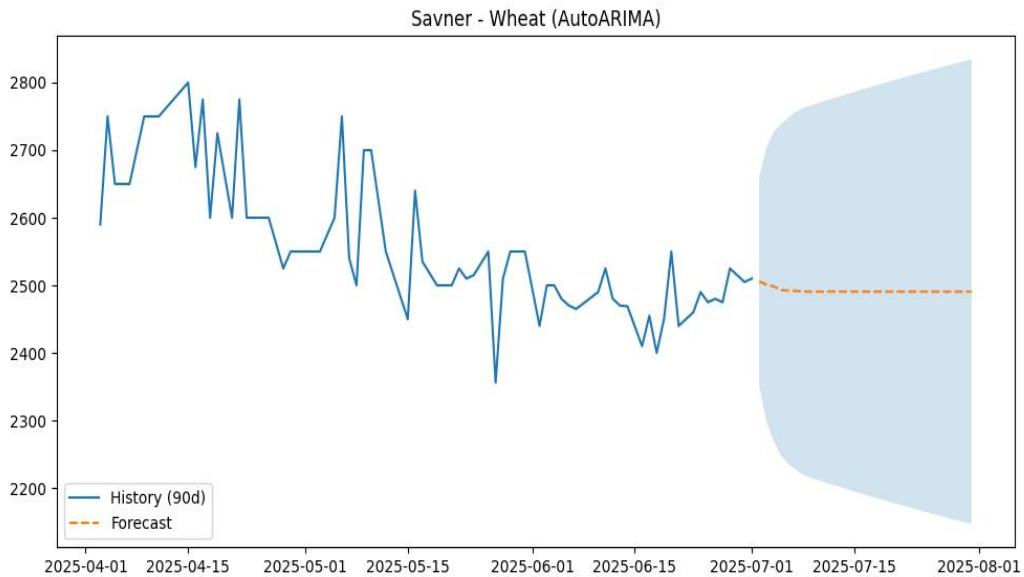
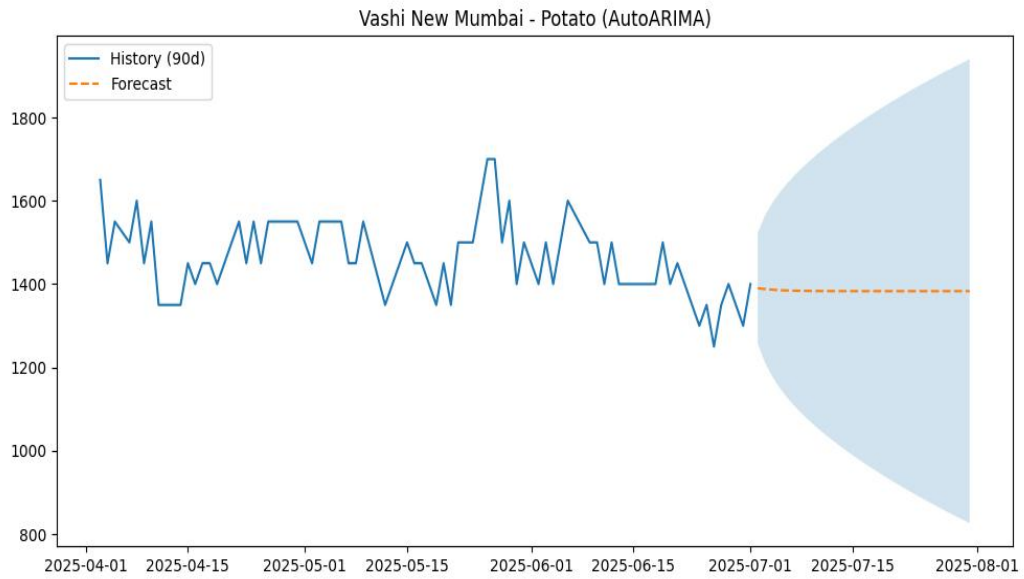
- **Advantages of SARIMAX:**

SARIMAX extends the traditional ARIMA framework by incorporating both seasonality and exogenous variables, making it a powerful tool for commodity price prediction. One of its main advantages is that it naturally models seasonality, which is highly relevant in agricultural markets where price cycles are influenced by harvest seasons, monsoons, and festivals. This seasonal adjustment allows SARIMAX to better capture repeating patterns that are often missed by basic ARIMA. In addition, the “X” in SARIMAX refers to exogenous variables external features that can be included as regressors. For commodity price forecasting, this means weather data (rainfall, temperature), policy decisions, or global market indicators can be directly incorporated into the model, enriching its predictive power. SARIMAX also provides interpretable parameters, which is useful in explaining the effects of both time-series dynamics and external factors on price movements. Its statistical foundation ensures that the forecasts are not just predictive but also explanatory, which is valuable for policymakers, economists, and stakeholders seeking to understand how different drivers impact commodity prices.

- **Disadvantage:**

SARIMAX also has limitations that should be recognized. Like ARIMA, it assumes linear relationships and stationarity, which may not hold in volatile commodity markets where nonlinear shocks or regime shifts occur. This means that significant preprocessing, such as differencing, transformations, and careful selection of seasonal orders, is often required before the model can be applied effectively. Moreover, identifying the right parameters (p , d , q , P , D , Q , s) and choosing suitable exogenous variables can be time-consuming and computationally heavy, especially when working with multiple commodities and markets. While SARIMAX is capable of including external regressors, its performance depends heavily on the quality and relevance of these variables; poorly chosen features may add noise instead of improving accuracy. Another limitation is that SARIMAX is not inherently designed for large-scale datasets with high dimensionality, making it less efficient than modern machine learning models like XGBoost or Random Forest for massive or complex datasets. It may also underperform when sudden shocks occur that were not captured in the historical data or external regressors, as it tends to extrapolate from past patterns.

- **Plots**



```
if HAS_PMD:
```

```
    try:
```

```
        m_ar = pm.auto_arima(y_train, exogenous=exog_train, seasonal=True, m=7,
                             stepwise=True, suppress_warnings=True, error_action='ignore')
```

```
        fc = m_ar.predict(n_periods=len(y_test), exogenous=exog_test)
```

```
        model_scores['AutoARIMA'].append(rmse(y_test, fc))
```

```
        last_fold_preds['AutoARIMA'] = pd.Series(fc, index=y_test.index)
```

```
    except Exception:
```

```
        model_scores['AutoARIMA'].append(np.nan)
```

2. Exponential Smoothing (ETS) State-Space Models

Exponential Smoothing, or ETS, is another popular family of statistical models. The basic idea is that more recent observations are more important than older ones, so we should give them more weight when making a forecast. The "smoothing" part comes from the fact that these models produce forecasts that are weighted averages of past observations.

Modern ETS models are set up in what's called a state-space framework. This just means the model has a few key components that it tracks over time:

- **Level (L):** The baseline value of the series.
- **Trend (T):** The direction the series is moving in (up, down, or flat).
- **Seasonality (S):** A repeating pattern or cycle.

The models are named based on how they handle these components. For example, a model might have an Additive trend (meaning the trend adds a fixed amount each period) or a Multiplicative seasonal pattern (meaning the seasonal effect is a percentage of the level). This gives you a whole

family of different models, and you can choose the one that best fits the data. For example, the Holt-Winters method is a well-known ETS model that can handle both trend and seasonality. The great thing about this framework is that it can automatically select the best combination of components for your data, making it quite flexible.

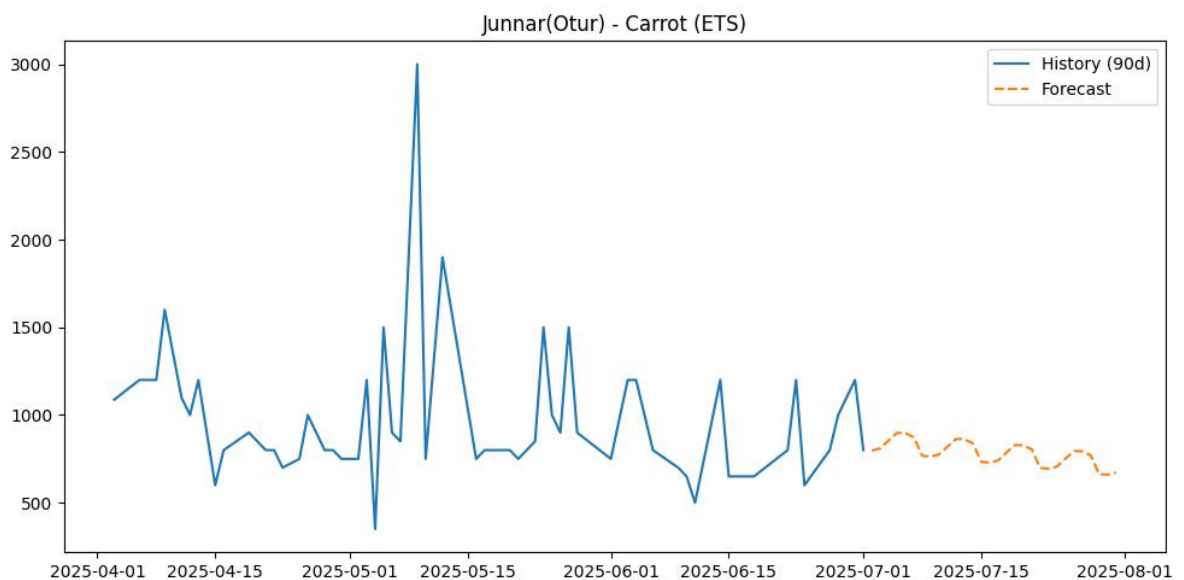
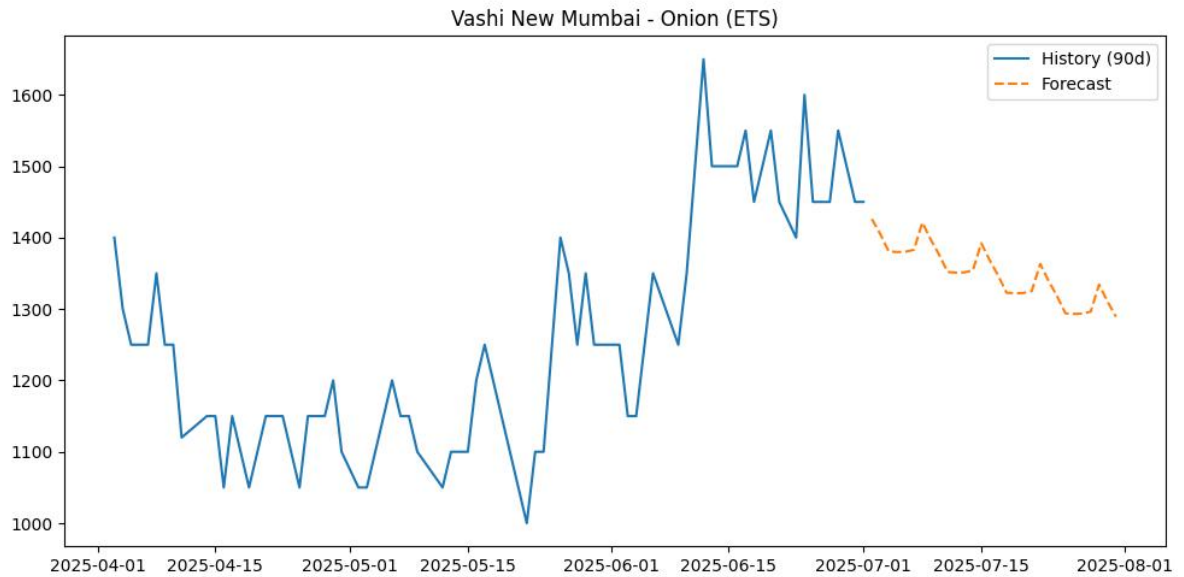
- **Advantages:**

The Exponential Smoothing (ETS) family of models is another important approach for time-series forecasting, and it brings unique advantages to a commodity price prediction project. ETS models are built on the principle of exponential smoothing, where more recent observations are given higher weights while older observations gradually lose influence. This makes ETS particularly effective in capturing local trends and short-term price movements, which are common in agricultural commodities that often experience sudden but temporary fluctuations. ETS is flexible in modeling different components of a time series, namely error, trend, and seasonality. For commodity prices that display seasonal cycles (for example, higher demand during festivals or lower supply during off-harvest months), ETS can explicitly model both additive and multiplicative seasonal effects, providing forecasts that adapt to these patterns. Another strength of ETS is its interpretability: its decomposition into trend and seasonal components offers clear insights into the structure of the time series. It is also relatively easy to implement, computationally efficient, and performs well when the dataset is clean, stationary, and primarily driven by recurring seasonal and trend-based patterns.

- **Disadvantage:**

ETS has limitations that can reduce its effectiveness for commodity price prediction. Unlike machine learning methods such as Random Forest or XGBoost, ETS does not handle nonlinearities or interactions with external variables. It is fundamentally a univariate model, meaning it relies only on the historical values of the target series and cannot easily incorporate external drivers like rainfall, temperature, or policy changes, which are often critical in agricultural markets. As a result, it may fail to capture price shocks that are not embedded in historical seasonal or trend structures. ETS can also underperform in highly volatile or irregular series where sudden structural breaks occur, as it tends to extrapolate recent patterns forward without adapting to unforeseen events. Another drawback is that ETS models often require the series to be reasonably stable; they may not generalize well when applied across multiple commodities and markets with very different seasonalities or volatility profiles. While ETS is efficient and simple, it may lack the flexibility needed for large-scale forecasting pipelines that need to incorporate diverse features and handle highly complex behaviors.

- **Plots**



try:

```

y_train_stationary = ensure_stationary(y_train)
if len(y_train_stationary) > 30:
    best_s = float('inf'); best_fc = None
    for trend, seasonal in [('add','add'), ('add', None), (None, 'add')]:
        try:
            m_ets = ExponentialSmoothing(y_train_stationary, trend=trend, seasonal=seasonal,
seasonal_periods=ETS_SEASONAL_PERIODS).fit()

            fc = m_ets.forecast(len(y_test))

            s = rmse(y_test, fc)

            if s < best_s:

```

```
        best_s = s; best_fc = fc
    except Exception:
        continue
    model_scores['ETS'].append(best_s)
    last_fold_preds['ETS'] = pd.Series(best_fc, index=y_test.index)
else:
    model_scores['ETS'].append(np.nan)
except Exception:
    model_scores['ETS'].append(np.nan)
```

3. Support Vector Regression (SVR)

Support Vector Regression, or SVR, is a machine learning model that thinks about regression problems in a unique way. Instead of trying to find a line that is as close to as many data points as possible (like in simple linear regression), SVR tries to fit a "tube" or "corridor" around the data.

The model defines a margin of error, called epsilon, on either side of its regression line. The key idea is that the SVR model doesn't care about any data points that fall *inside* this tube it considers

their error to be zero. It only focuses on the points that fall *outside* the tube. The model's goal is to make this tube as wide as possible while still capturing the general trend of the data and minimizing the errors for the points outside of it. The points that lie on the edge of this tube are called "support vectors," and they are the critical points that define where the regression line is.

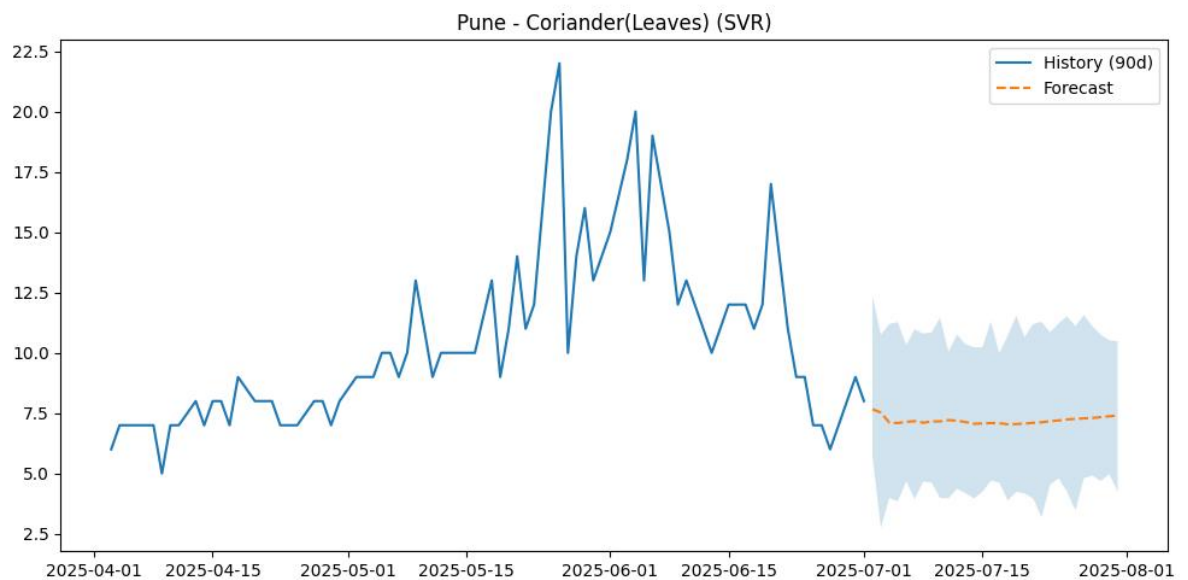
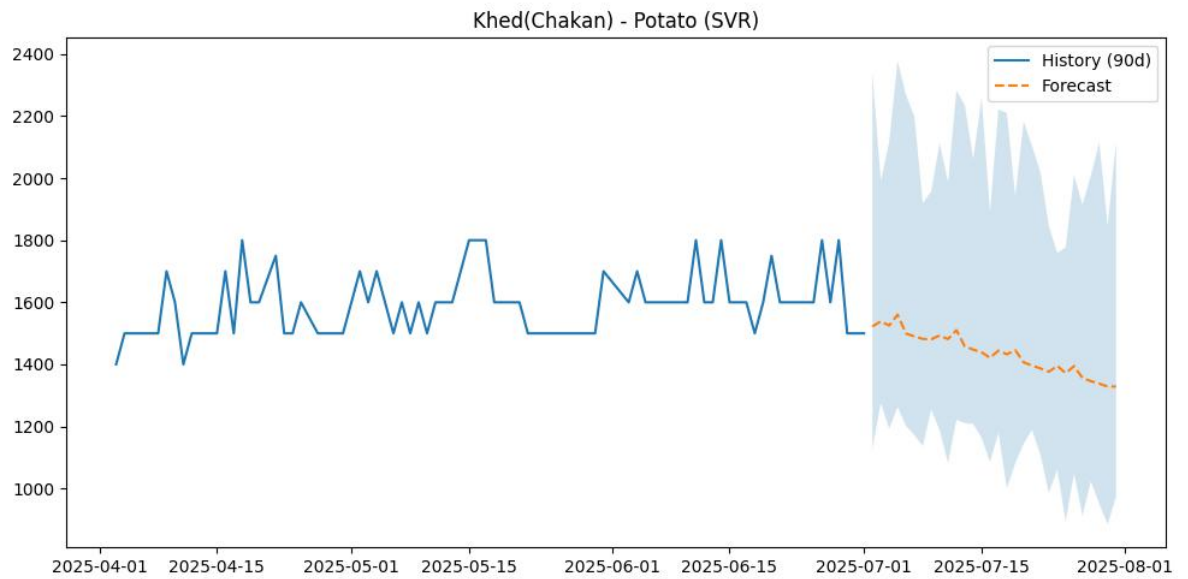
- **Advantages:**

Support Vector Regression (SVR) brings some notable strengths to commodity price prediction. One of its key advantages is its ability to model nonlinear relationships through the use of kernel functions. Commodity prices are often influenced by complex, nonlinear interactions such as the effect of rainfall combined with market demand or the impact of government policies layered over seasonal patterns. By using kernels like radial basis functions (RBF), SVR can capture these nonlinear dynamics more effectively than traditional linear models. Another benefit is its robustness to outliers and noise, since SVR focuses on fitting within a specified margin of tolerance (the epsilon-insensitive zone) rather than trying to minimize overall error directly. This is especially useful in mandi price datasets, which may contain noisy or inconsistent records. SVR also works well with high-dimensional feature spaces, making it a good option when multiple external factors—such as weather, holidays, or macroeconomic indicators—are introduced. Additionally, it is less prone to overfitting compared to some machine learning methods, as it uses regularization to balance model complexity with generalization.

- **Disadvantage:**

SVR also has significant limitations in the context of commodity price forecasting. A major drawback is scalability: SVR is computationally expensive in both training time and memory usage, particularly with large datasets that cover many commodities and long historical periods. This can make it impractical for real-world agricultural data pipelines. Another limitation is interpretability. Unlike ARIMA or Prophet, which provide clear decompositions into trend and seasonality, SVR operates as a black-box model, offering little transparency into why a particular prediction is made. Moreover, while it can handle nonlinearities, SVR does not inherently model sequential dependencies like seasonality, trend, or lagged effects, which are central to commodity price dynamics. To make it effective for time-series data, significant feature engineering—such as creating lag features, moving averages, or seasonal indicators—is required. In addition, SVR is highly sensitive to hyperparameter tuning, including the choice of kernel, the penalty parameter (C), and epsilon. Poorly tuned parameters can result in either underfitting (overly smooth predictions) or overfitting (capturing noise instead of signal). Finally, SVR tends to struggle with extrapolation, making it less reliable in situations where future prices fall outside the historical range due to unexpected shocks.

- **Plots**



try:

```
best_params = optuna_tune("SVR", X_train_feat, y_train_ml, n_trials=n_trials) or {}
scaler = StandardScaler().fit(X_train_feat)
model_svr = SVR(**best_params)
model_svr.fit(scaler.transform(X_train_feat), y_train_ml)

fc_series = iterative_forecast_ml(model_svr, y_train, exog_train, exog_test, len(y_test),
LAGS, feature_cols, scaler=scaler)

model_scores['SVR'].append(rmse(y_test, fc_series))

last_fold_preds['SVR'] = fc_series

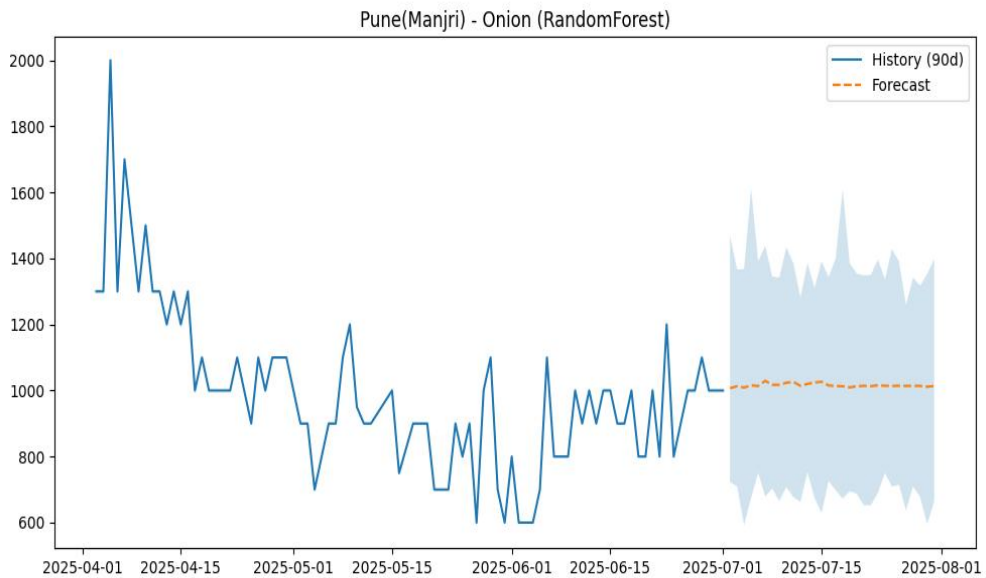
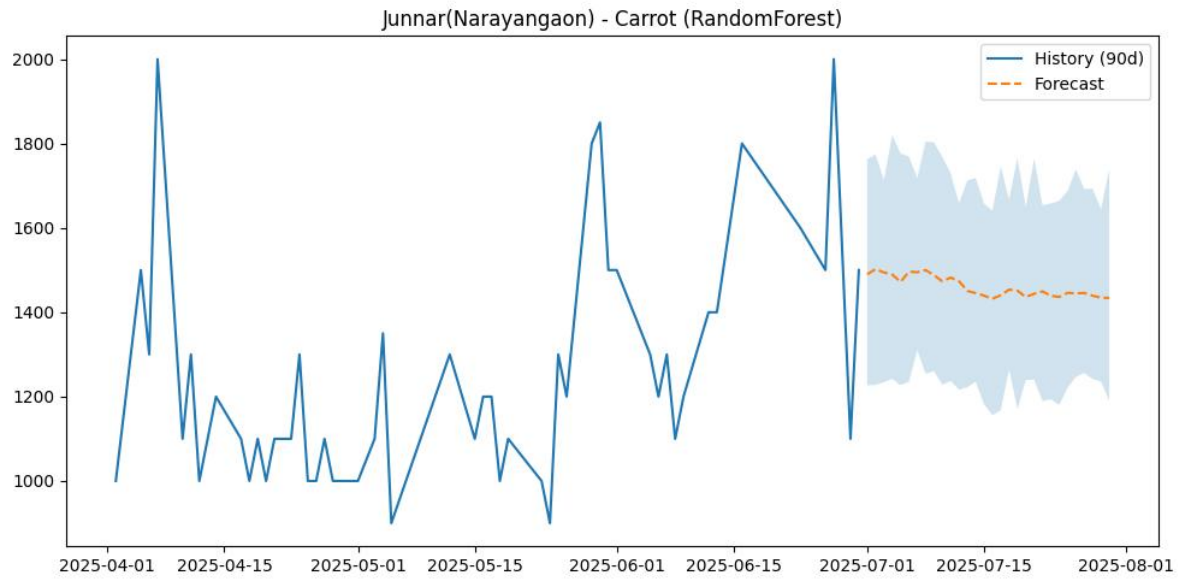
except Exception:

    model_scores['SVR'].append(np.nan
```


4. Random Forest

- **Random Forest (RF)**, as the name suggests, is made up of a large number of individual decision trees. A decision tree is a simple model that makes predictions by asking a series of "if-then" questions about the data. A single decision tree can easily overfit the data, meaning it learns the noise in the training data too well and doesn't generalize to new data. Random Forest gets around this by introducing randomness in two ways. First, each tree is built using only a random sample of the original data. Second, at each split in the tree, it only considers a random subset of the features. The final prediction from the Random Forest is simply the average of the predictions from all the individual trees. This averaging process makes the final model much more robust and accurate than any single tree would be.
- **Advantages :**
Random Forest has several advantages when applied to commodity price prediction. One of its strongest benefits is its ability to capture nonlinear relationships between variables, which is important because commodity prices are influenced by a wide range of factors such as demand fluctuations, weather conditions, seasonal patterns, and even sudden policy changes. Unlike simple linear models, Random Forest can uncover complex interactions between these features, giving it an edge in predictive accuracy for such volatile data. Another advantage is its robustness against overfitting compared to single decision trees. By averaging the predictions of many trees, it reduces variance and provides more stable outputs even when the input data is noisy, as is often the case with market-reported mandi prices. Random Forest also works effectively with high-dimensional datasets, making it well-suited when additional features like temperature, rainfall, or festival indicators are incorporated into the dataset. Moreover, it provides feature importance scores, which help in understanding which variables—say rainfall or month of the year—play a major role in influencing commodity prices. These qualities make Random Forest a reliable model for structured tabular data such as commodity price datasets.
- **Disadvantages:**
Random Forest also comes with its share of limitations. The most significant drawback is that it is not inherently designed for sequential or time-series data, which is at the core of commodity price forecasting. Unlike models such as ARIMA or LSTM, Random Forest does not account for temporal dependencies like trends, lags, or seasonality. To use it effectively, one must manually engineer lag features or add temporal indicators, which increases complexity. Additionally, Random Forest can become computationally intensive when applied to large datasets covering multiple commodities and markets over long time periods, consuming more time and memory resources. Interpretability is another challenge, as even though feature importance can be extracted, the internal working of the model remains a black box compared to statistical models where coefficients clearly indicate the relationship between variables. Another limitation is that Random Forest struggles with extrapolation: if future prices rise or fall outside the historical range due to sudden shocks such as export bans or crop failures, the model may fail to predict these movements accurately since it mostly relies on patterns observed in the past. Finally, because the algorithm tends to favor majority patterns, it may underpredict rare but critical price spikes or drops, which are often the most important events for farmers and policymakers to anticipate.

- **Plots**



try:

```
best_params = optuna_tune("RandomForest", X_train_feat, y_train_ml, n_trials=n_trials) or {}
model_rf = RandomForestRegressor(**best_params, random_state=42)
model_rf.fit(X_train_feat, y_train_ml)
fc_series = iterative_forecast_ml(model_rf, y_train, exog_train, exog_test, len(y_test), LAGS,
feature_cols)
model_scores['RandomForest'].append(rmse(y_test, fc_series))
last_fold_preds['RandomForest'] = fc_series
```

except Exception:

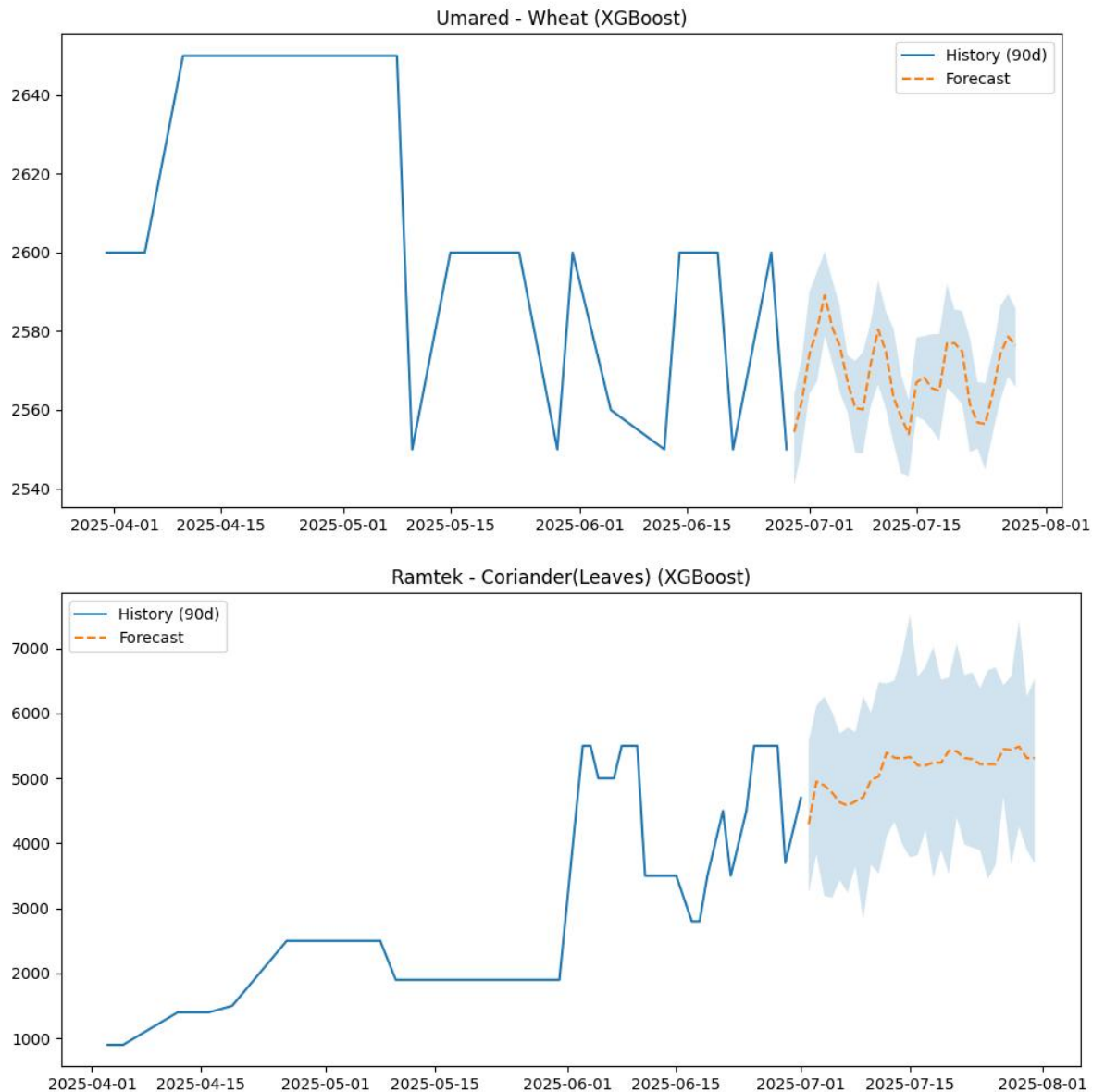
```
model_scores['RandomForest'].append(np.nan)
```

5. XGBoost

- **Extreme Gradient Boosting (XGBoost)** is another ensemble method that uses decision trees, but it builds them in a different way. Instead of building all the trees independently, XGBoost builds them one after another. The first tree makes a prediction, and then the second tree is trained to correct the errors that the first tree made. The third tree then corrects the errors of the first two combined, and so on. This process is called "boosting." XGBoost is a particularly powerful and efficient implementation of this idea. It includes some clever optimizations and a regularization feature that helps prevent overfitting, which is why it's often the winning model in many machine learning competitions.
- **Advantages:**
XGBoost offers several advantages when applied to commodity price prediction. One of its key strengths is its ability to handle nonlinear relationships and complex interactions between variables with remarkable efficiency. This is particularly valuable in agricultural markets, where prices are shaped by a combination of supply-demand imbalances, weather fluctuations, seasonality, and external shocks such as government policies or international trade changes. Unlike traditional statistical models, XGBoost can incorporate many explanatory features—such as lagged prices, rainfall, temperature, or festival periods—while still maintaining strong predictive performance. It is also designed for scalability and speed, making it well-suited for large datasets that cover multiple commodities, mandis, and years of data. Another advantage is its built-in mechanisms for regularization, which help control overfitting and ensure that the model generalizes well to unseen data. Furthermore, XGBoost provides feature importance measures and allows interpretability through techniques such as SHAP values, enabling analysts to better understand which factors are driving price predictions. These qualities make XGBoost an appealing choice for commodity price forecasting projects that demand both accuracy and the ability to integrate diverse features.
- **Disadvantage:**
Despite its strengths, XGBoost also has limitations in this domain. Like Random Forest, it is not inherently designed for time-series forecasting, and therefore does not directly capture sequential patterns such as trends, lags, and seasonality. To apply it effectively, significant feature engineering is required—for example, generating lag variables, moving averages, or seasonal indicators. Without such enhancements, the model may fail to capture the temporal structure of commodity prices. Additionally, XGBoost can be sensitive to hyperparameter tuning; achieving the best performance often requires careful experimentation with settings such as learning rate, maximum depth, and number of trees. This tuning process can be computationally expensive and time-consuming, especially on large datasets. Another challenge is that while XGBoost provides strong predictive accuracy, it remains a complex and somewhat opaque model compared to simpler statistical approaches, which can make it harder for policymakers or farmers to fully trust or interpret its predictions. Finally, as with other tree-based ensemble methods, XGBoost does not extrapolate well; if future commodity

prices fall outside historical ranges due to sudden disruptions, the model may struggle to predict those extreme values accurately.

- **Plots**



if HAS_XGB:

try:

```
best_params = optuna_tune("XGBoost", X_train_feat, y_train_ml, n_trials=n_trials) or
{}
```

```
model_xgb = XGBRegressor(**best_params, random_state=42, verbosity=0)
```

```
model_xgb.fit(X_train_feat, y_train_ml)
```

```
fc_series = iterative_forecast_ml(model_xgb, y_train, exog_train, exog_test, len(y_test),
LAGS, feature_cols)
```

```
model_scores['XGBoost'].append(rmse(y_test, fc_series))

last_fold_preds['XGBoost'] = fc_series

except Exception:

    model_scores['XGBoost'].append(np.nan)
```

6. Facebook's Prophet

Prophet is a forecasting tool developed by the data science team at Facebook. It was designed to be easy to use and to work well on time series data that has strong seasonal patterns, like you might see in business data.

Prophet works by breaking a time series down into a few main components:

- **A trend component:** This captures the overall direction the data is moving in over the long term. Prophet is smart about this and can automatically detect points where the trend changes.
- **A seasonality component:** This models repeating patterns, like weekly cycles (e.g., sales are lower on weekends) and yearly cycles (e.g., prices are higher in winter). It uses a flexible method called Fourier series to capture these patterns.
- **A holiday component:** This allows you to tell the model about special events, like holidays, that can have a big impact on the data but don't follow a regular seasonal pattern.

The model then simply adds these components together to make its forecast. Because it's essentially a curve-fitting exercise, Prophet is good at handling missing data and is not too sensitive to outliers, making it a very practical and robust tool.

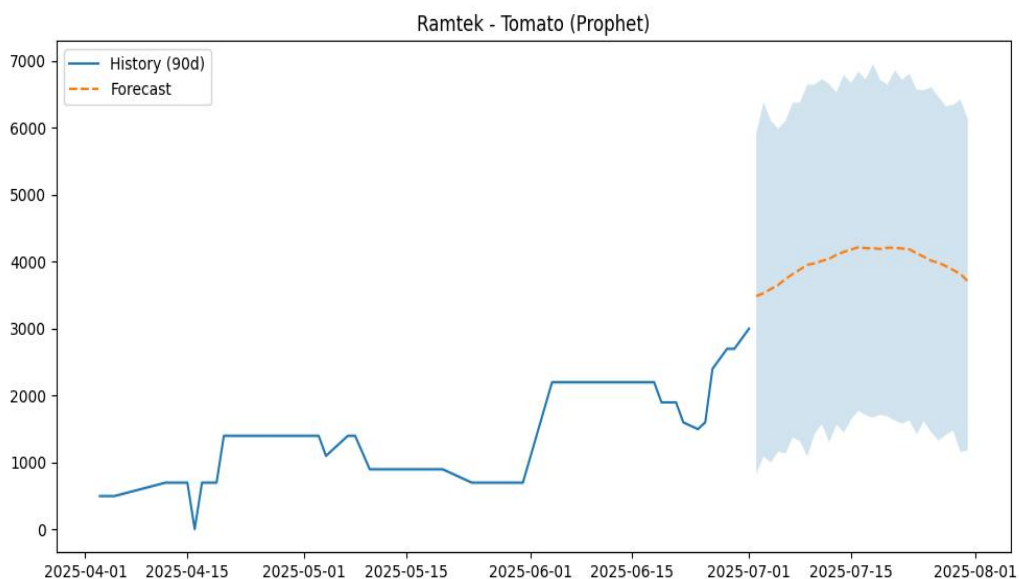
- **Prophet:**
Prophet has clear advantages when applied to commodity price prediction, especially because it is specifically designed for time-series data. One of its main strengths is its ability to automatically model trend, seasonality, and holiday effects, which are highly relevant in agricultural markets where prices follow seasonal cycles and are influenced by events like monsoon onset, harvest periods, or festivals. Prophet's additive model framework allows for flexible decomposition of the price series into interpretable components, making it easier to understand the underlying drivers of price movement. For example, it can clearly separate long-term growth or decline in commodity prices from seasonal effects such as higher demand during festive seasons. Another advantage is that Prophet is user-friendly and requires minimal feature engineering compared to machine learning models like Random

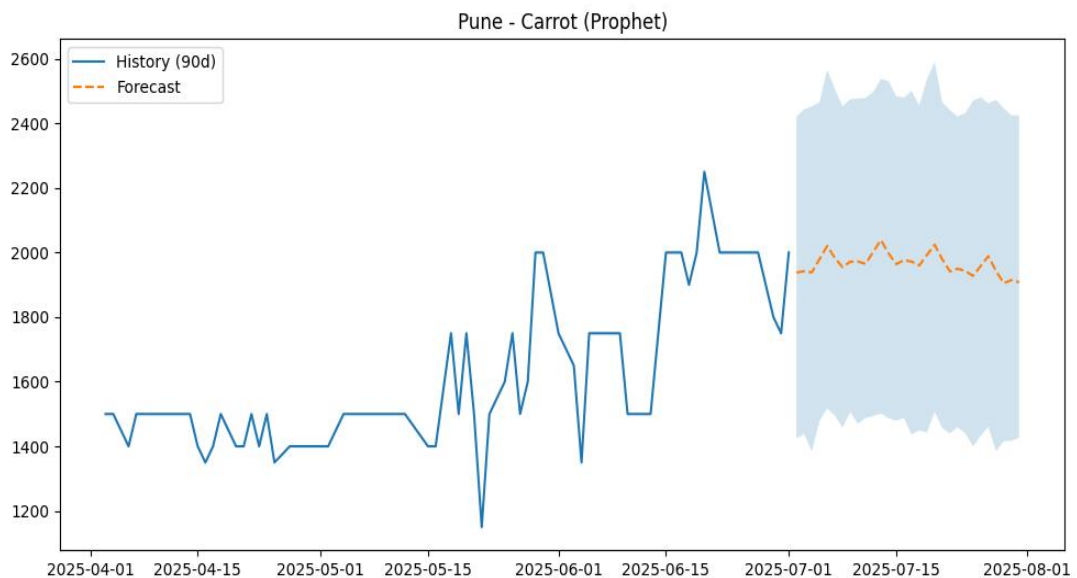
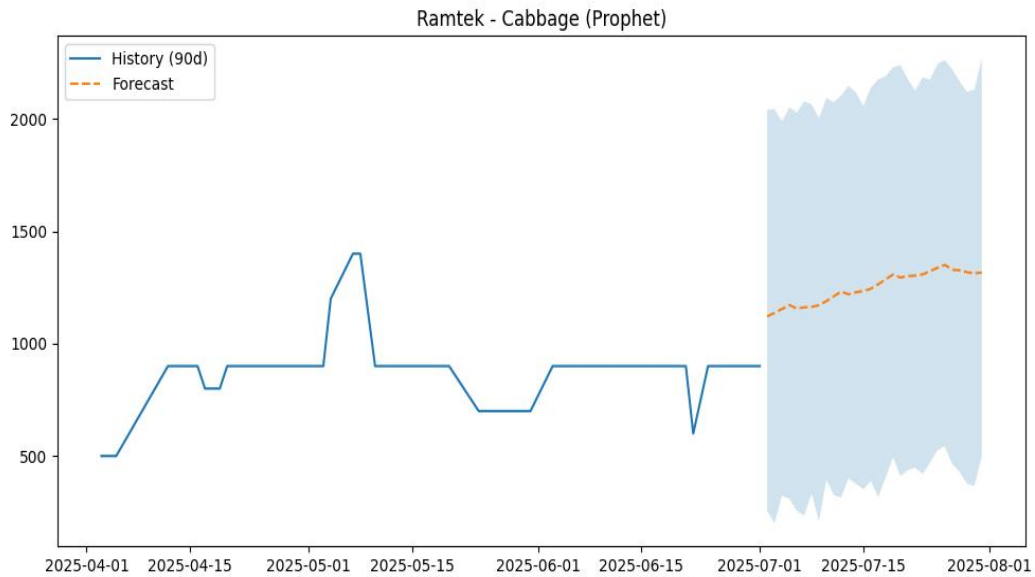
Forest or XGBoost. With only the date and target variable as mandatory inputs, it can deliver reliable forecasts quickly. It is also robust to missing data and outliers, which are common in mandi price records, and can produce forecast intervals that help quantify uncertainty—a valuable feature for farmers and policymakers who need to plan under risk. Prophet is particularly well-suited for projects that need both accuracy and interpretability, as it provides clear visualizations of trends and seasonal patterns along with its forecasts.

- **Disadvantage:**

Prophet also comes with notable disadvantages in the context of commodity price prediction. While it handles seasonality and trends effectively, it may struggle with sudden shocks or irregular fluctuations that do not follow past patterns, such as government interventions, export bans, or unexpected weather events. Unlike models such as XGBoost or deep learning methods, Prophet does not capture nonlinear interactions between external variables unless those variables are explicitly added as regressors. This means that to account for factors like rainfall, temperature, or demand spikes, additional feature engineering and careful configuration are required. Another limitation is that Prophet's performance can be inconsistent when the dataset is small or when seasonality is weak, sometimes producing overly smooth forecasts that miss short-term volatility. In addition, while it is relatively simple to use, fine-tuning Prophet's parameters (such as changepoint prior scale or seasonality mode) is often necessary to achieve optimal accuracy, and this can be challenging without sufficient expertise. Compared to ensemble or deep learning models, Prophet may also underperform in highly complex datasets with many interacting influences.

- **Plots**





```
if HAS_PROPHET:
    try:
        best_s = float('inf'); best_fc = None
        for cps, sps in [(0.01,0.1),(0.1,1.0),(0.5,10.0)]:
            try:
                dfp = y_train.reset_index().rename(columns={'Arrival_Date':'ds','Modal_Price':'y'})
                m_prop = Prophet(changepoint_prior_scale=cps, seasonality_prior_scale=sps)
                m_prop.fit(dfp)
```

```

future = pd.DataFrame({'ds': y_test.index})

fc = m_prop.predict(future)['yhat'].values

s = rmse(y_test, fc)

if s < best_s:

    best_s = s; best_fc = fc

except Exception:

    continue

model_scores['Prophet'].append(best_s)

last_fold_preds['Prophet'] = pd.Series(best_fc, index=y_test.index)

except Exception:

    model_scores['Prophet'].append(np.nan)

```

Time Series Split Cross Validation

One of the most important methodological choices in time series forecasting is the strategy used for model validation. In conventional machine learning tasks, k-fold cross-validation is often applied by randomly splitting the dataset into training and validation folds. However, random partitioning is fundamentally unsuitable for time series data because it violates the temporal ordering of observations.

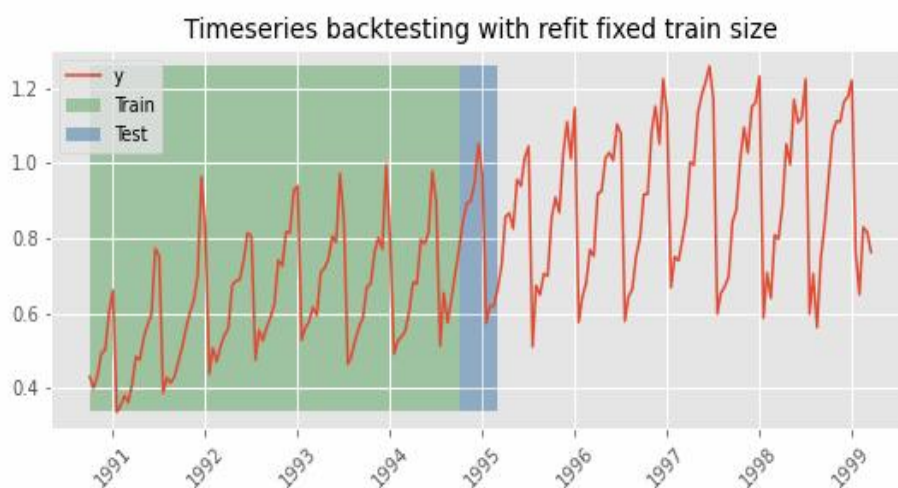
A forecasting model must always be trained on the past and tested on the future. If training data includes observations from the future relative to the validation period, information leakage occurs, leading to overly optimistic performance estimates.

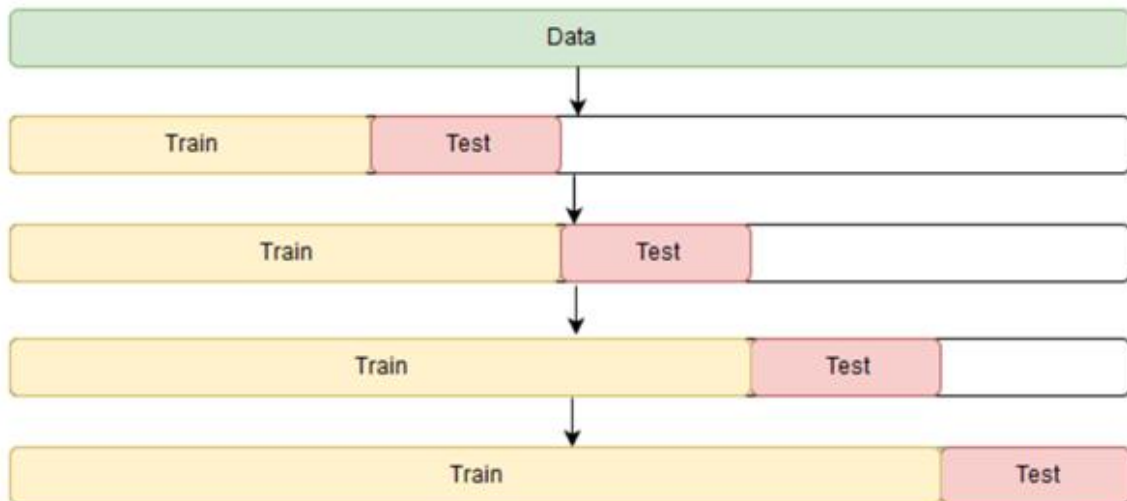
To avoid this problem, we adopted a cross-validation strategy designed specifically for time series, known as rolling or forward chaining cross-validation. This method respects temporal order by using a series of expanding or sliding training windows, each followed by a validation horizon. For our project, the validation horizon was fixed at thirty days, reflecting the real-world forecasting task of producing one-month-ahead predictions. By aligning the validation window with the operational horizon, we ensured that model evaluation directly measured performance under deployment conditions.

In practice, the dataset for each market–commodity pair was split into several folds using scikit-learn’s **TimeSeriesSplit**. For each fold, the model was trained on all data available up to a certain cutoff point and then validated on the subsequent thirty days. The cutoff was then moved forward, and the process was repeated, producing multiple train–test splits. This approach mimics the real-time use of a forecasting system, where each month the model is updated with the latest data and used to predict the next period.

Two design choices were especially important in structuring this procedure.

- The first was whether to use expanding or sliding windows. In the expanding window variant, each training set grows larger with each fold, accumulating all past data. This approach allows the model to learn from the maximum amount of history and is particularly useful for stable commodities where long-term patterns carry predictive value. In the sliding window variant, the training set is fixed to a certain length and the oldest data is discarded as the window moves forward. This is better suited for commodities subject to structural changes, where very old data may no longer be relevant. In our implementation, we primarily used the expanding window approach because it preserved all historical information, but we also experimented with sliding windows for highly volatile commodities to test whether discarding outdated data improved accuracy.
- The second key design choice was the number of folds. Too few folds limit the ability to generalize performance across different periods, while too many folds increase computation. We settled on three to five folds for most market–commodity pairs, depending on data availability. For commodities with long historical records, five folds gave a more robust estimate of performance. For those with shorter series, three folds were used to preserve sufficient data for training in each iteration. Each fold therefore represented a distinct forecasting exercise, with the model evaluated on unseen future periods.
- This rolling validation framework provided a more realistic and conservative measure of model performance than a single train–test split. For example, a model might perform well on one thirty-day window but poorly on others. By averaging performance across multiple folds, we reduced the risk of overfitting to a specific period. Moreover, the variability of error metrics across folds gave us an indication of model stability. A model with consistent performance across all folds was considered more reliable than one with large fluctuations.
- An additional advantage of this method was its alignment with practical deployment. Stakeholders will not evaluate the model once and for all but will rely on it to produce forecasts repeatedly over time. Rolling cross-validation therefore mirrored not only the data structure but also the decision-making context in which the forecasts would be used. It ensured that when we selected the “best” model for a market–commodity pair, that model was one that had demonstrated stable performance across multiple forward-looking scenarios.





Performance Evaluation Metrics

Model evaluation relied on multiple error metrics to ensure robust comparison. Root Mean Squared Error (RMSE) was the primary metric because it penalizes large deviations more heavily than small ones. This was appropriate in agricultural contexts where extreme forecasting errors can have disproportionate consequences. RMSE was defined mathematically as:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

We also computed Mean Absolute Error (MAE), which provides a more interpretable measure of average error magnitude, and Mean Absolute Percentage Error (MAPE), which expresses errors in relative terms. These secondary metrics allowed us to cross-check that models with low RMSE did not perform poorly in terms of average or relative errors.

MAPE was defined mathematically as:

$$MAPE = \frac{1}{n} \sum_{t=1}^n \left| \frac{A_t - F_t}{A_t} \right|$$

MAE was defined mathematically as:

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

Hyperparameter Tuning with Optuna

Hyperparameter tuning was a critical step in ensuring that the machine learning models in our pipeline achieved their best possible performance. Unlike statistical models such as AutoARIMA and ETS, which rely primarily on time series properties and information criteria, machine learning models depend heavily on the correct configuration of hyperparameters. Poorly chosen hyperparameters can lead to severe underfitting or overfitting, degrading forecast accuracy. To address this, we integrated Optuna, a modern hyperparameter optimization framework that uses efficient search strategies to navigate large parameter spaces.

Optuna improves upon naive grid search or random search by using a technique called sequential model-based optimization. Instead of testing all possible parameter combinations, which is computationally expensive, Optuna builds a probabilistic model of the relationship between hyperparameters and model performance. It then uses this surrogate model to suggest promising sets of parameters in subsequent trials. As more trials are conducted, Optuna refines its understanding of the search space, balancing exploration of new regions with exploitation of known good regions. This process allowed us to find near-optimal hyperparameter configurations within a reasonable computational budget.

We applied Optuna tuning primarily to XGBoost, Random Forest, and Support Vector Regression, as these were the models most sensitive to parameter choices. Each model required a carefully defined search space based on parameters that strongly influence performance.

1.XGBoost Tuning

XGBoost is a gradient boosting framework that constructs an ensemble of decision trees sequentially, with each tree correcting the errors of the previous one. Its performance is governed by a wide range of hyperparameters.

- In our tuning process, the most influential parameters included the maximum depth of each tree, which controls how complex each individual learner can be, and the learning rate, which scales the contribution of each new tree.
- A high learning rate allows the model to fit quickly but risks overfitting, while a low learning rate requires more trees but often results in more stable generalization.
- We also tuned the number of estimators, representing the total number of boosting rounds. Too few estimators can cause underfitting, while too many can lead to unnecessary complexity.
- Subsampling parameters were equally important. The subsample parameter determines the fraction of training samples used to grow each tree, while colsample_bytree controls the fraction of features considered at each split. These parameters act as regularizers, improving generalization by introducing randomness.

By tuning these parameters with Optuna, we ensured that the XGBoost models adapted effectively to the diverse characteristics of different commodities and markets. For example, highly volatile commodities benefited from shallower trees with lower learning rates, which prevented overfitting to local noise, while more stable commodities tolerated deeper trees and more estimators.

2.Random Forest Tuning

Random Forest is less sensitive to hyperparameters than XGBoost, but its performance still depends on the correct balance of tree complexity and ensemble size.

- The most critical parameter is the number of trees, or `n_estimators`. Too few trees result in high variance, while too many trees increase computational cost without proportional improvement. We typically searched between 50 and 500 trees depending on the dataset size.
- The depth of each tree, controlled by `max_depth`, was also tuned. Shallow trees risk missing complex nonlinear relationships, while very deep trees can overfit to noise. Since Random Forest averages across many trees, deeper structures can sometimes be tolerated, but careful tuning was necessary to avoid unnecessary variance.
- The `max_features` parameter, which specifies the number of features considered when splitting each node, played a significant role in balancing bias and variance. Lower values encourage diversity among trees, while higher values allow more consistent and powerful splits.

Through Optuna's sequential optimization, we identified configurations that balanced ensemble size and depth. For instance, in markets with larger datasets, deeper trees with more features improved accuracy, while in smaller markets, shallower trees prevented overfitting.

3.Support Vector Regression Tuning

Support Vector Regression requires careful calibration of kernel and margin parameters to perform well. Unlike tree-based models, which can learn directly from raw feature interactions, SVR relies on kernel transformations to capture nonlinearity. The three most important parameters are **C**, **epsilon**, and **gamma**.

- The C parameter controls the penalty for errors outside the margin of tolerance. A small value of C results in a wider margin but potentially higher bias, while a large C enforces stricter fitting of the training data at the risk of overfitting.
- The epsilon parameter defines the size of the margin of tolerance within which errors are not penalized. Choosing epsilon too small makes the model sensitive to noise, while too large can lead to underfitting.
- Finally, gamma is relevant when using the radial basis function (RBF) kernel, and it controls the influence of single training points. A small gamma makes the kernel smoother, capturing only large-scale trends, while a large gamma allows the model to capture fine details but risks fitting noise.

Optuna's search process helped balance these parameters automatically. For commodities with stable and smooth series, smaller values of C and moderate epsilon produced good forecasts. For volatile series with abrupt changes, higher values of C combined with smaller epsilon captured sudden movements more effectively.

In all cases, the tuning process was guided by cross-validated RMSE as the optimization objective. Optuna was allowed to run for a limited number of trials per model to maintain computational feasibility, typically between 20 and 50 trials depending on the dataset size. By systematically exploring the parameter space, we ensured that each model family was represented at or near its best performance level rather than being compared on default configurations. This allowed our model comparison to be both fair and rigorous.

```
def optuna_tune(model_name, X, y, n_trials=OPTUNA_TRIALS):
```

```
    if len(X) < 10:
```

```
        return None
```

```

split = int(len(X)*0.8)
X_tr, X_val = X.iloc[:split], X.iloc[split:]
y_tr, y_val = y.iloc[:split], y.iloc[split:]

def objective(trial):
    if model_name == "XGBoost":
        params = {
            "max_depth": trial.suggest_int("max_depth", 2, 8),
            "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.3),
            "n_estimators": trial.suggest_int("n_estimators", 50, 300),
            "subsample": trial.suggest_float("subsample", 0.5, 1.0),
            "colsample_bytree": trial.suggest_float("colsample_bytree", 0.5, 1.0),
            "random_state": 42
        }
        model = XGBRegressor(**params, verbosity=0)
    elif model_name == "RandomForest":
        params = {
            "n_estimators": trial.suggest_int("n_estimators", 50, 300),
            "max_depth": trial.suggest_int("max_depth", 3, 15),
            "max_features": trial.suggest_categorical("max_features", ["sqrt", "log2", None]),
            "random_state": 42
        }
        model = RandomForestRegressor(**params)
    elif model_name == "SVR":
        params = {
            "C": trial.suggest_float("C", 0.1, 50.0, log=True),
            "epsilon": trial.suggest_float("epsilon", 0.001, 1.0, log=True),
            "gamma": trial.suggest_categorical("gamma", ["scale", "auto"])
        }
        model = SVR(**params)
    else:
        raise ValueError("Unsupported model")

```

```
if model_name == "SVR":
    scaler = StandardScaler().fit(X_tr)
    model.fit(scaler.transform(X_tr), y_tr)
    preds = model.predict(scaler.transform(X_val))
else:
    model.fit(X_tr, y_tr)
    preds = model.predict(X_val)
return rmse(y_val, preds)

study = optuna.create_study(direction="minimize")
study.optimize(objective, n_trials=n_trials, show_progress_bar=False)
return study.best_params
```

Model Building

```

# Mount Drive (Colab)
from google.colab import drive
drive.mount('/content/drive')

# Imports
import os, time, math, warnings, uuid, joblib, pickle, multiprocessing as mp, glob
from functools import partial
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.ensemble import RandomForestRegressor
from sklearn.svm import SVR
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error
from sklearn.model_selection import TimeSeriesSplit
import optuna

from statsmodels.tsa.holtwinters import ExponentialSmoothing
from statsmodels.tsa.seasonal import STL
from statsmodels.tsa.stattools import adfuller

warnings.filterwarnings("ignore")
%matplotlib inline

```

These are the libraries imported.

```

CSV_PATH = "/content/drive/MyDrive/agric.csv"
OUTPUT_DIR = "/content/drive/MyDrive/agric_forecast_results_run"
PLOT_DIR = os.path.join(OUTPUT_DIR, "plots")
MODEL_DIR = os.path.join(OUTPUT_DIR, "models")
PARTIAL_DIR = os.path.join(OUTPUT_DIR, "partials")
for d in [OUTPUT_DIR, PLOT_DIR, MODEL_DIR, PARTIAL_DIR]:
    os.makedirs(d, exist_ok=True)

MASTER_FINAL = os.path.join(OUTPUT_DIR, "master_summary_cv.csv")
BEST_FINAL = os.path.join(OUTPUT_DIR, "best_model_summary.csv")
FORECAST_FINAL = os.path.join(OUTPUT_DIR, "future_forecast.csv")
forecast_ckpt = os.path.join(PARTIAL_DIR, "future_forecast_partial.csv")

# Parameters
MIN_OBS = 365
LAGS = [1, 2, 3, 7, 14, 21, 28]
N_CV_SPLITS = 2
ETS_SEASONAL_PERIODS = 7
OPTUNA_TRIALS = 10 # Number of tuning trials for XGBoost, RandomForest, and SVR
HORIZON = 30
RANDOM_STATE = 42
N_WORKERS = 2
AUTO_MERGE_EVERY = 10
BOOTSTRAP_N = 200
ALPHA = 0.05

np.random.seed(RANDOM_STATE)

```

```

def sanitize_name(s):
    return "".join(c if c.isalnum() or c in (" ", "_", "-") else "_" for c in str(s)).replace(" ", "_")

```

- Cleans market/commodity names for filenames.
- Replaces spaces and bad characters with underscores.

- Used when saving models, plots, and artifacts.

```
def rmse(y_true, y_pred): return math.sqrt(mean_squared_error(y_true, y_pred))
def mae(y_true, y_pred): return mean_absolute_error(y_true, y_pred)
def mape(y_true, y_pred):
    y_true, y_pred = np.array(y_true), np.array(y_pred)
    mask = y_true != 0
    if mask.sum() == 0:
        return np.nan
    return np.mean(np.abs((y_true[mask] - y_pred[mask]) / y_true[mask])) * 100
```

- `rmse(y_true, y_pred)`
 - Returns Root Mean Squared Error.
 - Penalizes large errors strongly.
 - Common error metric in forecasting.
- `mae(y_true, y_pred)`
 - Returns Mean Absolute Error.
 - Easier to interpret, doesn't penalize large errors as harshly.
 - Good when outliers aren't critical.
- `mape(y_true, y_pred)`
 - Mean Absolute Percentage Error (%).
 - Shows relative error in percentage terms.
 - Useful for comparing across commodities with different scales.

```
def adf_pvalue(series):
    try:
        return adfuller(series.dropna())[1]
    except Exception:
        return np.nan

def ensure_stationary(y):
    try:
        pval = adfuller(y.dropna())[1]
        if pval > 0.05:
            return y.diff().dropna()
        return y
    except Exception:
        return y
```

- `adf_pvalue(series)`
 - Runs Augmented Dickey-Fuller test for stationarity.
 - Returns p-value → small = stationary.
 - Used only for reference, not for forcing differencing.
- `ensure_stationary(y)`
 - If series is not stationary (ADF $p > 0.05$), it differences the series once.
 - Returns the transformed series.

- Used for ETS final fitting where stationarity matters.

```
def clean_full_df(df):
    df = df.copy()
    df = df.drop_duplicates()
    df['Modal_Price'] = pd.to_numeric(df['Modal_Price'], errors='coerce')
    if 'Rainfall_mm' in df.columns:
        df['Rainfall_mm'] = pd.to_numeric(df['Rainfall_mm'], errors='coerce')
    if 'Avg_Temp_C' in df.columns:
        df['Avg_Temp_C'] = pd.to_numeric(df['Avg_Temp_C'], errors='coerce')
    df = df.dropna(subset=['Arrival_Date', 'Market', 'Commodity', 'Modal_Price'])
    df['Arrival_Date'] = pd.to_datetime(df['Arrival_Date'], errors='coerce')
    df = df.dropna(subset=['Arrival_Date'])
    low_q = df['Modal_Price'].quantile(0.005)
    high_q = df['Modal_Price'].quantile(0.995)
    df['Modal_Price'] = df['Modal_Price'].clip(lower=low_q, upper=high_q)
    return df
```

```
def make_lag_features(series, exog_df, lags):
    X = pd.DataFrame(index=series.index)
    for lag in lags:
        X[f"lag_{lag}"] = series.shift(lag)
    X["roll_mean_7"] = series.shift(1).rolling(7).mean()
    if exog_df is not None and not exog_df.empty:
        X = X.join(exog_df)
    X = X.dropna()
    y_aligned = series.loc[X.index]
    return X, y_aligned
```

```

def iterative_forecast_ml(model, y_train, exog_train, exog_future, horizon, lags, feature_cols, scaler=None):
    history = y_train.copy().astype(float)
    full_exog = None
    if exog_train is not None and exog_future is not None:
        full_exog = pd.concat([exog_train, exog_future])
    elif exog_train is not None:
        full_exog = exog_train
    elif exog_future is not None:
        full_exog = exog_future
    preds = []
    for h in range(1, horizon+1):
        idx = history.index[-1] + pd.Timedelta(days=1)
        feat = {}
        for lag in lags:
            look_idx = idx - pd.Timedelta(days=lag)
            if look_idx in history.index:
                feat[f"lag_{lag}"] = history.loc[look_idx]
            else:
                feat[f"lag_{lag}"] = history.iloc[-1]
        feat['roll_mean_7'] = history.iloc[-7:].mean() if len(history) >= 7 else history.mean()
        if full_exog is not None:
            for c in full_exog.columns:
                if idx in full_exog.index:
                    feat[c] = full_exog.loc[idx, c]
                else:
                    feat[c] = full_exog.iloc[-1][c] if len(full_exog) > 0 else np.nan
        for c in feature_cols:
            if c not in feat:
                feat[c] = np.nan
        feat_df = pd.DataFrame([feat], index=[idx])[feature_cols]
        feat_df = feat_df.fillna(method='ffill').fillna(method='bfill').fillna(0.0)
        X_pred = scaler.transform(feat_df) if scaler is not None else feat_df
        p = model.predict(X_pred)
        pval = float(p[0]) if hasattr(p, '__len__') else float(p)
        preds.append(pval)
        history.loc[idx] = pval
    idxs = pd.date_range(start=y_train.index[-1] + pd.Timedelta(days=1), periods=horizon, freq='D')
    return pd.Series(preds, index=idxs)

```

```

def bootstrap_intervals(pred_series, residuals, horizon=30, n_boot=200, alpha=0.05):
    pred_vals = np.array(pred_series)
    boot = []
    for _ in range(n_boot):
        noise = np.random.choice(residuals, size=horizon, replace=True)
        boot.append(pred_vals + noise)
    boot = np.array(boot)
    lower = np.percentile(boot, 100*alpha/2, axis=0)
    upper = np.percentile(boot, 100*(1-alpha/2), axis=0)
    lower = np.maximum(lower, 0.0)
    return lower, upper

```

```

def select_best_metric(scores):
    errs = {}
    for k in ['rmse', 'mae', 'mape']:
        v = scores.get(k, np.nan)
        if not np.isnan(v):
            errs[k.upper()] = v
    if not errs:
        return None
    best_err = min(errs, key=errs.get)
    vals = list(errs.values())
    if max(vals) - min(vals) < 0.05 * min(vals):
        return "Balanced"
    return best_err

```

```

def optuna_tune(model_name, X, y, n_trials=OPTUNA_TRIALS, random_state=RANDOM_STATE):
    if not HAS_OPTUNA or n_trials <= 0 or len(X) < 10:
        return None
    split = int(len(X)*0.8)
    X_tr, X_val = X.iloc[:split], X.iloc[split:]
    y_tr, y_val = y.iloc[:split], y.iloc[split:]

    def objective(trial):
        if model_name == "XGBoost":
            params = {
                "max_depth": trial.suggest_int("max_depth", 2, 8),
                "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.3),
                "n_estimators": trial.suggest_int("n_estimators", 50, 300),
                "subsample": trial.suggest_float("subsample", 0.5, 1.0),
                "colsample_bytree": trial.suggest_float("colsample_bytree", 0.5, 1.0),
                "random_state": random_state
            }
            m = XGBRegressor(**params, verbosity=0)
            m.fit(X_tr, y_tr)
            preds = m.predict(X_val)
            return rmse(y_val, preds)
        elif model_name == "RandomForest":
            params = {
                "n_estimators": trial.suggest_int("n_estimators", 50, 300),
                "max_depth": trial.suggest_int("max_depth", 3, 20),
                "max_features": trial.suggest_categorical("max_features", ["sqrt", "log2", None]),
                "random_state": random_state
            }
            m = RandomForestRegressor(**params)
            m.fit(X_tr, y_tr)
            preds = m.predict(X_val)
            return rmse(y_val, preds)
        elif model_name == "SVR":
            params = {
                "C": trial.suggest_float("C", 0.1, 50.0, log=True),
                "epsilon": trial.suggest_float("epsilon", 0.001, 1.0, log=True),
                "gamma": trial.suggest_categorical("gamma", ["scale", "auto"])
            }
            scaler = StandardScaler().fit(X_tr)
            m = SVR(**params)
            m.fit(scaler.transform(X_tr), y_tr)
            preds = m.predict(scaler.transform(X_val))
            return rmse(y_val, preds)
    study = optuna.create_study(direction="minimize", sampler=optuna.samplers.TPESampler(seed=random_state))
    study.optimize(objective, n_trials=n_trials, show_progress_bar=False)
    return study.best_params

```

```

def run_pair_worker(args):
    market, commodity, df_full = args
    uid = f"{os.getpid()}_{uuid.uuid4().hex[:8]}"
    try:
        sub_raw = df_full[(df_full['Market'] == market) & (df_full['Commodity'] == commodity)].copy()
        if len(sub_raw) < MIN_OBS:
            return {"status": "skip", "market": market, "commodity": commodity, "reason": "too_few_obs", "count": len(sub_raw)}

        sub_raw['Arrival_Date'] = pd.to_datetime(sub_raw['Arrival_Date'])
        sub_agg = sub_raw.groupby('Arrival_Date').agg({'Modal_Price': 'median', 'Rainfall_mm': 'mean', 'Avg_Temp_C': 'mean'}).asfreq('D').interpolate()
        y = sub_agg['Modal_Price'].dropna()
        if y.empty:
            return {"status": "skip", "market": market, "commodity": commodity, "reason": "empty_after_agg"}

        exog = sub_agg[['Rainfall_mm', 'Avg_Temp_C']].loc[y.index] if 'Rainfall_mm' in sub_agg.columns else None

        # STL plot (optional)
        try:
            stl = STL(y, period=7).fit()
            fig = stl.plot()
            plt.suptitle(f"STL - {market} - {commodity}")
            pfn = os.path.join(PLOT_DIR, f"STL_{sanitize_name(market)}_{sanitize_name(commodity)}.png")
            plt.tight_layout(); plt.savefig(pfn, bbox_inches='tight'); plt.close()
        except Exception:
            pass

```


- **run_pair_worker** trains all candidate models on one market–commodity pair with CV, averages their errors, selects the best model, and saves both metrics and the trained artifact.

```
# Cross-validation
tscv = TimeSeriesSplit(n_splits=N_CV_SPLITS, test_size=30)
model_scores = {m: {"rmse": [], "mae": [], "mape": []} for m in ['AutoARIMA', 'ETS', 'Prophet', 'XGBoost', 'RandomForest', 'SVR']}
last_preds = {}

for train_idx, test_idx in tscv.split(y):
    y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]
    exog_train = exog.iloc[train_idx] if exog is not None else None
    exog_test = exog.iloc[test_idx] if exog is not None else None
```

- This block sets up rolling time-series cross-validation, splits the target and exogenous data into train/test folds, and initializes structures to store each model's errors and last-fold predictions.

```
# ----- AutoARIMA -----
if HAS_PMD:
    try:
        m_ar = pm.auto_arima(y_train, exogenous=exog_train, seasonal=True, m=7,
                             stepwise=True, suppress_warnings=True, error_action='ignore',
                             max_p=2, max_q=2)
        fc = m_ar.predict(n_periods=len(y_test), exogenous=exog_test)
        model_scores['AutoARIMA']['rmse'].append(rmse(y_test, fc))
        model_scores['AutoARIMA']['mae'].append(mae(y_test, fc))
        model_scores['AutoARIMA']['mape'].append(mape(y_test, fc))
        last_preds['AutoARIMA'] = pd.Series(fc, index=y_test.index)
    except Exception:
        pass

# ----- ETS (candidates) -----
try:
    best_fc = None; best_err = np.inf
    for cand in [{'trend': 'add', 'seasonal': 'add'},
                  {'trend': 'add', 'seasonal': 'None'},
                  {'trend': 'None', 'seasonal': 'add'}]:
        try:
            m_ets = ExponentialSmoothing(y_train, trend=cand['trend'], seasonal=cand['seasonal'],
                                          seasonal_periods=ETS_SEASONAL_PERIODS).fit()
            fc = m_ets.forecast(len(y_test))
            s = rmse(y_test, fc)
            if s < best_err:
                best_err = s; best_fc = fc
        except Exception:
            continue
    if best_fc is not None:
        model_scores['ETS']['rmse'].append(rmse(y_test, best_fc))
        model_scores['ETS']['mae'].append(mae(y_test, best_fc))
        model_scores['ETS']['mape'].append(mape(y_test, best_fc))
        last_preds['ETS'] = pd.Series(best_fc, index=y_test.index)
except Exception:
    pass
```

- **AutoARIMA**

Fits an AutoARIMA model with exogenous features, predicts on test data, and stores RMSE, MAE, and MAPE along with last-fold predictions.

- **ETS**

Tries multiple ETS (Exponential Smoothing) trend/seasonal configurations, selects the best one by RMSE, and logs its metrics and forecasts.

```

# ----- Prophet -----
if HAS_PROPHET:
    try:
        best_fc = None; best_err = np.inf
        for cps, sps in [(0.01,0.1),(0.05,0.5),(0.1,1.0)]:
            try:
                dfp = y_train.reset_index().rename(columns={'Arrival_Date':'ds','Modal_Price':'y'})
                m_prop = Prophet(changepoint_prior_scale=cps, seasonality_prior_scale=sps)
                m_prop.fit(dfp)
                fut = pd.DataFrame({'ds': y_test.index})
                fc = m_prop.predict(fut)['yhat'].values
                s = rmse(y_test, fc)
                if s < best_err:
                    best_err = s; best_fc = fc
            except Exception:
                continue
        if best_fc is not None:
            model_scores['Prophet']['rmse'].append(rmse(y_test, best_fc))
            model_scores['Prophet']['mae'].append(mae(y_test, best_fc))
            model_scores['Prophet']['mape'].append(mape(y_test, best_fc))
            last_preds['Prophet'] = pd.Series(best_fc, index=y_test.index)
    except Exception:
        pass

# ----- ML features -----
X_train_feat, y_train_ml = make_lag_features(y_train, exog_train, LAGS)
if X_train_feat.empty:
    continue
feature_cols = list(X_train_feat.columns)

# ---- XGBoost ----
if HAS_XGB:
    try:
        best_params = optuna_tune("XGBoost", X_train_feat, y_train_ml, n_trials=OPTUNA_TRIALS)
        if best_params:
            model_xgb = XGBRegressor(**best_params, random_state=RANDOM_STATE, verbosity=0, n_jobs=-1)
        else:
            model_xgb = XGBRegressor(random_state=RANDOM_STATE, verbosity=0, n_jobs=-1)
        model_xgb.fit(X_train_feat, y_train_ml)
        fc_series = iterative_forecast_ml(model_xgb, y_train, exog_train, exog_test,
                                         len(y_test), LAGS, feature_cols)
        model_scores['XGBoost']['rmse'].append(rmse(y_test, fc_series))
        model_scores['XGBoost']['mae'].append(mae(y_test, fc_series))
        model_scores['XGBoost']['mape'].append(mape(y_test, fc_series))
        last_preds['XGBoost'] = fc_series
    except Exception:
        pass

```

- **Prophet**

Runs Prophet with different hyperparameter candidates (changepoint_prior_scale, seasonality_prior_scale), picks the best forecast, and stores its metrics.

- **ML Features**

Generates lag and rolling mean features from training data (plus exogenous variables) to use in ML models like XGBoost, RandomForest, and SVR.

- **XGBoost**

Tunes hyperparameters with Optuna if enabled, trains an XGBoost model, performs iterative forecasts, and records metrics and last predictions.

```

# ---- RandomForest ----
try:
    best_params = optuna_tune("RandomForest", X_train_feat, y_train_ml, n_trials=OPTUNA_TRIALS)
    if best_params:
        rf = RandomForestRegressor(**best_params, random_state=RANDOM_STATE, n_jobs=-1)
    else:
        rf = RandomForestRegressor(n_estimators=200, random_state=RANDOM_STATE, n_jobs=-1)
    rf.fit(X_train_feat, y_train_ml)
    fc_series = iterative_forecast_ml(rf, y_train, exog_train, exog_test,
                                     len(y_test), LAGS, feature_cols)
    model_scores['RandomForest']['rmse'].append(rmse(y_test, fc_series))
    model_scores['RandomForest']['mae'].append(mae(y_test, fc_series))
    model_scores['RandomForest']['mape'].append(mape(y_test, fc_series))
    last_preds['RandomForest'] = fc_series
except Exception:
    pass

# ---- SVR ----
try:
    best_params = optuna_tune("SVR", X_train_feat, y_train_ml, n_trials=OPTUNA_TRIALS)
    scaler = StandardScaler().fit(X_train_feat)
    if best_params:
        svr = SVR(**best_params)
    else:
        svr = SVR()
    svr.fit(scaler.transform(X_train_feat), y_train_ml)
    fc_series = iterative_forecast_ml(svr, y_train, exog_train, exog_test,
                                     len(y_test), LAGS, feature_cols, scaler=scaler)
    model_scores['SVR']['rmse'].append(rmse(y_test, fc_series))
    model_scores['SVR']['mae'].append(mae(y_test, fc_series))
    model_scores['SVR']['mape'].append(mape(y_test, fc_series))
    last_preds['SVR'] = fc_series
except Exception:
    pass

```

- **RandomForest**

Tunes hyperparameters with Optuna if enabled, fits a RandomForest model, makes iterative forecasts, and logs RMSE, MAE, MAPE.

- **SVR**

Tunes hyperparameters with Optuna if enabled, scales features with StandardScaler, fits an SVR model, performs iterative forecasts, and stores metrics.

```

# Aggregate CV results
comp_rows = []
for mname, met in model_scores.items():
    if met['rmse']:
        avg_rmse = float(np.nanmean(met['rmse']))
        avg_mae = float(np.nanmean(met['mae']))
        avg_mape = float(np.nanmean(met['mape']))
        best_metric = select_best_metric({'rmse':avg_rmse, 'mae':avg_mae, 'mape':avg_mape})
        comp_rows.append({
            "Market": market,
            "Commodity": commodity,
            "Model": mname,
            "Avg_RMSE": avg_rmse,
            "Avg_MAE": avg_mae,
            "Avg_MAPE": avg_mape,
            "Best_Metric": best_metric
        })

master_partial = pd.DataFrame(comp_rows)
if not master_partial.empty:
    mfn = os.path.join(PARTIAL_DIR, f"master_partial_{uid}.csv")
    master_partial.to_csv(mfn, index=False)
    # pick best
    chosen_metric = master_partial.iloc[0]["Best_Metric"]
    if chosen_metric == "MAE":
        best_row = master_partial.sort_values("Avg_MAE").iloc[[0]]
    elif chosen_metric == "MAPE":
        best_row = master_partial.sort_values("Avg_MAPE").iloc[[0]]
    elif chosen_metric == "Balanced":
        best_row = master_partial.sort_values("Avg_RMSE").iloc[[0]]
    else:
        best_row = master_partial.sort_values("Avg_RMSE").iloc[[0]]
    bfn = os.path.join(PARTIAL_DIR, f"best_partial_{uid}.csv")
    best_row.to_csv(bfn, index=False)

    return {"status":"ok", "market":market, "commodity":commodity}
except Exception as e:
    return {"status":"error", "market":market, "commodity":commodity, "error":str(e)}

```

- **Aggregate CV Results**

Averages metrics across CV folds for each model, determines the best metric using tier logic, saves per-model and best-model summaries, and outputs the chosen winner.


```

def future_forecasting(df_full, best_df, horizon=30, alpha=0.05, n_boot=200):
    forecasts = []
    done_pairs = set()
    if os.path.exists(forecast_ckpt):
        try:
            existing = pd.read_csv(forecast_ckpt)
            done_pairs = set(zip(existing['Market'], existing['Commodity']))
            forecasts.append(existing)
            print(f"Resuming forecast: {len(done_pairs)} pairs already forecasted.")
        except Exception:
            pass

    for _, row in best_df.iterrows():
        market, commodity = row["Market"], row["Commodity"]
        best_model = row.get("Model") or row.get("Best_Model")
        if (market, commodity) in done_pairs:
            continue

        sub_raw = df_full[(df_full['Market'] == market) & (df_full['Commodity'] == commodity)].copy()
        sub_raw['Arrival_Date'] = pd.to_datetime(sub_raw['Arrival_Date'])
        sub_agg = sub_raw.groupby('Arrival_Date').agg({
            'Modal_Price': 'median',
            'Rainfall_mm': 'mean',
            'Avg_Temp_C': 'mean'
        }).asfreq('D').interpolate()

        y = sub_agg['Modal_Price'].dropna()
        if y.empty:
            continue
        exog = sub_agg[['Rainfall_mm', 'Avg_Temp_C']].loc[y.index] if 'Rainfall_mm' in sub_agg.columns else None
        future_exog = pd.DataFrame(index=pd.date_range(start=y.index[-1] + pd.Timedelta(days=1),
            periods=horizon, freq='D'))

        if exog is not None:
            for c in exog.columns:
                future_exog[c] = exog[c].iloc[-7:].mean()

        # 1 hour, 1 month, 1 day, 1 hour, 1 month, 1 day, 1 hour, 1 month, 1 day

```

```

try:
    if best_model == "AutoARIMA" and HAS_PMD:
        m_ar = pm.auto_arima(y, exogenous=exog, seasonal=True, m=7,
            stepwise=True, suppress_warnings=True)
        fc, conf = m_ar.predict(n_periods=horizon, exogenous=future_exog,
            return_conf_int=True, alpha=alpha)
        lower, upper = conf[:,0], conf[:,1]
        fc_index = future_exog.index
    elif best_model == "ETS":
        y_stationary = ensure_stationary(y)
        m_ets = ExponentialSmoothing(y_stationary, trend="add", seasonal="add",
            seasonal_periods=ETS_SEASONAL_PERIODS).fit()
        fc_series = m_ets.forecast(horizon)
        fc, fc_index = fc_series.values, fc_series.index
        fitted = m_ets.fittedvalues
        residuals = (y - fitted).dropna().values
        lower, upper = bootstrap_intervals(fc, residuals, horizon, n_boot, alpha)
    elif best_model == "Prophet" and HAS_PROPHET:
        dfp = y.reset_index().rename(columns={"Arrival_Date": "ds", "Modal_Price": "y"})
        mp = Prophet(interval_width=1-alpha).fit(dfp)
        fut = pd.DataFrame({"ds": future_exog.index})
        pred = mp.predict(fut)
        fc, lower, upper = pred["yhat"].values, pred["yhat_lower"].values, pred["yhat_upper"].values
        fc_index = pd.DatetimeIndex(pred["ds"].values)
    else:
        # load saved artifact if exists
        art_path = _find_artifact_for_pair(market, commodity, best_model)
        model_obj = None; scaler = None; feature_cols = None
        if art_path and os.path.exists(art_path):
            art = joblib.load(art_path)
            if isinstance(art, dict):
                model_obj = art.get('model')
                scaler = art.get('scaler')
                feature_cols = art.get('feature_cols')
        X_feat, y_ml = make_lag_features(y, exog, LAGS)
        if feature_cols is None:
            feature_cols = list(X_feat.columns)
        if model_obj is None:
            if best_model == 'XGBoost' and HAS_XGB:
                model_obj = XGBRegressor(random_state=RANDOM_STATE).fit(X_feat, y_ml)
            elif best_model == 'RandomForest':
                model_obj = RandomForestRegressor(n_estimators=200, random_state=RANDOM_STATE).fit(X_feat, y_ml)
            elif best_model == 'SVR':
                scaler = StandardScaler().fit(X_feat)
                model_obj = SVR().fit(scaler.transform(X_feat), y_ml)
        if model_obj is not None:
            fc_series = iterative_forecast_ml(model_obj, y, exog, future_exog,

```

```

        fc_series = iterative_forecast_ml(model_obj, y, exog, future_exog,
                                         horizon, LAGS, feature_cols, scaler=scaler)
        fc, fc_index = fc_series.values, fc_series.index
        fitted = model_obj.predict(scaler.transform(X_feat) if scaler else X_feat)
        residuals = y_ml.values - fitted
        lower, upper = bootstrap_intervals(fc, residuals, horizon, n_boot, alpha)

    if fc is not None and fc_index is not None:
        tmp = pd.DataFrame({
            "Date": fc_index,
            "Market": market,
            "Commodity": commodity,
            "Forecast": fc,
            "Lower_CI": list(lower) if lower is not None else [np.nan]*len(fc),
            "Upper_CI": list(upper) if upper is not None else [np.nan]*len(fc),
            "Model": best_model
        })
        forecasts.append(tmp)
        pd.concat(forecasts, ignore_index=True).to_csv(forecast_ckpt, index=False)
    except Exception as e:
        print(f"Forecast failed for {market}-{commodity}: {e}")

if forecasts:
    out = pd.concat(forecasts, ignore_index=True)
    out.to_csv(os.path.join(OUTPUT_DIR, "future_forecast.csv"), index=False)
    return out
else:
    return pd.DataFrame()

```

- Loads previous forecast checkpoints (resume-safe), loops through best models per market-commodity, prepares input data, and sets up future exogenous variables.
- Depending on the best model (Sarimax, ETS, Prophet, or ML models), trains/loads the model, generates forecasts, and calculates confidence intervals via bootstrap or model CI.
- Builds the forecast DataFrame with predictions and intervals, saves checkpoints, appends results, and finally writes the full future_forecast.csv.

```

# MAIN
# -----
def main_parallel_full(csv_path=CSV_PATH, n_workers=N_WORKERS, auto_merge_every=AUTO_MERGE EVERY):
    if not os.path.exists(csv_path):
        raise FileNotFoundError(f"CSV not found at {csv_path}")
    df = pd.read_csv(csv_path)
    df = clean_full_df(df)

    pairs_df = df.groupby(['Market', 'Commodity']).size().reset_index(name='count')
    pairs = pairs_df[['Market', 'Commodity']].to_records(index=False).tolist()
    total_pairs = len(pairs)

    done_pairs = set()
    if os.path.exists(BEST_FINAL):
        try:
            best_done = pd.read_csv(BEST_FINAL)
            done_pairs = set(zip(best_done['Market'], best_done['Commodity']))
            print(f"Resuming run: {len(done_pairs)} pairs already completed.")
        except Exception:
            pass

    pairs_to_run = [(m, c, df) for (m, c) in pairs if (m, c) not in done_pairs]
    print(f"Total pairs: {total_pairs} | Remaining: {len(pairs_to_run)} | Workers: {n_workers}")

    start_time = time.time()
    completed = 0
    failed = 0

    if pairs_to_run:
        with mp.Pool(processes=n_workers) as pool:
            for res in pool.imap_unordered(run_pair_worker, pairs_to_run):
                completed += 1
                elapsed = time.time() - start_time
                avg_time = elapsed / completed if completed > 0 else 0
                remaining = len(pairs_to_run) - completed
                eta_minutes = (remaining * avg_time) / 60.0
                if isinstance(res, dict) and res.get("status") == "error":
                    failed += 1
                print(f"✅ Completed {completed}/{len(pairs_to_run)} this run | ETA: {eta_minutes:.1f} min | Failed: {failed}")
                if auto_merge_every and completed % auto_merge_every == 0:
                    merge_partials(PARTIAL_DIR, MASTER_FINAL, BEST_FINAL)

    master_all, best_all = merge_partials(PARTIAL_DIR, MASTER_FINAL, BEST_FINAL)
    if not best_all.empty:
        future_forecasting(df, best_all, horizon=HORIZON, alpha=ALPHA, n_boot=BOOTSTRAP_N)
    return master_all, best_all

```

- **main_parallel_full** runs the whole pipeline: loads data, distributes pairs to workers, merges results, and calls forecasting.

Model Comparison Results

- The cross-validated error metrics (RMSE, MAE, and MAPE) revealed that forecasting accuracy varied considerably across commodities, markets, and models. A consistent observation was that no single approach dominated across the full dataset. Instead, the most appropriate model depended heavily on the specific commodity and the level of market volatility. This validated the pipeline's design: by not committing to a single method, it enabled each market-commodity pair to be evaluated independently and assigned the best-fit model.
- For relatively stable cereals such as **Wheat**, statistical models consistently produced low errors. In many markets, AutoARIMA achieved the smallest RMSE and correspondingly low MAE and MAPE values by capturing the strong autoregressive patterns in wheat prices. For example, in Dound, AutoARIMA reduced errors substantially (RMSE ~70, MAE ~55, MAPE under 6%), outperforming other methods. In contrast, in Baramati, Random Forest provided better accuracy (RMSE ~48, MAE ~36, MAPE ~4.5%), showing that machine learning could capture nonlinearities even in stable series. This comparison demonstrated that while wheat prices follow stable seasonal cycles, localized trade dynamics could shift which model minimized RMSE, MAE, and MAPE most effectively.
- In the case of **Onion**, a commodity well known for price volatility, the results were less uniform. Onions often experience sudden spikes due to weather shocks or supply disruptions, which inflated both MAE and MAPE despite reasonable RMSE scores. For instance, in Hingna, Random Forest delivered the lowest RMSE (~192) with MAE around 150 and MAPE close to 12%. Meanwhile, in other markets Prophet and XGBoost showed better balance, with slightly higher RMSE but lower MAPE, meaning their forecasts were relatively more stable in percentage terms. These differences underscored the value of a multi-model

pipeline: Prophet was more robust where seasonality prevailed, while Random Forest and XGBoost were better suited where abrupt price jumps dominated.

- For **leafy vegetables such as Coriander (Leaves)**, volatility created even larger errors. In Hingna, XGBoost emerged as the best performer, yet still had high error scores (RMSE > 660, MAE ~520, MAPE > 20%), far above levels observed for wheat or onion. This highlighted the extreme unpredictability of coriander prices, where sharp swings limited forecast accuracy across all metrics. Similarly, **Green Peas** produced some of the highest error rates overall, with RMSEs averaging above 1,000, MAE above 800, and MAPE often exceeding 25%. These outcomes indicated that perishability, irregular arrivals, and local shocks make certain commodities inherently difficult to predict, regardless of method.
- Other vegetables such as **Cabbage and Carrot** showed intermediate predictability. For Cabbage in Hingna, ETS produced the lowest RMSE (~119) along with reasonably low MAE (~95) and MAPE (~9%), suggesting that seasonal statistical models can still be effective where cycles are well defined. Carrot, however, remained harder to forecast, with RMSE often above 300, MAE around 250, and MAPE beyond 15%. These results pointed to commodity-specific structures: cabbage prices are governed by clearer seasonal cycles, whereas carrots are more erratic, likely due to perishability and inconsistent arrivals.

Market	Commodity	Best_Model	RMSE	MAE	MAPE
Baramati	Wheat	RandomForest	47.59978	37.97913	1.678555
Dound	Wheat	AutoARIMA	70.00178	55.85334	2.540549
Hingna	Cabbage	ETS	119.3641	95.23877	7.730931
Hingna	Coriander(Leaves)	XGBoost	669.1503	533.9047	13.67027
Hingna	Onion	RandomForest	192.0345	153.2214	6.204273
Hingna	Potato	AutoARIMA	165.1239	131.7498	6.639646
Hingna	Tomato	XGBoost	391.7355	312.5597	17.30446
Junnar	Onion	XGBoost	181.4514	144.7772	8.34087
Junnar(Alephata)	Onion	SVR	181.9376	145.1652	8.306104
Junnar(Narayangaon)	Cabbage	Prophet	143.8579	114.782	13.62424
Junnar(Narayangaon)	Carrot	RandomForest	205.239	163.757	10.20693
Junnar(Narayangaon)	Coriander(Leaves)	RandomForest	436.4942	348.272	21.7737
Junnar(Narayangaon)	Onion	Prophet	129.2007	103.0872	6.709857
Junnar(Narayangaon)	Potato	AutoARIMA	299.1214	238.6643	16.59064
Junnar(Narayangaon)	Tomato	SVR	404.7143	322.9153	23.55993
Junnar(Otur)	Cabbage	Prophet	142.058	113.3459	14.60111
Junnar(Otur)	Carrot	ETS	351.0657	280.1099	22.44131
Junnar(Otur)	Green Peas	Prophet	634.8575	506.543	8.894897
Junnar(Otur)	Onion	ETS	157.6201	125.7626	6.797247
Junnar(Otur)	Potato	AutoARIMA	219.4013	175.0569	14.43504
Kalmeshwar	Cabbage	XGBoost	105.4554	84.14125	8.124798
Kalmeshwar	Coriander(Leaves)	AutoARIMA	627.3381	500.5434	13.80205
Kalmeshwar	Tomato	Prophet	278.582	222.2763	13.22094
Kamthi	Cabbage	Prophet	195.8744	156.2852	11.1647
Kamthi	Carrot	ETS	246.3041	196.5222	8.081657

Kamthi	Coriander(Leaves)	RandomForest	660.6132	527.0931	14.35675
Kamthi	Green Peas	SVR	786.9393	627.8867	22.06983
Kamthi	Onion	AutoARIMA	382.9891	305.5811	13.67316
Kamthi	Potato	AutoARIMA	204.7469	163.3644	8.410182
Kamthi	Tomato	XGBoost	295.3082	235.6218	13.96839
Katol	Wheat	AutoARIMA	65.07913	51.92563	2.449115
Khed	Cabbage	Prophet	99.55098	79.43019	9.93537
Khed	Coriander(Leaves)	Prophet	296.7152	236.7445	99.35092
Khed(Chakan)	Cabbage	Prophet	552.0349	440.4601	50.27929
Khed(Chakan)	Carrot	RandomForest	651.8137	520.0721	27.14292
Khed(Chakan)	Coriander(Leaves)	SVR	461.1871	367.9741	44.33422
Khed(Chakan)	Onion	SVR	182.8044	145.8568	8.824509
Khed(Chakan)	Potato	SVR	132.3115	105.5693	6.480408
Khed(Chakan)	Tomato	ETS	426.0844	339.9662	21.86917
Manchar	Cabbage	Prophet	160.8339	128.3269	16.56803
Manchar	Carrot	XGBoost	294.435	234.9251	17.19832
Manchar	Coriander(Leaves)	AutoARIMA	495.4259	395.2927	15.40325
Manchar	Onion	XGBoost	247.9072	197.8013	11.97865
Manchar	Potato	SVR	266.5216	212.6535	17.73739
Manchar	Tomato	ETS	438.8945	350.1872	18.66222
Mumbai	Cabbage	Prophet	162.7045	129.8194	11.04378
Mumbai	Carrot	AutoARIMA	295.6807	235.9191	9.653996
Mumbai	Coriander(Leaves)	ETS	234.4115	187.0333	11.75654
Mumbai	Green Peas	Prophet	3973.365	3170.287	36.70559
Mumbai	Tomato	ETS	223.0303	177.9524	8.495682
Mumbai	Wheat	SVR	57.85279	46.15985	1.265101
Nagpur	Cabbage	Prophet	146.348	116.7688	13.17995
Nagpur	Carrot	Prophet	412.8433	329.4013	19.23645
Nagpur	Coriander(Leaves)	ETS	966.7439	771.3501	25.72627
Nagpur	Onion	ETS	108.0947	86.24708	4.286225
Nagpur	Potato	AutoARIMA	67.09758	53.53612	3.39814
Nagpur	Tomato	Prophet	483.8123	386.0264	20.50851
Nagpur	Wheat	RandomForest	237.7249	189.6771	6.80828
Pune	Cabbage	Prophet	101.1271	80.68778	8.975155
Pune	Carrot	Prophet	150.915	120.4128	7.422512
Pune	Coriander(Leaves)	SVR	2.933644	2.340709	29.93487
Pune	Green Peas	AutoARIMA	213.2942	170.1842	1.668541
Pune	Onion	SVR	130.8494	104.4027	6.903229
Pune	Potato	ETS	88.85571	70.8966	4.129981
Pune	Tomato	XGBoost	398.5439	317.9921	23.88305
Pune	Wheat	SVR	39.60597	31.60099	0.712031
Pune(Khadiki)	Cabbage	ETS	127.0124	101.3412	12.19225
Pune(Khadiki)	Carrot	Prophet	100.9213	80.52354	5.723607
Pune(Khadiki)	Coriander(Leaves)	AutoARIMA	2.668456	2.12912	26.53668

Pune(Khadiki)	Onion	XGBoost	120.8796	96.44796	6.465131
Pune(Khadiki)	Potato	RandomForest	93.71431	74.7732	5.231835
Pune(Khadiki)	Tomato	RandomForest	143.4202	114.4328	9.385603
Pune(Manjri)	Cabbage	Prophet	215.8111	172.1923	20.53594
Pune(Manjri)	Coriander(Leaves)	Prophet	3.984041	3.178805	46.65058
Pune(Manjri)	Onion	RandomForest	280.5238	223.8256	14.14413
Pune(Manjri)	Potato	AutoARIMA	191.2624	152.6053	8.501949
Pune(Manjri)	Tomato	SVR	558.4507	445.5792	27.11851
Pune(Moshi)	Cabbage	Prophet	166.4113	132.777	15.30557
Pune(Moshi)	Carrot	Prophet	538.8215	429.9174	20.57782
Pune(Moshi)	Coriander(Leaves)	AutoARIMA	3.405624	2.717295	29.06541
Pune(Moshi)	Onion	SVR	171.4951	136.8333	11.55414
Pune(Moshi)	Potato	AutoARIMA	113.6684	90.69425	7.033593
Pune(Moshi)	Tomato	RandomForest	453.2854	361.6694	27.42866
Pune(Pimpri)	Cabbage	Prophet	169.5474	135.2792	11.78317
Pune(Pimpri)	Coriander(Leaves)	SVR	1.644952	1.312482	15.3789
Pune(Pimpri)	Onion	ETS	241.7603	192.8968	11.03593
Pune(Pimpri)	Tomato	XGBoost	217.5577	173.5859	10.80528
Ramtek	Cabbage	Prophet	201.7877	161.0033	18.1143
Ramtek	Coriander(Leaves)	XGBoost	712.8929	568.8062	16.80444
Ramtek	Onion	ETS	97.51597	77.80649	4.015381
Ramtek	Potato	ETS	71.99576	57.4443	3.194226
Ramtek	Tomato	Prophet	478.2346	381.576	27.97736
Savner	Wheat	AutoARIMA	90.14968	71.92904	3.320068
Shirur	Wheat	SVR	131.3708	104.8188	5.96792
Umared	Wheat	XGBoost	40.80378	32.55671	1.403153
Vashi New Mumbai	Onion	ETS	142.5774	113.7603	6.171102
Vashi New Mumbai	Potato	AutoARIMA	88.57606	70.67347	4.640556

- From the perspective of model-level evaluation using RMSE, MAE, and MAPE, the results highlighted that **Random Forest** and **AutoARIMA** were most frequently chosen as the best-performing models, though for distinct reasons. Random Forest consistently produced competitive scores across all three error metrics. Its ability to capture nonlinear relationships enabled it to keep RMSE and MAE relatively low while also containing percentage errors (MAPE), making it a strong general-purpose choice across many commodities and markets.
- **AutoARIMA**, on the other hand, excelled where autoregressive patterns were strong, particularly in cereals like wheat. In such cases, it delivered the lowest RMSE alongside stable MAE and MAPE values, reflecting both accurate absolute predictions and consistent percentage reliability.
- **XGBoost**, while not always the most frequently selected, often emerged as the leading performer for highly volatile crops such as tomatoes and coriander. In these contexts, XGBoost achieved favorable balances across RMSE, MAE, and MAPE, especially where lagged price interactions with weather or seasonal factors were influential. This showed that while its RMSE sometimes matched Random Forest, its lower MAPE often indicated more reliable proportional forecasting under extreme volatility.

- **ETS** and **Prophet** demonstrated niche strengths. ETS tended to minimize RMSE and MAE on commodities with strong seasonal cycles, such as cabbage, and achieved comparatively low MAPE where seasonality was stable. Prophet, by contrast, often excelled in onion markets, where cyclical patterns allowed it to maintain reasonable RMSE values and the lowest MAPEs in some cases, showing robustness against irregular shocks.
- Finally, **SVR** appeared less frequently as the top choice but showed value in smaller markets with limited data. In such cases, its errors (RMSE and MAE) were competitive with larger models, while its percentage-based performance (MAPE) remained acceptable despite constraints in training size.

Summary of Average RMSE, MAE & MAPE by Model Type

Best_Model	RMSE_count	RMSE_mean	RMS_E_min	RMS_E_max	MAE_mean	MAE_min	MAE_max	MAPE_%_mean	MAPE_%_min	MAPE_%_max
AutoARIMA	18	199.7239	2.668456	627.3381	159.3566	2.12912	500.5434	10.43125	1.668541	29.06541
ETS	16	252.5832	71.99576	966.7439	201.5322	57.4443	771.3501	11.03663	3.194226	25.72627
Prophet	25	397.608	3.984041	3973.365	317.2453	3.178805	3170.287	21.12359	5.723607	99.35092
RandomForest	11	309.3148	47.59978	660.6132	246.7975	37.97913	527.0931	13.12379	1.678555	27.42866
SVR	15	234.0413	1.644952	786.9393	186.7379	1.312482	627.8867	15.34314	0.712031	44.33422
XGBoost	12	306.3434	40.80378	712.8929	244.4267	32.55671	568.8062	12.49557	1.403153	23.88305

- Looking beyond raw numbers, the interpretation of **RMSE, MAE, and MAPE magnitudes** provides critical practical context. For **Onions**, where typical modal prices range between ₹1,000 and ₹3,000 per quintal, an RMSE of ~190 corresponds to an MAE of roughly 150 and a MAPE between 10–15%. This indicates that forecast deviations stay within a manageable band relative to actual price levels, making such predictions operationally useful for procurement planning and short-term decision-making.
- In contrast, for **Green Peas**, error magnitudes were far more severe. With RMSE values often in the thousands, MAE exceeding 800, and MAPE surpassing 25%, forecast deviations represented a substantial fraction of the commodity's price. Such large absolute and percentage errors rendered predictions unreliable for guiding day-to-day market strategies.
- This comparison highlights that forecast **usability is inherently commodity-specific**. In some cases, such as onions or cabbage, RMSE, MAE, and MAPE values remain within tolerable limits, enabling forecasts to provide actionable insights. In others particularly perishable and volatile commodities like green peas or coriander the errors are too high across all three metrics, underscoring the limits of even advanced modeling.

Commodity	count	mean	min	max
-----------	-------	------	-----	-----

Cabbage	15	173.86608760128928	99.55098254274276	552.0348909749181
Carrot	10	324.80393350460554	100.92128766590396	651.8137126260928
Coriander(Leaves)	13	373.92855250686085	2.6684558838682637	966.7439275565572
Green Peas	4	1402.1140767792795	213.2942165103466	3973.3653363899225
Onion	13	189.82980780216388	108.09468577354562	382.9891108533184
Potato	11	167.43863171568398	67.09757994437967	299.1213759348558
Tomato	12	374.655152084099	143.42020822028366	558.4507242704735
Wheat	6	86.31072790606146	39.60596574340966	237.72492983041653

- In conclusion, the model comparison results reinforced three critical findings.
- First, there is no universal “best” model; performance depends on commodity-specific characteristics and local market dynamics.
- Second, forecast difficulty varies significantly: cereals and onions are moderately predictable, cabbage and carrot moderately difficult, and commodities like coriander and green peas extremely challenging.
- Third, RMSE/MAE/MAPE interpretations in absolute terms must be grounded in commodity price scales what is a tolerable error for onions may be unacceptable for peas.

These insights justify the pipeline’s adaptive design and provide stakeholders with a realistic understanding of forecasting accuracy.

Future Forecasting

Once the model comparison stage was complete and the best-performing algorithm for each commodity market pair had been identified, we moved to the most practical and impactful step of the pipeline: generating forecasts for the future. The core goal of this stage was to produce thirty-day ahead predictions of modal prices, complete with uncertainty estimates, in a format that could be communicated effectively to stakeholders such as farmers, traders, and policymakers.

Retraining of Best Models

For each commodity–market pair, the winning model identified during cross-validation was retrained on the entire historical dataset. Retraining was necessary to ensure that the model incorporated all available information up to the most recent date, maximizing its predictive strength. For instance, if XGBoost emerged as the best model for onions in a particular market, it was re-fitted using all historical onion price data and the corresponding weather variables for that market. This retraining process guaranteed that the model’s parameters were optimized for the full history rather than just the subsets used in validation.

Statistical models such as AutoARIMA, ETS, and Prophet naturally supported this retraining step by recalculating seasonal and trend components on the full dataset. Machine learning models such as

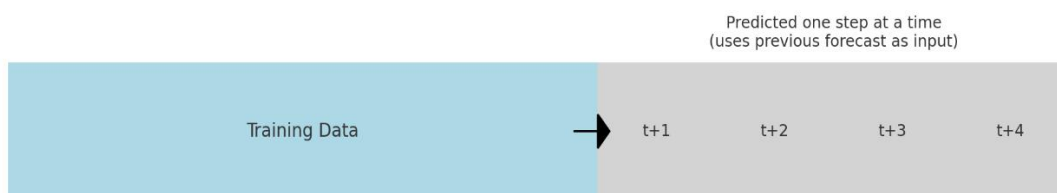
Random Forest, XGBoost, and SVR were retrained using the engineered lag features, rolling averages, and exogenous weather regressors derived from the complete dataset.

Forecast Horizon and Iterative Prediction

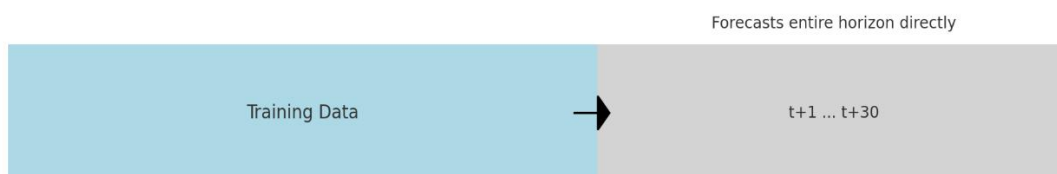
The target forecast horizon was thirty days. For statistical models like AutoARIMA, ETS, and Prophet, this was straightforward: the models directly produced a thirty-day forward projection. However, for machine learning models, forecasts were generated iteratively. The process began by predicting the price for day $t+1$ using features derived from the training set. This predicted value was then appended to the series, lagged features were recomputed, and the model predicted day $t+2$. Repeating this procedure allowed forecasts to roll forward until the thirty-day horizon was completed.

While iterative forecasting carries the risk of error propagation, it enabled machine learning models to operate in a genuine forecasting context. To mitigate compounding errors, we tested stability across folds during validation, ensuring that the models chosen for forecasting did not deteriorate sharply as the horizon extended.

Iterative Forecasting (Machine Learning Models)



Direct Multi-Step Forecasting (Statistical Models)



Confidence Intervals

Forecasts are incomplete without a measure of uncertainty. Point estimates provide a central trajectory, but stakeholders require information on the possible range of outcomes to make informed decisions. Confidence intervals (CIs) were therefore incorporated into every forecast.

For AutoARIMA, ETS, and Prophet, confidence intervals are generated natively based on the models' statistical assumptions about residual variance. These intervals reflect both estimation uncertainty and forecast horizon uncertainty, widening as predictions move further into the future.

For machine learning models, confidence intervals were constructed using a bootstrap-based residual simulation approach. After fitting the model, we collected its residuals from the training period and resampled them with replacement. By adding these resampled residuals to the model's point forecasts and repeating the process across many bootstrap iterations, we generated a distribution of possible outcomes for each future date. The 5th and 95th percentiles of this distribution were used as lower and upper confidence bounds, respectively. This approach provided a practical approximation of uncertainty, even though the underlying models were not probabilistic.

Forecast Outputs

The outputs of this forecasting stage were stored in a structured format for downstream use. A consolidated CSV file contained the following columns:

- Date
- Market
- Commodity
- Forecasted price (point estimate)
- Lower confidence interval (5th percentile)
- Upper confidence interval (95th percentile)
- Model name used for the forecast

This output format ensured interoperability with dashboards, decision-support tools, and further research. For example, forecasts could be easily visualized in a web interface, filtered by commodity or market, or combined with additional indicators such as rainfall projections.

Date	Market	Commodity	Forecast	Lower_CI	Upper_CI	Model
01-07-2025	Baramati	Wheat	2565.441	2539.705	2587.591	RandomForest
30-06-2025	Dound	Wheat	2479.084	2448.88	2509.287	AutoARIMA
01-07-2025	Junnar(Nar ayangaon)	Onion	1306.354	658.1029	2035.179	Prophet
27-07-2025	Junnar(Nar ayangaon)	Tomato	3285.533	2800.07	4709.708	SVR
10-07-2025	Kamthi	Tomato	1965.283	1327.243	2925.633	XGBoost

Conclusion

Agriculture in Maharashtra runs on uncertainty. Farmers work hard to grow crops, but when prices swing wildly from one week to the next, it often feels like their future is out of their control. This project set out to change that, at least in part, by showing that price forecasting can give farmers and other stakeholders a clearer picture of what lies ahead.

We tested a wide range of models—from traditional statistical methods like ARIMA and SARIMAX to modern machine learning approaches like Random Forest, XGBoost, SVR, and Prophet. What we found is that there's no "one size fits all" solution. Stable crops such as wheat responded well to statistical models, while highly unpredictable vegetables like onions and coriander benefited from the flexibility of machine learning. This balance shows that the best way forward is not to depend on a single method but to build a pipeline that can adapt to the unique behavior of each crop and market.

Along the way, we made sure that our approach wasn't just about chasing accuracy. We used robust validation, automatic hyperparameter tuning, and even tools like SHAP values to understand why models made certain predictions. The aim was always practical: to build forecasts that people can trust, not just numbers that look good on paper.

The real value of this project lies in how these forecasts can be used. For farmers, it can mean knowing whether to sell their produce today or wait a little longer. For policymakers, it can mean preparing early for price spikes or shortages. For traders and consumers, it means greater stability in supply and pricing. In short, forecasting is not just about prediction—it's about empowerment.

While we achieved encouraging results, challenges remain. Some crops, especially perishable ones, are still very hard to predict because of sudden shocks. But this work is a solid step toward building a reliable market intelligence system for Maharashtra. With future improvements—such as adding deep learning models, integrating real-time weather and policy data, and developing user-friendly dashboards—this kind of forecasting could become a daily tool for farmers, helping them make smarter decisions and reducing their financial risks.

In the end, the project shows that data and technology, when applied thoughtfully, can bring a measure of stability to the uncertainty of farming. And even a little bit of predictability can make a big difference in the lives of farmers and the communities that depend on them.

Directions for Future Research

This project is just a starting point, and there are a lot of ways this work could be taken further.

- **Expansion to Other Commodities and Regions:** The same framework I used here could be applied to other important crops, like tomatoes, and could be tested in other states to see how well it works elsewhere.
- **Incorporation of Advanced Data Sources:** It would be interesting to try and add new types of data to the model. For example, you could use natural language processing to analyze news articles and social media to get a sense of market sentiment, which might be a powerful predictor.
- **Exploration of Deep Learning Models:** I focused on tree-based machine learning models, but there's a whole other category of models called deep learning models, like LSTMs, that are specifically designed for sequential data. It would be great to see how they compare.
- **Development of a Real-Time System:** The goal would be to turn this research project into a live, real-time forecasting system. This would mean building a pipeline to automatically collect new data, retrain the model regularly, and display the forecasts on a simple dashboard that farmers and others could easily use.

In the end, I believe this project shows that by using modern data science tools, we can make real progress on the difficult but important problem of forecasting agricultural prices, which could ultimately help make farming a more stable and profitable business for the farmers of Maharashtra.

